



Ain Shams University – Faculty of Engineering

Computer Engineering and Systems Program

CSE245: Design and Analysis of Algorithm

Project Report

Team Project – Algorithms Research

Spring 2026

Members

Name	ID
Arsany Hany Anwar	2200655
Andrew Ehab Tharwat	2200249
Hazem Mohamed Salah ElDin	2300774
Maria Karam Albert	2200593
Ziad Ahmed Orabi	2200399
Marten Amgad Zakaria	2200751
Mathew Mina Youssef Aziz	2300910
Hanya Ibrahim Aref	2201045
Julia Nasser Mosaad	2200760

Table of Contents

Members	ii
List of Figures	xviii
List of Tables	xxi
1. Introduction	1
1. Sorting Trominoes	2
1.1 Problem description	2
1.2 Assumptions	2
1.3 Methodology	3
1.3.1 Approach 1: Divide and Conquer	3
1.3.2 Approach 2: Brute Force	3
1.4 Pseudocode	4
1.5 Implementation code	7
1.5.1 Divide and Conquer code	7
1.5.2 Brute Force code	16
1.6 Test cases	23
1.6.1 Output of 4 x 4 board	23
1.6.2 Output of 16 x 16 board	24

1.7 Time and space complexity study	26
1.7.1 Divide and Conquer:	26
1.7.2 Brute Force:.....	26
1.8 Comparative evaluation.....	26
1.9 Alternative approaches.....	28
1.10 Summary	28
2. Knight's Tour Problem	29
2.1 Background	29
2.2 Task statement.....	30
2.3 Implemented features	30
2.4 Comprehensive description	31
2.5 Problem solution.....	32
2.5.1 Methodology.....	32
2.5.2 Pseudocode.....	33
2.5.3 Implementational code.....	34
2.6 Algorithm analysis.....	37
2.7 Answers to task questions	37
2.8 Sample output.....	39
2.9 Alternative solutions.....	42

2.10 Key findings and insights.....	42
3. Introduction	43
3.1 Underlying assumptions	43
3.2 Problem description.....	44
3.2.1 Classical Three-Peg Tower of Hanoi.....	44
3.2.2 The Four-Peg Extension	44
3.2.3 Does DP Solve It in 33 Moves?.....	44
3.3 Algorithmic Techniques	45
3.3.1 Technique 1: Dynamic Programming (Frame-Stewart)	45
3.3.2 Technique 2: Brute-Force Breadth-First Search (BFS)	48
3.3.3 Technique 3: A* search with heuristics	51
3.3.4 Technique 4: Classical Divide and Conquer (Three-Peg Recursion)	53
3.3.5 Technique 5: IDA* (Iterative Deepening A*)	55
3.4 Solution description	58
3.4.1 Complete pseudocode	58
3.4.2 Java implementation.....	60
3.5 Sample outputs and explanations	62
3.5.1 Dynamic Programming table (n = 1 to 12).....	62
3.5.2 Complete 33-move sequence for 8 disks.....	63

3.5.3 Representative console output	65
3.6 Complexity analysis.....	66
3.6.1 Summary complexity table	66
3.7 Comparative evaluation of all techniques	68
3.7.1 Master comparison table.....	68
3.7.2 Failure analysis table.....	69
3.7.3 Conclusion.....	70
3.8 Key findings and insights.....	71
4. Knight Swap.....	72
4.1 Problem description.....	72
4.1.1 What is the Knight Swap Puzzle?	72
4.1.2 Initial and goal states	72
4.1.3 Movement rules.....	73
4.1.4 Challenges	73
4.2 Assumptions.....	74
4.3 Solution methodology.....	75
4.3.1 Algorithm choice: BFS	75
4.3.2 State representation	75
4.3.3 Methodology.....	75

4.3.4 Data structures used	76
4.4 Pseudocode	77
4.5 Implementation code	80
4.5.1 Constants and board definitions	80
4.5.2 BoardState struct	80
4.5.3 findAllNextMoves()	81
4.5.4 solveWithBFS()	82
4.6 Time and space complexity analysis	84
4.6.1 State space size	84
4.6.2 Time complexity	84
4.6.3 Space complexity	85
4.7 Comparative evaluation of techniques	86
4.8 Sample output	87
4.9 Key findings and insights	91
5. Moving Target	92
5.1 Problem description	92
5.2 Assumptions	92
5.3 Methodology	93
5.3.1 Key insight	93

5.3.2 Step-by-Step description	93
5.4 Pseudocode.....	94
5.5 Implementation code.....	95
5.6 Test cases	97
5.6.1 Test cases where the technique succeeds.....	97
5.6.2 Test Cases Where the Technique Fails	98
5.7 Complexity analysis.....	100
5.7.1 Time complexity.....	100
5.7.2 Space complexity	100
5.8 Alternative Approaches	102
5.8.1 Decrease and Conquer.....	102
5.8.2 Random Shooting.....	102
5.8.3 Brute Force / Exhaustive Search	102
5.8.4 Advantages of Divide and Conquer over other techniques.....	103
5.8.5 Disadvantages of Divide and Conquer	103
5.8.6 Possible improvements to the Divide-and-Conquer technique	103
5.9 Sample outputs	105
5.9.1 Sample output for $n = 4$	105
5.9.2 Sample output for $n = 5$	105

5.9.3 Sample Output for $n = 8$	106
5.10 Summary and key findings.....	107
6. Lattice Crossing	108
6.1 Abstract.....	108
6.2 Assumptions.....	109
6.2.1 Lattice definition	109
6.2.2 Segment rules	109
6.2.3 Connected-Path constraint.....	109
6.2.4 Problem scope	110
6.2.5 Candidate Endpoint Set (DP Approach).....	110
6.2.6 Coordinate Convention.....	110
6.3. Problem Description	111
6.3.1 Formal Statement	111
6.3.2 Origin and Motivation.....	111
6.3.3 $2n-2$ lines.....	111
6.4. Technique 1: Dynamic Programming with Backtracking	113
6.4.1 Abstract.....	113
6.4.2 Methodology.....	113
6.4.3 Pseudocode.....	114

6.4.4 Implementation code (C++)	115
6.4.5 Advantages of DP + Backtracking	125
6.4.6 Disadvantages of DP + Backtracking.....	125
6.4.7 Failure cases and enhancements.....	126
6.5 Technique 2: Geometric Spiral Construction.....	127
6.5.1 Abstract.....	127
6.5.2 Methodology.....	127
6.5.3 Pseudocode.....	128
6.5.4 Implementation code (C++)	130
6.5.5 Advantages.....	139
6.5.6 Disadvantages	139
6.5.7 Failure cases.....	139
6.6 Time and space complexity analysis	140
6.6.1 DP + Backtracking.....	140
6.6.2 Geometric Construction.....	141
6.7 Alternative techniques.....	142
6.7.1 Pure Backtracking	142
6.7.2 Greedy algorithm	142
6.7.3 Breadth-First Search (BFS)	143

6.7.4 Recursive Frame Decomposition	143
6.8 Comparative Evaluation of Techniques	144
6.9 Sample output and explanations	145
6.9.1 DP Solution for $n = 3$	145
6.9.2 DP Solution for $n = 4$	145
6.9.3 Geometric solution for $n = 5$ (Odd).....	146
6.9.4 Geometric solution for $n = 4$ (Even).....	147
6.10 Summary and key findings.....	148
7. Job Scheduling.....	149
7.1 Problem description.....	149
7.2 Assumptions.....	149
7.3 Brute-Force Algorithm	150
7.3.1 Methodology.....	150
7.3.2 Step-by-Step description	150
7.3.3 Pseudocode.....	151
7.3.4 Implementation	152
7.3.5 Test cases	155
7.3.6 Complexity analysis.....	157
7.4. Divide and Conquer algorithm	158

7.4.1 Methodology.....	158
7.4.2 Step-by-Step Description	158
7.4.3 Pseudocode.....	159
7.4.4 Implementation	161
7.4.5 Test Cases.....	165
7.4.6 Complexity Analysis	167
7.5 Dynamic Programming algorithm.....	168
7.5.1 Methodology.....	168
7.5.2 Step-by-Step description	168
7.5.3 Pseudocode.....	169
7.5.4 Implementation	171
7.5.5 Test cases	176
7.5.6 Complexity analysis.....	178
7.6 Technique comparison.....	179
7.7 Alternative approaches.....	180
7.7.1 Greedy – Earliest Deadline First (EDF)	180
7.7.2 Branch-and-Bound	180
7.7.3 Approximation algorithms	180
7.8 Summary	181

8. Minimum Cut	182
8.1 Assumptions.....	182
8.1.1 Graph model assumptions	182
8.1.2 Weight assumptions	182
8.1.3 Cut definition assumption.....	182
8.1.4 Objective function assumptions	183
8.1.5 Input/Output and indexing assumptions.....	183
8.1.6 Algorithmic scope assumptions	184
8.1.7 Randomization and reproducibility assumptions	184
8.2 Problem description.....	185
8.2.1 Problem statement	185
8.2.2 Challenges	185
8.2.3 Required output.....	186
8.3 Methodology and solutions	187
8.4 Brute Force.....	187
8.4.1 Step-by-Step methodology	187
8.4.2 Brute Force pseudocode	188
8.4.3 Worked example.....	189
8.4.4 Time and space complexity.....	191

8.5 Iterative Improvement.....	192
8.5.1 Solution representation	192
8.5.2 Neighborhood structure	192
8.5.3 Delta for flipping one vertex.....	193
8.5.4 Best-Improvement strategy.....	194
8.5.5 Multi-Start strategy.....	194
8.5.6 Step-by-Step methodology (Iterative Improvement + Multi-Start)	194
8.5.7 Iterative Improvement pseudocode	195
8.5.8 Worked example.....	196
8.5.9 Notes on limitations.....	201
8.5.10 Time and space complexity.....	201
8.6 Implementation details.....	202
8.6.1 Implementation overview.....	202
8.6.2 Data structures	202
8.6.3 Helper functions.....	203
8.6.4 Implementation code.....	203
8.6.5 Notes	213
8.7 Comparative evaluation.....	214
8.7.1 Evaluation criteria	214

8.7.2 Experimental setup	214
8.7.3 Comparative results on sample graphs	215
8.7.4 Interpretation and scalability discussion	216
8.8 Alternative approaches	217
8.8.1 Exact Global Minimum Cut in Undirected Graphs (Stoer–Wagner)	217
8.8.2 Exact s–t Minimum Cut via Maximum Flow (Max-Flow Min-Cut)	217
8.8.3 Randomized Algorithms for Min Cut (Alternative Exact/Probabilistic Perspective) .	218
8.8.4 Stronger Heuristics than Basic Local Improvement (Escaping Local Minima)	218
8.9 Sample input and output	219
9. Cluster Points into Clusters	221
9.1 Problem description	221
9.2 Assumptions	221
9.3 Methodology	222
9.3.1 Brute-Force	222
9.3.2 Divide and Conquer	222
9.3.3 Iterative Improvement	223
9.4 Pseudocode	224
9.4.1 Helper Functions	224
9.4.2 Brute-Force	226

9.4.3 Divide and Conquer	230
9.4.4 Iterative Improvement.....	232
9.5 Code implementation	235
9.5.1 Helper code.....	235
9.5.2 Brute-Force	239
9.5.3 Divide and Conquer	244
9.5.4 Iterative Improvement.....	246
9.6 Test cases	249
9.6.1 Chosen test cases.....	249
9.6.2 Output.....	250
9.7 Complexity analysis.....	252
9.7.1 Brute Force.....	252
9.7.2 Divide and Conquer	253
9.7.3 Iterative Improvement.....	255
9.8 Technique comparison.....	256
9.8.1 Comparative Summary Table.....	256
9.8.2 Deep dive analysis.....	257
9.9 Alternative approaches.....	258
9.9.1 BFR (Bradley-Fayyad-Reina) Algorithm.....	258

9.9.2 CURE (Clustering Using REpresentatives) Algorithm	258
9.10 Summary	259
10. Coin Flipping	260
10.1 Problem description.....	260
10.2 Assumptions.....	260
10.3 Algorithm Design.....	261
10.3.1 Step-by-step Execution:	261
10.3.2 Pseudocode:.....	262
10.3.3 Code Implementation:	263
10.4 Correctness and case coverage	265
10.5 Complexity Analysis	266
10.6 Algorithms Comparative Analysis	267
10.7 Sample outputs	269
10.8 Approach downsides.....	270
10.9 Improvements.....	270
10.10 Conclusion.....	270
11. Research Conclusions.....	271
12. References	272

List of Figures

<i>Figure 1 - Divide and Conquer result (4x4)</i>	23
<i>Figure 2 - Brute Force result (4x4)</i>	24
<i>Figure 3 - Divide and Conquer Result (16x16)</i>	24
<i>Figure 4 - Brute Force result (16x16)</i>	25
<i>Figure 6</i>	39
<i>Figure 7</i>	39
<i>Figure 8</i>	39
<i>Figure 9</i>	40
<i>Figure 10</i>	40
<i>Figure 11</i>	40
<i>Figure 12</i>	40
<i>Figure 13</i>	41
<i>Figure 14</i>	41
<i>Figure 15</i>	41
<i>Figure 16</i>	41
<i>Figure 17 - Pegs of Hanoi with 3 Disks and 4 Pegs</i>	65
<i>Figure 19 - - Pegs of Hanoi with 8 Disks and 4 Pegs</i>	65
<i>Figure 21</i>	87
<i>Figure 22</i>	88
<i>Figure 23</i>	88

Figure 24.....	89
Figure 25.....	90
Figure 26.....	90
Figure 27.....	105
Figure 28 - 2 Output for $N = 5$	105
Figure 29.....	106
Figure 30.....	156
Figure 31.....	157
Figure 32.....	166
Figure 33.....	167
Figure 34.....	177
Figure 35.....	178
Figure 36.....	183
Figure 37 -Example Cut in a weighted graph.....	183
Figure 38.....	190
Figure 39 -Previous example.....	190
Figure 40.....	193
Figure 41.....	198
Figure 42 - Weighted Graph for this example.....	198
Figure 43.....	198
Figure 44 - Initial partition and its cut weight computed from crossing edges.	198

<i>Figure 45</i>	199
<i>Figure 46 - Delta calculation for flipping one vertex using only incident edges.</i>	199
<i>Figure 47</i>	200
<i>Figure 48 -Partition after one improving move, resulting in a smaller cut weight.</i>	200
<i>Figure 49</i>	219
<i>Figure 50</i>	219
<i>Figure 51</i>	220
<i>Figure 52</i>	250
<i>Figure 53</i>	250
<i>Figure 54</i>	251
<i>Figure 55</i>	252
<i>Figure 56</i>	263
<i>Figure 64 Main function</i>	263
<i>Figure 58</i>	264
<i>Figure 66 Backtracking function</i>	264
<i>Figure 67 Test cases</i>	269
<i>Figure 68 Console outputs</i>	269

List of Tables

<i>Table 1</i>	4
<i>Table 2</i>	7
<i>Table 3</i>	16
<i>Table 4 - Knight's Tour cases for all n</i>	37
<i>Table 5 - Knight's Tour sample output</i>	39
<i>Table 6</i>	62
<i>Table 7</i>	63
<i>Table 8</i>	66
<i>Table 9</i>	68
<i>Table 10</i>	69
<i>Table 11</i>	72
<i>Table 12</i>	73
<i>Table 13</i>	74
<i>Table 14</i>	75
<i>Table 15</i>	76
<i>Table 16</i>	84
<i>Table 17</i>	85
<i>Table 18</i>	85
<i>Table 19</i>	86
<i>Table 20</i>	97

<i>Table 21</i>	98
<i>Table 22</i>	100
<i>Table 23</i>	102
<i>Table 24</i>	104
<i>Table 25</i>	116
<i>Table 26</i>	131
<i>Table 27</i>	138
<i>Table 28</i>	140
<i>Table 29</i>	140
<i>Table 30</i>	141
<i>Table 31</i>	144
<i>Table 32</i>	145
<i>Table 33</i>	146
<i>Table 34</i>	146
<i>Table 35</i>	147
<i>Table 36</i>	155
<i>Table 37</i>	156
<i>Table 38</i>	157
<i>Table 39</i>	165
<i>Table 40</i>	166
<i>Table 41</i>	167

<i>Table 42</i>	176
<i>Table 43</i>	177
<i>Table 44</i>	178
<i>Table 45</i>	179
<i>Table 46</i>	215
<i>Table 47</i>	256
<i>Table 48 Algorithmic Comparison</i>	268
<i>Table 49</i>	268

1. Introduction

Algorithm design serves as the fundamental framework for solving computational problems with quantifiable efficiency and precision. This research project provides a comprehensive investigation into ten distinct algorithmic challenges, ranging from classical geometric puzzles to complex optimization tasks. Each problem is evaluated through the application of core algorithmic paradigms, including Divide and Conquer, Dynamic Programming, Greedy algorithms, Brute Force, and Iterative Improvement.

Given that no single technique is universally optimal, this study emphasizes that algorithm selection must be dictated by a problem's specific structure, constraints, and performance requirements. Our methodology follows a rigorous engineering lifecycle: establishing formal problem definitions, devising, and implementing solution strategies, and conducting thorough time and space complexity analyses. For tasks where multiple techniques are applied, comparative evaluations are provided to highlight critical performance trade-offs.

Ultimately, this project illustrates the efficacy of principled algorithmic thinking in addressing a broad spectrum of computational hurdles. It serves to bridge the gap between theoretical analysis and practical implementation, reinforcing the importance of selecting the right tool for every computational challenge.

1. Sorting Trominoes

1.1 Problem description

For this task, I need to take a square grid with exactly one missing square and fill the entire rest of the board using right trominoes (L shaped blocks made of 3 squares). The tricky part isn't just making the pieces fit, but also having to paint each piece using only three colors. The golden rule is that if two trominoes share a border, they cannot be the same color. It was solved using both a Divide-and-Conquer approach and a Brute Force approach with a comparison between them.

1.2 Assumptions

Before writing the code, a few ground rules were set to make sure everything worked properly:

- ❖ Board Size: The classic L tromino puzzle relies on the board being a power of two to divide evenly. So, the solution was structured for a board size of 2^n times 2^n (like 4x4, 8x8, 16x16).
- ❖ The Missing Square: The missing square can be anywhere, with a toggle in the main function to either randomize its placement or force it near the middle for testing.
- ❖ What Adjacent Means: The rules say no two trominoes of the same color can share an edge, it was interpreted as flat edges only. If two pieces of the same color only touch diagonally at the corners, that's perfectly fine.
- ❖ Grid Values: the missing square had a value of -1 (printed as X), 0 for empty spots, and the numbers 1, 2, and 3 to represent the three colors.

1.3 Methodology

1.3.1 Approach 1: Divide and Conquer

Instead of brute forcing the placements, this method relies on the geometry of the grid. Basically, the algorithm splits the main board into four equal quarters. Since there is only one missing tile on the entire board, it's guaranteed to fall into just one of those quadrants. That leaves the other three completely empty.

The main trick to making this work is placing a single L shaped piece right in the exact center of the board. If oriented correctly, it covers exactly one square in each of those three empty quadrants.

Once that center piece is down, each quadrant acts like a smaller version of the original problem, since they all have exactly one missing or occupied square. From there, recursion is used to keep dividing the board down until the function hits a base case of a 2x2 grid. A 2x2 grid with one missing tile is literally just the shape of one L tromino, so the code drops the piece in and returns.

Handling the Colors:

Trying to figure out the colors at the same time as placing the pieces is difficult. To keep the logic clean, the function is split to two phases. The tiling function just assigns a unique ID integer to every piece it places. After the entire board is full, a separate function (`colorBoardRecursive`) runs a pass over the IDs. It tests colors 1 through 3, checks the adjacent neighbors to ensure no rules are broken, and just backtracks if it hits a dead end. This methodology could be subject to further optimization.

1.3.2 Approach 2: Brute Force

The brute force method is essentially just a large guessing and verifying loop. The algorithm scans through the 2D array row by row until it hits a 0, meaning it found an empty spot,

then it tries to drop an L tromino into that space. Since the piece can be rotated, the four possible shapes are tested by the program to discover which one fits.

If a piece fits without going out of bounds or overlapping another tile, the code immediately tries to paint it with the first color. It checks the adjacent tiles, and if that color doesn't break the adjacency rule, it locks it in and recursively moves on to the next empty square. But if it hits a dead end, when no rotation fits or all three colors are already taken by the surrounding pieces, the backtracking starts to take process. It basically reverts the last move, pulls the piece off the board, and tries the next color or rotation until it eventually brute forces a valid combination.

1.4 Pseudocode

Table 1

```
FUNCTION solveDivideAndConquer(board, size, missRow, missCol, currentId):  
  
    IF size == 2:  
  
        Fill the 3 empty squares with currentId  
  
        currentId = currentId + 1  
  
        RETURN  
  
    halfSize = size / 2  
  
    Find the center coordinates of the current board section  
  
    Determine which of the 4 quadrants contains (missRow, missCol)  
  
    Place a tromino (using currentId) in the center to cover the 3 empty  
    quadrants
```

```
currentId = currentId + 1
```

```
solveDivideAndConquer(top_left_quadrant)
```

```
solveDivideAndConquer(top_right_quadrant)
```

```
solveDivideAndConquer(bottom_left_quadrant)
```

```
solveDivideAndConquer(bottom_right_quadrant)
```

```
FUNCTION colorBoard(currentTromino, totalTrominoes):
```

```
    IF currentTromino > totalTrominoes:
```

```
        RETURN True
```

```
    FOR color = 1 TO 3:
```

```
        IF color does not match any adjacent tromino:
```

```
            Assign color to currentTromino
```

```
            IF colorBoard(currentTromino + 1) is True:
```

```
                RETURN True
```

```
            Remove color
```

```
    RETURN False
```

```
FUNCTION solveBruteForce(board):
```

```
    Find the first empty square (r, c)
```

```
    IF no empty squares exist:
```

```
        RETURN True
```

```
    FOR each of the 4 tromino rotations:
```

```
        IF the rotation fits at (r, c):
```

```
            FOR color = 1 TO 3:
```

```
                IF color is valid:
```

```
                    Place the rotated tromino and assign the color
```

```
                    IF solveBruteForce(board) is True:
```

```
                        RETURN True
```

```
                    Remove the tromino
```

```
    RETURN False
```

1.5 Implementation code

1.5.1 Divide and Conquer code

Table 2

```
#include <iostream>

#include <vector>

#include <cmath>

#include <iomanip>

#include <cstdlib>

#include <ctime>

using namespace std;

const int MISSING_SQUARE = -1;

const int EMPTY_SQUARE = 0;

void solveDivideAndConquer(vector<vector<int>>& board, int startRow, int
startCol, int boardSize, int missRow, int missCol, int& currentId) {
```

```

// base case which is 2x2

if (boardSize == 2) {

    for (int i = 0; i < 2; i++) {

        for (int j = 0; j < 2; j++) {

            // fill last 3 squares without missing

            if (!(startRow + i == missRow && startCol + j == missCol)){

                board[startRow + i][startCol + j] = currentId;

            }

        }

    }

    currentId++;

    return;

}

int half = boardSize / 2;

int centerRow = startRow + half;

int centerCol = startCol + half;

int myTrominoId = currentId++;

```

```

// missing square for each quarter

int nextMissRowTL, nextMissColTL;

int nextMissRowTR, nextMissColTR;

int nextMissRowBL, nextMissColBL;

int nextMissRowBR, nextMissColBR;


// top left

if (missRow < centerRow && missCol < centerCol) {

    nextMissRowTL = missRow; nextMissColTL = missCol;

} else {

    board[centerRow - 1][centerCol - 1] = myTrominoId;

    nextMissRowTL = centerRow - 1; nextMissColTL = centerCol - 1;

}


// top right

if (missRow < centerRow && missCol >= centerCol) {

```



```

        nextMissRowTR = missRow; nextMissColTR = missCol;

    } else {

        board[centerRow - 1][centerCol] = myTrominoId;

        nextMissRowTR = centerRow - 1; nextMissColTR = centerCol;

    }


    // bottom left

    if (missRow >= centerRow && missCol < centerCol) {

        nextMissRowBL = missRow; nextMissColBL = missCol;

    } else {

        board[centerRow][centerCol - 1] = myTrominoId;

        nextMissRowBL = centerRow; nextMissColBL = centerCol - 1;

    }


    // bottom right

    if (missRow >= centerRow && missCol >= centerCol) {

        nextMissRowBR = missRow; nextMissColBR = missCol;

    } else {

        board[centerRow][centerCol] = myTrominoId;

        nextMissRowBR = centerRow; nextMissColBR = centerCol;
    }

```

```

    }

    // recursion does the job

    solveDivideAndConquer(board, startRow, startCol, half, nextMissRowTL,
nextMissColTL, currentId);

    solveDivideAndConquer(board, startRow, centerCol, half, nextMissRowTR,
nextMissColTR, currentId);

    solveDivideAndConquer(board, centerRow, startCol, half, nextMissRowBL,
nextMissColBL, currentId);

    solveDivideAndConquer(board, centerRow, centerCol, half, nextMissRowBR,
nextMissColBR, currentId);
}

```

```

bool colorBoardRecursive(vector<vector<int>>& board, vector<int>&
assignedColors, int currentTromino, int totalTrominoes) {

```

```

    if (currentTromino > totalTrominoes) return true;

    for (int color = 1; color <= 3; color++) {

        bool isSafe = true;

        for (int i = 0; i < board.size(); i++) {

            for (int j = 0; j < board.size(); j++) {

                if (board[i][j] == currentTromino) {

                    if ((i > 0 && assignedColors[board[i-1][j]] == color)
||
                        (i < board.size()-1 &&
assignedColors[board[i+1][j]] == color) ||
                        (j > 0 && assignedColors[board[i][j-1]] == color)
||
                        (j < board.size()-1 &&
assignedColors[board[i][j+1]] == color)) {

                        isSafe = false;

                        break;

                    }

                }

            }

        }

        if (!isSafe) break;
    }

```

```

    }

    if (isSafe) {

        assignedColors[currentTromino] = color;

        if (colorBoardRecursive(board, assignedColors, currentTromino +
1, totalTrominoes)) return true;

        assignedColors[currentTromino] = 0;

    }

}

return false;
}

void applyColorsToDC(vector<vector<int>>& board, int maxId) {

    vector<int> assignedColors(maxId + 1, 0);

    colorBoardRecursive(board, assignedColors, 1, maxId);

    for (int i = 0; i < board.size(); i++) {

        for (int j = 0; j < board.size(); j++) {

            if (board[i][j] != MISSING_SQUARE) {

                board[i][j] = assignedColors[board[i][j]];

```

```

        }

    }

}

```

```

void printBoard(const vector<vector<int>>& board) {

    for (const auto& row : board) {

        for (int val : row) {

            if (val == MISSING_SQUARE) cout << setw(4) << "X";

            else cout << setw(4) << val;

        }

        cout << endl;

    }

}

```

```

int main() {

    int n = 4; // change this, 5 will probably not even finish for bf

    int boardSize = pow(2, n);

```

```

int missR, missC;

bool randomizeMissing = true; // set to true to randomize missing coord

if (randomizeMissing) {

    srand(time(0));

    missR = rand() % boardSize;

    missC = rand() % boardSize;

} else {

    // missing close to center

    missR = boardSize / 2 - 1;

    missC = boardSize / 2 - 1;

}

vector<vector<int>> startGrid(boardSize, vector<int>(boardSize,
EMPTY_SQUARE));

startGrid[missR][missC] = MISSING_SQUARE;

// dac

vector<vector<int>> dcBoard = startGrid;

```

```

    int trominoCounter = 1;

    solveDivideAndConquer(dcBoard, 0, 0, boardSize, missR, missC,
trominoCounter);

    applyColorsToDC(dcBoard, trominoCounter - 1);

    cout << "Divide and Conquer Result (" << boardSize << "x" << boardSize
<< ")" << endl;

    printBoard(dcBoard);

    return 0;

}

```

1.5.2 Brute Force code

Table 3

```

#include <iostream>

#include <vector>

#include <cmath>

#include <iomanip>

#include <cstdlib>

```

```
#include <ctime>

using namespace std;

const int MISSING_SQUARE = -1;

const int EMPTY_SQUARE = 0;

// for adjacent color validation

bool isColorValid(const vector<vector<int>>& board, int r, int c, int
colorChoice) {

    int dr[] = {-1, 1, 0, 0};

    int dc[] = {0, 0, -1, 1};

    for (int i = 0; i < 4; i++) {

        int nr = r + dr[i];

        int nc = c + dc[i];
```



```

        if (nr >= 0 && nr < board.size() && nc >= 0 && nc < board.size()) {

            if (board[nr][nc] == colorChoice) return false;

        }

    }

    return true;
}

bool solveBruteForce(vector<vector<int>>& board) {

    int boardSize = board.size();

    int targetRow = -1, targetCol = -1;

    // looking for first empty

    for (int i = 0; i < boardSize; i++) {

        for (int j = 0; j < boardSize; j++) {

            if (board[i][j] == EMPTY_SQUARE) {

                targetRow = i; targetCol = j;

                break;

            }

        }

        if (targetRow != -1) break;
    }
}

```

```

}

if (targetRow == -1) return true;

// tromino orient

int shapes[4][2][2] = {

    {{0, 1}, {1, 0}}, {{0, 1}, {1, 1}},

    {{1, 0}, {1, 1}}, {{1, 0}, {1, -1}}

};

for (int i = 0; i < 4; i++) {

    int r2 = targetRow + shapes[i][0][0], c2 = targetCol +
shapes[i][0][1];

    int r3 = targetRow + shapes[i][1][0], c3 = targetCol +
shapes[i][1][1];

    if (r2 >= 0 && r2 < boardSize && c2 >= 0 && c2 < boardSize &&
board[r2][c2] == EMPTY_SQUARE &&

        r3 >= 0 && r3 < boardSize && c3 >= 0 && c3 < boardSize &&
board[r3][c3] == EMPTY_SQUARE) {

```

```

        // test all 3 colors

        for (int color = 1; color <= 3; color++) {

            if (isColorValid(board, targetRow, targetCol, color) &&

                isColorValid(board, r2, c2, color) &&

                isColorValid(board, r3, c3, color)) {

                board[targetRow][targetCol] = board[r2][c2] =
board[r3][c3] = color;

                if (solveBruteForce(board)) return true;

                board[targetRow][targetCol] = board[r2][c2] =
board[r3][c3] = EMPTY_SQUARE; // backtracking

            }

        }

    }

    return false;
}

```

```

void printBoard(const vector<vector<int>>& board) {

    for (const auto& row : board) {

        for (int val : row) {

            if (val == MISSING_SQUARE) cout << setw(4) << "X";

            else cout << setw(4) << val;

        }

        cout << endl;

    }

}

int main() {

    int n = 4; // change this, 5 will probably not even finish for bf

    int boardSize = pow(2, n);

    int missR, missC;

    bool randomizeMissing = true; // set to true to randomize missing coord

```

```

    if (randomizeMissing) {

        srand(time(0));

        missR = rand() % boardSize;

        missC = rand() % boardSize;

    } else {

        // missing close to center

        missR = boardSize / 2 - 1;

        missC = boardSize / 2 - 1;

    }


    vector<vector<int>> startGrid(boardSize, vector<int>(boardSize,
EMPTY_SQUARE));

    startGrid[missR][missC] = MISSING_SQUARE;


    // bf

    vector<vector<int>> bfBoard = startGrid;

    cout << "Brute Force Result (" << boardSize << "x" << boardSize << ")"
<< endl;

    if (solveBruteForce(bfBoard)) {

        printBoard(bfBoard);
    }

```

```

    } else {

        cout << "No solution found" << endl;

    }

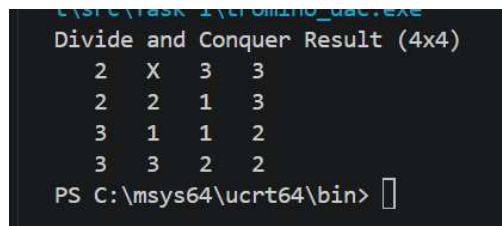
    return 0;
}

```

1.6 Test cases

1.6.1 Output of 4 x 4 board

1.6.1 Divide and Conquer result (4x4)



```

C:\src\task 1\tromino_dac.exe
Divide and Conquer Result (4x4)
 2  X  3  3
 2  2  1  3
 3  1  1  2
 3  3  2  2
PS C:\msys64\ucrt64\bin>

```

Figure 1 - Divide and Conquer result (4x4)

Here, the X is the missing square. The numbers 1, 2, and 3 are the trominoes. If you look closely at the shapes, you'll see that no two L shapes made of the same number ever touch on a flat side.

1.6.2 Brute Force result (4x4)

```

C:\src\task_1\chromino_bt.exe
Brute Force Result (4x4)
  1  1  2  2
  1  3  2  X
  2  3  3  1
  2  2  1  1
PS C:\msys64\ucrt64\bin>

```

Figure 2 - Brute Force result (4x4)

It gets the job done for a small 4x4 grid. But if you change the code to an 8x8, you won't get an output because the program will get stuck in a massive loop of guessing and backtracking.

1.6.2 Output of 16 x 16 board

Divide and Conquer Result (16x16)

```

C:\src\task_1\chromino_dc.exe
Divide and Conquer Result (16x16)
  2  2  3  3  2  2  3  3  2  2  3  3  2  2  3  3
  2  1  1  3  2  1  1  3  2  1  1  3  2  1  1  3
  3  1  2  2  3  3  1  2  3  1  2  2  3  3  1  2
  3  3  2  1  1  3  2  2  3  3  2  1  1  3  2  2
  2  2  3  3  1  2  3  3  2  2  3  3  1  2  3  3
  2  1  1  3  2  2  1  3  2  1  1  3  2  2  1  3
  3  3  1  2  3  1  1  2  3  3  1  2  3  1  1  2
  3  X  2  2  3  3  2  2  1  3  2  2  3  3  2  2
  2  2  3  3  2  2  3  1  1  2  3  3  2  2  3  3
  2  1  1  3  2  1  3  3  2  2  1  3  2  1  1  3
  3  1  2  2  3  1  1  2  3  1  1  2  3  3  1  2
  3  3  2  1  3  3  2  2  3  3  2  2  1  3  2  2
  2  2  3  1  1  2  3  3  2  2  3  1  1  2  3  3
  2  1  3  3  2  2  1  3  2  1  3  3  2  2  1  3
  3  1  1  2  3  1  1  2  3  1  1  2  3  1  1  2
  3  3  2  2  3  3  2  2  3  3  2  2  3  3  2  2
PS C:\msys64\ucrt64\bin>

```

Figure 3 - Divide and Conquer Result (16x16)

Just like the smaller board, the Divide and Conquer algorithm handles the 16x16 grid effortlessly, finishing almost instantly. The X marks the missing square, and the rest of the board is perfectly tiled with the three colors without breaking the adjacency rule. This perfectly

demonstrates how well the recursive geometric approach scales up compared to the brute force method, which struggles heavily at this size.

Brute Force result (16x16)

```
1 1 2 2 1 1 3 3 2 2 1 1 3 3 1 1
1 3 2 3 1 2 3 1 2 3 1 2 3 2 2 1
3 3 1 3 3 2 2 1 1 3 3 2 2 1 2 3
2 2 1 1 2 1 1 2 2 1 1 3 1 1 3 3
2 1 3 3 2 2 1 2 3 1 2 3 3 2 2 1
3 1 1 3 1 3 3 1 3 3 2 2 1 2 1 1
3 3 2 1 1 X 3 1 1 2 3 3 1 1 3 3
1 1 2 2 3 3 2 2 3 2 2 3 2 2 3 1
1 2 1 1 3 1 2 1 3 3 1 1 2 3 1 1
2 2 1 3 1 1 3 1 1 2 1 3 1 3 3 2
1 1 2 3 3 2 3 3 2 2 3 3 1 1 2 2
1 2 2 1 1 2 2 1 1 3 1 1 2 2 3 3
2 3 3 1 2 3 3 1 3 3 1 3 2 1 3 2
2 2 3 2 2 3 2 2 1 1 3 3 1 1 2 2
1 1 2 3 3 1 2 3 1 2 1 1 2 3 3 1
1 2 2 3 1 1 3 3 2 2 1 2 2 3 1 1
```

Figure 4 - Brute Force result (16x16)

The brute force algorithm manages to get the job done for a 16x16 grid, taking up to 30 seconds to find this valid layout. However, this is its absolute limit. If you change the code to a 32x32 board, the program dies and freezes because it gets stuck in a massive loop of guessing and backtracking.

1.7 Time and space complexity study

1.7.1 Divide and Conquer:

- ❖ Time Complexity: It runs in $O(N^2)$ time (where N is the side length of the board). I am basically just visiting and filling every square on the board a set number of times. The coloring phase adds a little bit of backtracking, but since the graph of the board is pretty simple, it finishes almost instantly.
- ❖ Space Complexity: $O(N^2)$ to store the board array, plus $O(\log N)$ overhead for the recursion stack since I keep cutting the board in half.

1.7.2 Brute Force:

- ❖ Time Complexity: Exponential. It's roughly $O(12^T)$ in the worst-case scenario (where T is the number of trominoes). For every single empty spot, the computer might try 4 shapes and 3 colors. As the board gets bigger, the number of guesses becomes huge, and time taken becomes too long and not worth it.
- ❖ Space Complexity: $O(N^2)$ for the board, and $O(N^2)$ for the recursion stack, because the recursion goes as far as the total number of pieces needed to be placed.

1.8 Comparative evaluation

The Divide-and-Conquer approach leverages the underlying mathematical structure of the problem to determine optimal placement analytically. By identifying exact coordinates for each component without the need for estimation, this method achieves high computational efficiency. Empirical testing demonstrates that for board dimensions of 16×16 and 32×32 , the algorithm produces solutions with near-instantaneous latency.

Conversely, the Brute Force (or exhaustive search) method lacks predictive logic, relying instead on systematic trial and error. This approach frequently results in suboptimal configurations where early placements obstruct subsequent moves. While computationally viable for a 4×4 grid, the method exhibits significant performance degradation at the 16×16 scale and is functionally impractical when applied to 32×32 boards or larger; the exponential

increase in "dead-end" combinations leads to computational stagnation, effectively preventing the completion of the task within a reasonable timeframe.

1.9 Alternative approaches

A better approach would be to treat the finished board as a mathematical graph and use a standard algorithm like Welsh Powell to color it.

Even better, there are mathematical formulas that can calculate the exact color of a tromino based on its X and Y coordinates on the board. If those formulas were used, coloring of the pieces would be done during the initial placement skipping the second phase entirely.

1.10 Summary

The primary finding of this study underscores the limited scalability of exhaustive search technique. While simple to implement, the exponential time complexity inherent in brute force approaches renders them impractical for large-scale applications.

In contrast, leveraging the geometric properties of the problem through a Divide-and-Conquer paradigm reduces computational latency from an intractable duration to near-instantaneous execution. This shift emphasizes the critical role of algorithmic design in transforming theoretically solvable tasks into computationally efficient solutions.

2. Knight's Tour Problem

2.1 Background

- A chess knight moves in an “L” shape: two squares in one direction and then one square perpendicular.
- On an $n \times n$ chessboard, a **knight's tour** is a sequence of moves that visits every square exactly once (landing on each square, not just passing over it).
- If the first and last squares of the tour are a knight's move apart, the tour is called **closed** or **re-entrant** (otherwise it is **open**).
- In the context of a closed (re-entrant) knight's tour, the tour must end on a square that is **exactly one** knight's move away from the starting square. This means that if you place a knight on the last square of the tour, it can move directly to the first square in a single legal knight move.
- Warnsdorff's rule is a **heuristic** for finding a knight's tour on a chessboard. It is a greedy algorithm that decides the next move at each step based on the number of **onward moves** available from each candidate square.
- In **graph theory**, a **Hamiltonian path** is a path in a graph that visits every vertex exactly once.
A **Hamiltonian circuit** (or **Hamiltonian cycle**) is a Hamiltonian path that starts and ends at the same vertex, forming a cycle.
- **Greedy algorithm** is a problem-solving approach that makes the **locally optimal choice** at each step, hoping that these local optima will lead to a global optimum solution.

2.2 Task statement

1. **8×8 board:** Is it possible for a knight to visit all 64 squares of a standard chessboard exactly once, ending on a square that is one knight's move away from the starting square? Justify your answer.
2. **General $n \times n$ board:**
 1. For which values of n does a closed knight's tour exist?
 2. For all $n > 8$, is a closed tour always possible? If not, describe the cases where it is impossible and explain why.
3. **Greedy algorithm:** Design a **greedy algorithm** that determine the **minimum number of moves** the knight needs to visit all squares exactly once.
4. Study of different n

2.3 Implemented features

- Get input from the user with the n rows or columns of the board.
- Get the start square from the user.
- Run greedy algorithm to find whether there is a closed Tour.
- Display the result and the minimum number of moves the chess knight needs.

2.4 Comprehensive description

The knight's tour is a classic mathematical and computational puzzle that asks whether a chess knight can visit every square of a given chessboard exactly once, moving according to the standard rules of chess. A knight moves in an "L" shape: two squares in one direction (horizontally or vertically) and then one square perpendicular, or vice versa. The knight does not jump over pieces, but the board is initially empty, only the knight's movement restrictions matter.

Two types of tours are considered:

1. **Open tour:** The knight visits all squares exactly once, ending on any square different from the starting square. The tour does not need to return to the start.
2. **Closed tour (re-entrant):** The knight visits all squares exactly once and the final square is exactly one knight's move away from the starting square, so the tour could be repeated indefinitely.

The problem can be modelled as a graph problem:

- Each square of the $n \times n$ board is a **vertex** (node).
- A legal knight move between two squares is an **edge**.
- An open knight's tour corresponds to a **Hamiltonian path** in this graph.
- A closed knight's tour corresponds to a **Hamiltonian circuit** (cycle).

2.5 Problem solution

2.5.1 Methodology

- At each step, from the current square, the knight examines all unvisited squares reachable by a legal knight move. For each such candidate square, it counts how many unvisited squares are reachable from that candidate (its *degree* in the remaining graph). The knight then moves to the candidate with the **smallest** such count.
- **Reasoning:** Squares with few onward moves are most “constrained” and should be visited early to avoid becoming dead ends later. This local greedy choice almost always leads to a complete tour.
- A closed (re-entrant) tour requires the first and last squares to be a knight’s move apart.
- **For high possibility creating closed Tour Start from a square with an even degree.**
The degree of a square is defined as the number of legal knight moves from that square on an empty board, the number of squares a knight can reach in one move, ignoring whether those squares have been visited. For example, on an 8×8 board, a corner has degree 2 (even), a square one step from a corner has degree 3 (odd), and a central square has degree 8 (even).

2.5.2 Pseudocode

ALGORITHM

//Finds a closed knight's tour on an $n \times n$ board.

//Input: An integer $n \geq 5$ (board dimension)

//Output: An $n \times n$ array with visit order, or "No solution"

create $board[n][n]$ initialized to -1 // $-1 = \text{unvisited}$

//lists all 8 possible L-shaped knight jumps

$moves \leftarrow [(2,1),(1,2),(-1,2),(-2,1),(-2,-1),(-1,-2),(1,-2),(2,-1)]$

// Before moving to any square, this checks three things: is (x, y) still inside the board, and has it not been visited yet

function $isValid(x, y)$:

return $(0 \leq x < n)$ **AND** $(0 \leq y < n)$ **AND** $(board[x][y] = -1)$

// counts how many moves the knight could make from candidate square to other unvisited squares

function $countOnward(x, y)$:

$count \leftarrow 0$

for each (dx, dy) **in** $moves$ **do**

if $isValid(x + dx, y + dy)$: $count \leftarrow count + 1$

return $count$

// Start from $(0, 0)$ — any position is valid

$x \leftarrow 0$; $y \leftarrow 0$

$board[x][y] \leftarrow 0$

for $step \leftarrow 1$ **to** $n \times n - 1$ **do**

$minDegree \leftarrow \infty$

$nextX \leftarrow -1$; $nextY \leftarrow -1$

// Try all possible knight moves

for each (dx, dy) **in** $moves$ **do**

$nx \leftarrow x + dx$; $ny \leftarrow y + dy$

if $isValid(nx, ny)$ **do**

$degree \leftarrow countOnward(nx, ny)$

if $degree < minDegree$

$minDegree \leftarrow degree$; $nextX \leftarrow nx$; $nextY \leftarrow ny$

// No valid move found — fail

if $nextX = -1$ **return** "No solution"

// Advance knight

$x \leftarrow nextX$; $y \leftarrow nextY$

$board[x][y] \leftarrow step$

2.5.3 Implementational code

```
#include <iostream>
using namespace std;

int n;
int board[100][100];

int dx[8] = {2, 1, -1, -2, -2, -1, 1, 2};
int dy[8] = {1, 2, 2, 1, -1, -2, -2, -1};
```

Figure 5- Knight's Tour Includes & global variables

- n is the board size entered by the user.
- board[100][100] is declared globally so every function can access it without passing it around. It supports boards up to 100×100.
- dx and dy are the 8 knight move offsets. Each index i represents one L-shaped jump, where dx[i] is the row shift and dy[i] is the column shift.

```
10 bool isValid(int x, int y) {
11     return (x >= 0 && x < n && y >= 0 && y < n && board[x][y] == -1);
12 }
```

Figure 6 - Knight's Tour Validation function

Checks three things at once before allowing a move to (x, y):

- x >= 0 && x < n — row is inside the board
- y >= 0 && y < n — column is inside the board
- board[x][y] == -1 — square has not been visited yet
- All three must be true. If anyone fails, the move is rejected.

```

14  int countOnward(int x, int y) {
15      int count = 0;
16      for (int i = 0; i < 8; i++) {
17          if (isValid(x + dx[i], y + dy[i]))
18              count++;
19      }
20      return count;
21  }

```

Figure 7 - Knight's Tour Degree calculation

The core of the Algorithm is implemented in the *countOnward* function. For a given square (x, y) , the function determines the number of unvisited squares reachable by a single knight move.

It iterates over all eight possible move offsets and counts those that land within the board boundaries and on an unvisited cell. A small count indicates that the square has few remaining onward movements, making it a candidate that should be visited promptly to avoid becoming isolated and causing the tour to fail.

```

23 int GreedyAlgo(int startX, int startY) {
24     for (int i = 0; i < n; i++)
25         for (int j = 0; j < n; j++)
26             board[i][j] = -1;
27
28     int x = startX;
29     int y = startY;
30     board[x][y] = 0;
31
32     int moves = 0;
33
34     for (int step = 1; step < n * n; step++) {
35
36         int minDegree = 9;
37         int nextX = -1;
38         int nextY = -1;
39
40         for (int i = 0; i < 8; i++) {
41             int nx = x + dx[i];
42             int ny = y + dy[i];
43
44             if (isValid(nx, ny)) {
45                 int degree = countOnward(nx, ny);
46                 if (degree < minDegree) {
47                     minDegree = degree;
48                     nextX = nx;
49                     nextY = ny;
50                 }
51             }
52         }
53
54         if (nextX == -1)
55             return moves;
56
57         x = nextX;
58         y = nextY;
59         board[x][y] = step;
60         moves++;
61     }
62     return moves;
63 }

```

Figure 9 - Knight's Tour Greedy tour construction

- The initialization phase begins with a double loop that resets every cell of the board to -1, indicating an unvisited state. The knight is then placed at the user-specified starting square, which is marked with 0 to denote the first visited square. A counter “moves” records the number of successfully completed moves.
- The main loop executes exactly $n \times n - 1$ iterations, as the first square is already occupied. The variable **minDegree** is initialized to 9 because the maximum possible number of onward moves from any square is 8; thus, any valid candidate will immediately yield a smaller value.
- The **variables nextX** and **nextY** are set to -1 to serve as a sentinel indicating that no valid move has yet been found.
- Within each iteration, the algorithm examines all eight possible knight jumps from the current position. For each neighbor that lies inside the board and remains unvisited, the function **countOnward** computes its degree (the number of unvisited squares reachable from that neighbor).
- If this degree is lower than the current minimum, the candidate coordinates are stored in **nextX** and **nextY**. After evaluating all eight directions, (**nextX**, **nextY**) refers to the most constrained reachable square.
- If **nextX** remains -1 after the loop, no valid move exists; the knight is stuck, and the function returns the current value of moves (a partial tour). Otherwise, the knight moves to the selected square, records the step number in the board array, and increments moves. Upon completing a full tour, moves will equal $n \times n - 1$.

2.6 Algorithm analysis

- Time: $O(n^2)$ – each of the n^2 steps examine at most 8 neighbors and counts their onward moves (each also ≤ 8 checks).
- Space: $O(n^2)$ for the visited matrix and tour storage.

2.7 Answers to task questions

1) Yes — a closed knight's tour on 8×8 exists

2) **Closed tours exist for all even $n \geq 6$.** They are provably impossible for all odd n (parity argument) and for $n \leq 4$ (small-board arguments). So every $n > 8$ that is even supports a closed tour, while every odd $n > 8$ supports only an open tour.

3) Greedy algorithm is designed above

4) **Cases for all n :**

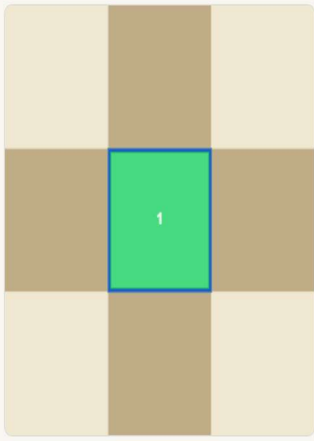
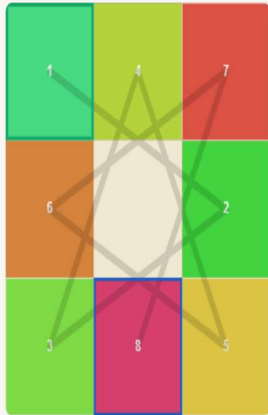
Table 4 - Knight's Tour cases for all n

n	Open tour	Closed tour	Min moves
2	X	X	—
3	✓	X	7
4	✓	X	15
5	✓	X	24
6	✓	✓	36
7	✓	X	48

8	✓	✓	64
9	✓	X	80
10	✓	✓	100
≥5 odd	✓	X	n^2-1
≥6 even	✓	✓	n^2

2.8 Sample output

Table 5 - Knight's Tour sample output

Test Case	Sample Output	Visualization (AI generated)
1	"C:\Users\zyado\OneDrive\Desktop\ Enter board size n: 3 Enter start row (0 to 2): 0 Enter start col (0 to 2): 0 Closed Tour: NO Minimum Moves: 7	 Theory — $n = 3$ ~ partial Open tour X Closed tour The center cell (1,1) has zero knight-reachable neighbours on a 3x3 board, so it can never be visited. Maximum reachable: 8/9 cells. Warnsdorff's result cells visited 1/9 min moves 0 Partial — 1 of 9 cells visited Animation speed slow -- -- fast Figure 5
2	Enter board size n: 3 Enter start row (0 to 2): 0 Enter start col (0 to 2): 0 Closed Tour: NO Minimum Moves: 7 Figure 6	Board size n = 3 Run Step Reset  Theory — $n = 3$ ~ partial Open tour X Closed tour The center cell (1,1) has zero knight-reachable neighbours on a 3x3 board, so it can never be visited. Maximum reachable: 8/9 cells. Warnsdorff's result cells visited 8/9 min moves 7 Partial — 8 of 9 cells visited Animation speed slow -- -- fast Figure 7

3

```

Enter board size n: 5
Enter start row (0 to 4): 1
Enter start col (0 to 4): 1
Closed Tour: NO
Minimum Moves: 24

```

Figure 8

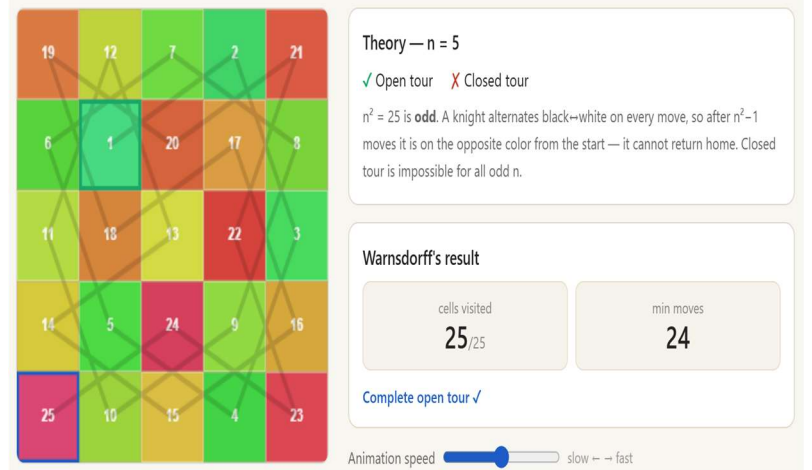


Figure 9

4

```

Enter board size n: 6
Enter start row (0 to 5): 0
Enter start col (0 to 5): 0
Closed Tour: YES
Minimum Moves: 35

```

Figure 10



Figure 11

5

```

Enter board size n: 7
Enter start row (0 to 6): 0
Enter start col (0 to 6): 0
Closed Tour: NO
Minimum Moves: 48

```

Figure 12

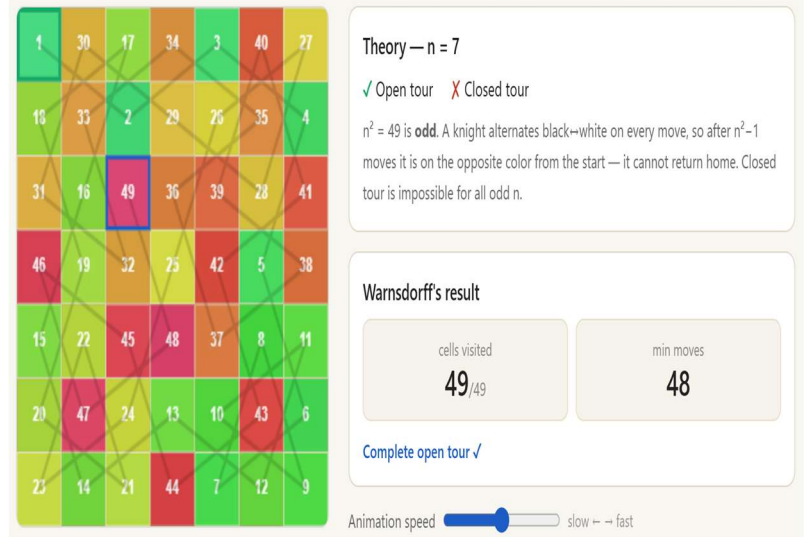


Figure 13

6

```

Enter board size n: 8
Enter start row (0 to 7): 0
Enter start col (0 to 7): 0
Closed Tour: YES
Minimum Moves: 63

```

Figure 14

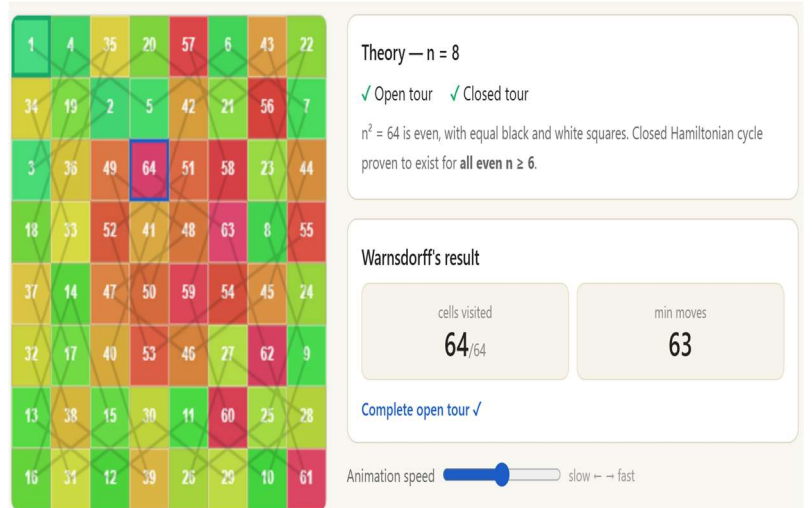


Figure 15

2.9 Alternative solutions

Max-Degree Heuristic (MDH)

Selects the node with the maximum onward degree, aiming to maintain high mobility. This, in turn, delays low-degree nodes, leading to late dead-ends, which is the opposite of Warnsdorff.

Edge-Avoidance Heuristic (EAH)

Avoids low-degree nodes (edges/corners) as an implicit degree-based strategy. This reduces early trapping but is less precise than explicit degree calculation.

Random Greedy (RG)

Makes a random choice among all valid moves at each step. This avoids deterministic traps but offers no guarantee and is unstable; often used with multiple retries.

Hybrid Greedy (HG – Best Practical)

Combines Warnsdorff as the primary rule with tie. This improves robustness, reduces failure cases, and is near-optimal in practice.

2.10 Key findings and insights

- The knight's tour can be modelled as a graph traversal problem, specifically that of finding a Hamiltonian path or cycle on the graph induced by the chessboard.
- **Closed tours require even $n \geq 6$** because odd boards have unequal black/white squares which disables the knight from returning to its start color, making closed tours mathematically impossible for all odd n .

3. Introduction

The Tower of Hanoi is one of the most celebrated problems in computer science and discrete mathematics. Originally conceived by the French mathematician Edouard Lucas in 1883, the classical three-peg variant has a closed-form solution requiring exactly $2^n - 1$ moves for n disks. Adding a fourth peg transforms the problem into a rich optimization challenge whose exact solution remained unproven for decades.

The primary algorithm implemented in Java is the Frame-Stewart Dynamic Programming approach, which produces the best known and verified-optimal solution of 33 moves for eight disks.

3.1 Underlying assumptions

The following assumptions govern all analysis and implementations in this report:

- Four pegs are labelled A (source), B and C (auxiliaries), and D (destination).
- Eight disks of strictly distinct sizes; the solution generalises to any n .
- Initially, all disks reside on peg A, largest at bottom, smallest at top.
- Objective: transfer all disks to peg D preserving the size-ordering invariant.
- Only one disk may be moved at a time; a move takes the topmost disk from one peg to another.
- A larger disk must never rest on top of a smaller disk (no-overlap constraint).
- The Frame-Stewart conjecture is accepted as producing the minimum move count; verified computationally for $n \leq 30$ (Bousch, 2014).
- All algorithms are sequential; no parallelism is considered.

3.2 Problem description

3.2.1 Classical Three-Peg Tower of Hanoi

With three pegs and n disks, the divide-and-conquer strategy recursively moves $n-1$ disks to an auxiliary peg, transfers the largest disk to the destination, then moves $n-1$ disks from the auxiliary to the destination. This yields $T(n,3) = 2^n - 1$. For eight disks: 255 moves.

3.2.2 The Four-Peg Extension

With four pegs, the Frame-Stewart algorithm introduces an optimized split: move k disks to a staging peg using all four pegs, transfer the remaining $n-k$ disks to the destination using only three pegs (with the staging peg frozen), then bring the k staged disks to the destination using all four pegs again. The recurrence is:

$$T(n,4) = \min_{k \in [1,n-1]} \{ 2 \cdot T(k,4) + T(n-k,3) \} \quad \text{where} \quad T(n,3) = 2^n - 1$$

3.2.3 Does DP Solve It in 33 Moves?

Yes. The DP table yields optimal split $k = 4$ for $n = 8$. The decomposition is: move disks 1–4 from A to B using 4 pegs (9 moves), move disks 5–8 from A to D using 3 pegs with B frozen (15 moves), move disks 1–4 from B to D using 4 pegs (9 moves). Total: $9 + 15 + 9 = 33$ moves, matching the DP prediction exactly.

3.3 Algorithmic Techniques

This section analyses every applicable technique across seven dimensions: definition, mechanism, advantages, disadvantages, success/failure cases, enhancement opportunities, and complexity.

3.3.1 Technique 1: Dynamic Programming (Frame-Stewart)

Dynamic Programming (DP) is an algorithm design paradigm that solves complex optimization problems by decomposing them into overlapping sub-problems, solving each sub-problem exactly once, and storing results in a table to avoid redundant computation. It is applicable when the problem exhibits the principle of optimality and overlapping sub-problems.

For four-peg Tower of Hanoi, DP is applied through the Frame-Stewart algorithm (Frame & Stewart, 1941): a bottom-up tabulation that finds the optimal disk-split k for every disk count from 1 to n .

The algorithm has two distinct phases. First, the DP pre-computation phase fills the table $dp4[0..n]$. For each disk count from 2 to $maxDisks$, every split point k from 1 to $n-1$ is evaluated. The candidate cost is $2*dp4[k] + (2^{(n-k)} - 1)$, which represents: two four-peg sub-problems of size k plus one three-peg sub-problem of size $n-k$.

The minimum candidate is stored alongside the optimal k . Second, the move generation phase uses the pre-computed split table to drive recursion: `hanoi4` calls `hanoi4` for the top- k sub-problem, `hanoi3` for the locked middle sub-problem, and `hanoi4` again for the top- k sub-problem.

```
PROCEDURE BuildDPTable(maxN) :  
    dp4[0]:=0; dp4[1]:=1; split[1]:=1  
    FOR n:=2 TO maxN DO  
        dp4[n]:=INF  
        FOR k:=1 TO n-1 DO  
            candidate := 2*dp4[k] + (2^(n-k) - 1)
```

```

    IF candidate < dp4[n] THEN
        dp4[n]:=candidate; split[n]:=k
PROCEDURE Hanoi4(n, src, dst, aux1, aux2):
    IF n=0 THEN RETURN
    IF n=1 THEN OUTPUT Move(src,dst); RETURN
    k := split[n]
    Hanoi4(k, src, aux1, dst, aux2) // park top-k disks on aux1
    Hanoi3(n-k, src, dst, aux2)      // shift n-k disks (3 pegs only)
    Hanoi4(k, aux1, dst, src, aux2) // finalize top-k to destination

```

Advantages:

- Achieves the minimum known move count for all n (conjectured optimal, verified $n \leq 30$).
- Pre-computation is $O(n^2)$; entirely negligible for practical n .
 - Outer loop: n iterations
 - Inner loop: up to n iterations checking k values
 - Total: $\Sigma(n) = O(n^2)$
 - For any practically solvable n (e.g., $n \leq 1000$), this is trivial
- Total space is $O(n)$: two arrays plus a bounded recursion stack.
 - `dp4[0..n]`: $O(n)$ space
 - `split[1..n]`: $O(n)$ space
 - Recursion stack: depth $\leq n$ (but typically much less due to the split strategy)
 - Total = $O(n)$
- Clean two-phase design separates table building from move generation.
- Naturally generalises to any number of pegs $p \geq 4$ by extending the recurrence.

Disadvantages:

- Optimality is conjectural, not formally proven for all n (though no counterexample exists).
- Deep recursion for very large n could overflow the call stack; iterative implementation avoids this.
- Slightly more complex than a simple three-peg recursion; requires understanding the DP formulation.

Success and failure analysis

Success Cases: Frame-Stewart DP succeeds for all tested values of n (1 through 30), always producing the minimum known move count. For $n=8$, it produces exactly 33 moves verified against the DP prediction.

Failure Cases: No practical failure case exists. The only theoretical failure would be a counterexample to the Frame-Stewart conjecture, something that has never been found in over 80 years of investigation.

Can Failure be Enhanced or Avoided?

- Stack overflow (large n): Replace recursive `hanoi4` and `hanoi3` with iterative versions using an explicit stack. This eliminates the depth limit entirely.
- Potential conjecture failure: If a counterexample to Frame-Stewart is discovered, a BFS or IDA* verification pass could be added to confirm optimality for any specific n .

Note: The conjecture has held for all verified $n \leq 30$, so this is a theoretical precaution.

- Extension to $p > 4$ pegs: Apply the generalised recurrence $T(n,p) = \min \{ 2T(k,p) + T(n-k,p-1) \}$, handled by an additional DP loop over p .

Time and Space Complexity

- Pre-computation: Time $O(n^2)$, Space $O(n)$.
- Move generation: Time $O(T(n,4)) \approx O(n \cdot 2^{(n/2)})$, Space $O(n)$ recursion stack.
- Can complexity be enhanced? The $O(n^2)$ pre-computation could be reduced to $O(n \log n)$ by exploiting monotonicity of optimal k , but this is a negligible improvement in practice. Move generation cannot be improved below $O(T(n,4))$ since every move must be output.

3.3.2 Technique 2: Brute-Force Breadth-First Search (BFS)

Breadth-First Search (BFS) is a complete graph traversal algorithm that explores all states reachable from a starting state in order of increasing path length (number of moves). Applied to Tower of Hanoi, each configuration of disks on pegs is a node in a state graph. BFS guarantees that the first time the goal state is reached, it is via the shortest possible path, yielding an optimal solution.

Each configuration is encoded as a state vector: an array of length n where $state[i]$ = the peg holding disk i . The BFS explores every state reachable from the initial configuration, layer by layer, maintaining a visited set to avoid cycles. When the goal configuration (all disks on peg D) is reached, the predecessor map is used to reconstruct the move sequence.

```
PROCEDURE BFS(n, start, goal):  
    queue := {start};  visited := {start};  parent := {}  
    WHILE queue is not empty DO  
        state := queue.dequeue()  
        IF state = goal THEN RETURN reconstruct(parent, state)  
        FOR each valid move m from state DO  
            next := applyMove(state, m)  
            IF next NOT IN visited THEN  
                visited.add(next);  parent[next] := (state, m);  queue.enqueue(next)  
    RETURN failure // should not occur for valid inputs
```

Advantages:

- Guaranteed optimal: always finds the minimum number of moves.
- Conceptually simple: no problem-specific mathematical insight required.
- Complete: will always find a solution if one exists.
- Can be used to verify solutions produced by other algorithms (as a correctness oracle for small n).

Disadvantages

- Catastrophic memory: the state space has size 4^n . For $n=8$: 65,536 states; for $n=20$: over 10^{12} states.
- Exponential time: $O(4^n \cdot n)$ to process all states with n -length vector comparisons.
- Impractical for $n > 13$ on standard hardware with typical memory constraints.
- Provides no move sequence insight; it simply finds the shortest path mechanically.

Success and failure analysis:

Success Cases: BFS succeeds for small n (≤ 12 comfortably, ≤ 15 with careful memory management). For $n=8$, it would correctly find 33 moves but consume roughly $65,536 \times 8$ bytes = ~ 500 KB of state memory plus substantial queue overhead.

Failure Cases: BFS fails for $n \geq 14$ due to memory exhaustion on typical machines. For $n=20$, storing $4^{20} \approx 10^{12}$ states is physically impossible. Failure is total and universal for large n , not recoverable without algorithm replacement.

Improvements:

- Bidirectional BFS: Search simultaneously from start and goal, meeting in the middle. Reduces effective state space to $O(2 \cdot 4^{(n/2)})$ in the best case; approximately halves the search depth.
- State compression: Encode each configuration as a single integer (2 bits per disk $\times$ n disks). Reduces memory by a constant factor but does not change asymptotic complexity.
- Switch to IDA*: Uses $O(n)$ space at the cost of more computation time. The most practical replacement for BFS when $n > 13$.
- Symmetry pruning: The four pegs can be permuted in $4! = 24$ ways; exploiting symmetry can reduce the effective state space by up to 24x.

Time and space complexity:

- Time: $O(4^n \cdot n)$ — exponential in n .
- Space: $O(4^n \cdot n)$ — must store the entire visited frontier.
- Can complexity be enhanced? Bidirectional BFS improves to $O(4^{n/2} \cdot n)$ in time and space, reducing the exponent by half. IDA* reduces space to $O(n)$ but does not improve worst-case time. No approach eliminates the exponential growth; Frame-Stewart DP is the correct algorithm for large n .

3.3.3 Technique 3: A* search with heuristics

A* is an informed graph search algorithm that uses a heuristic function $h(s)$ to estimate the remaining cost from any state 's' to the goal. It maintains a priority queue ordered by $f(s) = g(s) + h(s)$, where $g(s)$ is the actual cost from the start state to s, and $h(s)$ is the heuristic estimate of the cost from s to the goal. When $h(s)$ is admissible (never overestimates the true remaining cost), A* is guaranteed to find the optimal solution while exploring fewer states than uninformed BFS.

For Tower of Hanoi, an admissible heuristic is: $h(s)$ = number of disks not yet on the destination peg. This is admissible because each disk not on the destination peg requires at least one move to get there. A* explores states in order of $f(s) = \text{moves_so_far} + h(\text{state})$, prioritising states that appear closest to the goal.

```
PROCEDURE AStar(start, goal, h):  
  
  open := priority_queue ordered by f = g + h  
  open.insert(start, f=h(start));  g := {start: 0}  
  
  WHILE open not empty DO  
  
    state := open.extractMin()  
  
    IF state = goal THEN RETURN path  
  
    FOR each successor next of state DO  
  
      tentative_g := g[state] + 1  
  
      IF tentative_g < g.get(next, INF) THEN  
  
        g[next] := tentative_g  
  
        open.insert(next, f = tentative_g + h(next))
```

Advantages:

- Explores fewer states than BFS due to heuristic guidance.
- Guaranteed optimal solution with an admissible heuristic.
- Flexible: stronger heuristics reduce exploration dramatically.
- Widely applicable and well-studied in AI and robotics literature.

Disadvantages:

- Still requires exponential memory for storing the open list in the worst case.
- The best-known admissible heuristics for Tower of Hanoi are weak, offering limited pruning.
- Significantly more complex to implement than Frame-Stewart DP.
- Open list management (priority queue with decrease-key) adds constant overhead per state.

Success and failure analysis

Success Cases: A* succeeds for $n \leq 15$ with adequate memory. The heuristic reduces the number of states explored compared to pure BFS, making $n=8$ tractable with modest resources and correctly finding 33 moves.

Failure Cases: A* fails for $n \geq 16-18$ due to exponential growth of the open list. Even with the best available heuristics for this problem, A* cannot match the polynomial pre-computation of Frame-Stewart DP. Failure is due to memory, not algorithm correctness.

Improvements:

- IDA* (Iterative Deepening A*): Eliminates the open list, using only $O(n)$ space. Achieves optimal solutions for n up to approximately 20. The most practical enhancement.
- Pattern database heuristics: Pre-compute exact costs for sub-configurations of disks and use these as much tighter admissible heuristics. Can reduce explored states by orders of magnitude.
- Bidirectional A*: Search from both the start and goal simultaneously, meeting in the middle. Reduces effective depth from d to $d/2$, with significant speed improvement.

3.3.4 Technique 4: Classical Divide and Conquer (Three-Peg Recursion)

Divide and Conquer (D&C) is an algorithm design paradigm that recursively decomposes a problem into two or more sub-problems of the same type, solves each sub-problem independently, and combines the results.

The classical Tower of Hanoi solution is the canonical D&C example: an n -disk problem is divided into two $(n-1)$ -disk sub-problems plus one base-case move. The sub-problems are structurally identical to the original problem but smaller.

The three-peg recursive procedure is the purest form of divide and conquer applied to Tower of Hanoi. For four-peg boards, the same procedure can be applied while simply ignoring the fourth peg, producing a valid but sub-optimal result.

```
PROCEDURE Hanoi3(n, src, dst, aux):  
    IF n = 0 THEN RETURN  
    Hanoi3(n-1, src, aux, dst)    // Divide: move n-1 disks to aux  
    OUTPUT Move(src, dst)         // Conquer: move disk n to destination  
    Hanoi3(n-1, aux, dst, src)    // Combine: move n-1 from aux to dst  
// Recurrence:  $T(n) = 2T(n-1) + 1$   
// Solution:    $T(n) = 2^n - 1$ 
```

Advantages:

- Extremely simple: a three-line recursive function with immediate comprehensibility.
- Provably correct and optimal for three pegs by mathematical induction.
- Minimal implementation effort; no pre-computation required.
- Serves as the inner engine (hanoi3) inside Frame-Stewart for three-peg sub-problems.

Disadvantages:

- Exponentially sub-optimal when a fourth peg is available.
- For $n=8$: 255 moves vs. 33 moves (648% overhead).

- For $n=12$: 4,095 moves vs. 81 moves (4,959% overhead).
- The inefficiency grows exponentially with n ; completely wasteful of the fourth peg.

Success and failure analysis:

Success Cases: Three-peg D&C always produces a valid solution on a four-peg board by simply ignoring the fourth peg. The transfer completes correctly without any constraint violation, for all n .

Failure Cases: Three-peg D&C fails in move-count optimality for every case $n \geq 3$ where a fourth peg is available. The failure is systematic and universal: it always uses exponentially more moves than necessary. There are no exceptions.

Improvements:

- Natural enhancement: Frame-Stewart DP (Technique 1). Use the same recursive structure but replace the fixed split at $n-1$ with a DP-optimised split at k . This is precisely what Frame-Stewart accomplishes.
- Hybrid: Apply three-peg recursion only as the inner engine for the locked $n-k$ sub-problem inside Frame-Stewart, which is exactly the implemented solution.
- In practice: whenever more than three pegs are available, three-peg D&C should be completely replaced by Frame-Stewart DP. There is no reason to use it as the primary algorithm in a four-peg setting.

Time and space complexity:

- Time: $O(2^n)$ move generation — exponential.
- Space: $O(n)$ recursion stack depth.
- Can complexity be enhanced? Switching to Frame-Stewart DP reduces move count from $O(2^n)$ to $O(n \cdot 2^{(n/2)})$, a dramatic improvement. The $O(n)$ space is already optimal.

3.3.5 Technique 5: IDA* (Iterative Deepening A*)

IDA* (Korf, 1985) combines the memory efficiency of depth-first search with the heuristic guidance of A*. It performs a series of depth-first searches, each bounded by a threshold on the f-value ($f = g + h$). Any path whose f-value exceeds the threshold is pruned. After each iteration, the threshold is increased to the minimum f-value of any pruned node from the previous iteration. IDA* uses $O(d)$ space where d is the solution depth, compared to $O(b^d)$ for A*.

```
PROCEDURE IDAStar(start, goal, h):  
    threshold := h(start)  
  
    LOOP:  
        result := DFS_IDA(start, 0, threshold, h, goal)  
  
        IF result = FOUND THEN RETURN path  
  
        IF result = INF THEN RETURN no_solution  
  
        threshold := result // next threshold = min f exceeding current  
  
PROCEDURE DFS_IDA(state, g, threshold, h, goal):  
    f := g + h(state)  
  
    IF f > threshold THEN RETURN f // prune this branch  
  
    IF state = goal THEN RETURN FOUND  
  
    minExceeded := INF  
  
    FOR each successor next of state DO  
        t := DFS_IDA(next, g+1, threshold, h, goal)  
  
        IF t = FOUND THEN RETURN FOUND  
  
        minExceeded := min(minExceeded, t)  
  
    RETURN minExceeded
```

Advantages:

- Linear space $O(n)$: only the recursion stack is maintained; no open list.
- Optimal: guaranteed to find the minimum move count with an admissible heuristic.

- Avoids the catastrophic memory failure of BFS and A* for moderately large n .
- Simple to implement once the DFS framework is in place.

Disadvantages:

- Revisits states across iterations: the same state may be explored many times with different g -values.
- Sensitive to heuristic quality: a weak heuristic means many iterations with slight threshold increases.
- Still exponential time in the worst case; impractical for $n \geq 20$.
- More complex than Frame-Stewart DP for this specific problem.

Success and failure analysis:

Success Cases: IDA* succeeds for n up to approximately 18–20, producing optimal move counts with only $O(n)$ memory. For $n=8$, it correctly finds 33 moves. It represents the best memory-efficient alternative to Frame-Stewart DP.

Failure Cases: IDA* fails for $n \geq 20$ due to excessive computation time. The DFS must re-explore exponentially growing sub-trees in each iteration. With a weak heuristic such as the simple count of disks not on destination, even $n=12$ may take minutes. Failure is in time, not correctness.

Improvements:

- Pattern database heuristics: Pre-compute exact move counts for small disk sub-sets. A pattern DB heuristic for the top 5 disks can reduce iterations by orders of magnitude.
- Transposition tables: Cache recently evaluated states to avoid redundant re-exploration within a single IDA* iteration. Requires $O(m)$ space for m cached states.
- Parallel IDA*: Different branches of the search tree are explored in parallel threads. Each thread maintains $O(n)$ space; overall system uses $O(n \times \text{threads})$ space.
- For this specific problem: IDA* is outclassed by Frame-Stewart DP in every practical metric. Use IDA* only if Frame-Stewart optimality is in doubt and verification for a specific n is required.

Time and Space Complexity:

- Time: $O((b \cdot T(n,4))^2 / h_quality)$ in the worst case, where $h_quality \in (0,1]$ measures heuristic tightness. With a perfect heuristic ($h_quality=1$): $O(T(n,4))$.
- Space: $O(n)$: linear, the key advantage over A^* .
- Pattern databases can improve practical time dramatically. Space cannot be reduced below $O(n)$ for any correct optimal algorithm that traces the solution path. Frame-Stewart DP achieves $O(n^2)$ pre-computation with $O(n)$ space, making it the gold standard.

3.4 Solution description

3.4.1 Complete pseudocode

DP Table Construction

```
PROCEDURE BuildDPTable(maxDisks):  
  
    dp4[0]:=0; split[0]:=0  
  
    dp4[1]:=1; split[1]:=1  
  
    FOR n:=2 TO maxDisks DO  
  
        dp4[n]:=INFINITY  
  
        FOR k:=1 TO n-1 DO  
  
            candidate := 2*dp4[k] + (2^(n-k) - 1)  
  
            IF candidate < dp4[n] THEN  
  
                dp4[n]:=candidate; split[n]:=k  
  
            END FOR  
  
        END FOR  
  
    END PROCEDURE
```

Four-Peg Recursive Solver (Frame-Stewart)

```
PROCEDURE Hanoi4(n, src, dst, aux1, aux2, print):  
  
    IF n=0 THEN RETURN  
  
    IF n=1 THEN  
  
        moveCount++  
  
        IF print THEN OUTPUT 'Move Disk 1: '+src+' -> '+dst  
  
        RETURN  
  
    k := split[n] // optimal split from DP table  
  
    Hanoi4(k, src, aux1, dst, aux2, p) // Step 1: park top-k on aux1  
  
    Hanoi3(n-k, src, dst, aux2, p) // Step 2: move n-k (3 pegs only)  
  
    Hanoi4(k, aux1, dst, src, aux2, p) // Step 3: move k to destination  
  
END PROCEDURE
```

Classical Three-Peg Solver (Inner Engine)

```
PROCEDURE Hanoi3(n, src, dst, aux, print):  
    IF n=0 THEN RETURN  
    Hanoi3(n-1, src, aux, dst, print)  
    moveCount++  
    IF print THEN OUTPUT 'Move Disk '+n+: '+src+' -> '+dst  
    Hanoi3(n-1, aux, dst, src, print)  
END PROCEDURE
```

3.4.2 Java implementation

```
public static void buildDPTable(int maxDisks) {

    dp4 = new int[maxDisks + 1];    split = new int[maxDisks + 1];

    dp4[0] = 0;

    if (maxDisks >= 1) { dp4[1] = 1; split[1] = 1; }

    for (int n = 2; n <= maxDisks; n++) {

        dp4[n] = Integer.MAX_VALUE;

        for (int k = 1; k < n; k++) {

            int candidate = 2 * dp4[k] + ((1 << (n - k)) - 1);

            if (candidate < dp4[n]) { dp4[n]=candidate; split[n]=k; }

        }

    }

}

public static void hanoi4(int n, String src, String dst,

                           String aux1, String aux2, boolean print) {

    if (n == 0) return;

    if (n == 1) {

        moveCount++;

        if (print) System.out.printf("  Move %3d: Disk %d | %s --> %s%n",

                                      moveCount, n, src, dst);

        return;

    }

    int k = split[n];

    hanoi4(k, src, aux1, dst, aux2, print); // Step 1

    hanoi3(n - k, src, dst, aux2, print);    // Step 2

    hanoi4(k, aux1, dst, src, aux2, print);  // Step 3

}

public static void hanoi3(int n, String src, String dst,
```

```

        String aux, boolean print) {

    if (n == 0) return;

    hanoi3(n - 1, src, aux, dst, print);

    moveCount++;

    if (print) System.out.printf("  Move %3d: Disk %d | %s --> %s%n",
                                moveCount, n, src, dst);

    hanoi3(n - 1, aux, dst, src, print);

}

```

Step-by-Step walkthrough for 8 disks:

The eight-disk solution decomposes into exactly three top-level calls. $k = \text{split}[8] = 4$.

1. $\text{hanoi4}(4, A, B, D, C)$: Move disks 1–4 from A to B using all 4 pegs. This internally recurses with $\text{split}[4]=2$: move 2 to C (3 moves), move 2 to B (3 moves), move 2 from C to B (3 moves). Total: 9 moves.
2. $\text{hanoi3}(4, A, D, C)$: Move disks 5–8 from A to D using only 3 pegs. Peg B is frozen. $T(4,3) = 2^4 - 1 = 15$ moves.
3. $\text{hanoi4}(4, B, D, A, C)$: Move disks 1–4 from B to D using all 4 pegs. Mirror of step 1. Total: 9 moves.

Grand total: $9 + 15 + 9 = 33$ moves. This exactly matches the DP prediction.

3.5 Sample outputs and explanations

3.5.1 Dynamic Programming table (n = 1 to 12)

Table 1 shows the pre-computed DP results for all disk counts 1 through 12. The highlighted row n = 8 confirms 33 moves, saving 222 moves (87.1%) compared to the three-peg classical algorithm. Savings increase dramatically with n, reaching 98% at n = 12.

Table 6 - Dynamic Programming – Optimal Move Counts (n = 1 to 12 Disks)

n	3-Peg (2^n-1)	4-Peg DP Min	Optimal k	Moves Saved	Saving (%)
1	1	1	1	0	0.0%
2	3	3	1	0	0.0%
3	7	5	1	2	28.6%
4	15	9	1	6	40.0%
5	31	13	2	18	58.1%
6	63	17	3	46	73.0%
7	127	25	3	102	80.3%
8	255	33	4	222	87.1%
9	511	41	5	470	92.0%

n	3-Peg (2^n-1)	4-Peg DP Min	Optimal k	Moves Saved	Saving (%)
10	1023	49	6	974	95.2%
11	2047	65	6	1982	96.8%
12	4095	81	7	4014	98.0%

3.5.2 Complete 33-move sequence for 8 disks

Table 2 lists all 33 moves generated by the Frame-Stewart algorithm for $n = 8$. Moves 1–9 constitute the first four-peg phase (top 4 disks parked on B). Moves 10–24 constitute the three-peg locked phase (bottom 4 disks moved to D with B frozen). Moves 25–33 constitute the second four-peg phase (top 4 disks moved from B to D).

Table 7 - Complete Move Sequence for 8 Disks on 4 Pegs (33 Moves)

#	Disk	From → To	#	Disk	From → To
1	1	Peg A → Peg D	18	5	Peg C → Peg D
2	2	Peg A → Peg B	19	6	Peg C → Peg A
3	3	Peg A → Peg C	20	5	Peg D → Peg A
4	2	Peg B → Peg C	21	7	Peg C → Peg D
5	4	Peg A → Peg B	22	5	Peg A → Peg C

#	Disk	From → To	#	Disk	From → To
6	2	Peg C → Peg A	23	6	Peg A → Peg D
7	3	Peg C → Peg B	24	5	Peg C → Peg D
8	2	Peg A → Peg B	25	1	Peg B → Peg A
9	1	Peg D → Peg B	26	2	Peg B → Peg D
10	5	Peg A → Peg C	27	3	Peg B → Peg C
11	6	Peg A → Peg D	28	2	Peg D → Peg C
12	5	Peg C → Peg D	29	4	Peg B → Peg D
13	7	Peg A → Peg C	30	2	Peg C → Peg B
14	5	Peg D → Peg A	31	3	Peg C → Peg D
15	6	Peg D → Peg C	32	2	Peg B → Peg D
16	5	Peg A → Peg C	33	1	Peg A → Peg D
17	8	Peg A → Peg D			

3.5.3 Representative console output

```
=====
DEMO: 3 Disk(s), 4 Pegs | Pegs: A (src) --> D (dst)
=====

DP predicted minimum moves : 5
Optimal split k           : 1
3-peg comparison (2^n - 1) : 7
-----

Move sequence:
Move  1: Disk 1 | Peg A --> Peg B
Move  2: Disk 2 | Peg A --> Peg C
Move  3: Disk 3 | Peg A --> Peg D
Move  4: Disk 2 | Peg C --> Peg D
Move  5: Disk 1 | Peg B --> Peg D
-----

Actual moves generated: 5 | DP minimum: 5 | Match: YES
Move reduction vs 3-peg : 28.6%
=====
```

Figure 16 - Pegs of Hanoi with 3 Disks and 4 Pegs

```
=====
DEMO: 8 Disk(s), 4 Pegs | Pegs: A (src) --> D (dst)
=====

DP predicted minimum moves : 33
Optimal split k           : 4
3-peg comparison (2^n - 1) : 255
-----

Move sequence:
Move  1: Disk 1 | Peg A --> Peg D
Move  2: Disk 2 | Peg A --> Peg B
Move  3: Disk 3 | Peg A --> Peg C
Move  4: Disk 2 | Peg B --> Peg C
Move  5: Disk 4 | Peg A --> Peg B
Move  6: Disk 2 | Peg C --> Peg A
Move  7: Disk 3 | Peg C --> Peg B
Move  8: Disk 2 | Peg A --> Peg B
Move  9: Disk 1 | Peg D --> Peg B
Move 10: Disk 5 | Peg A --> Peg C
```

Figure 17 - - Pegs of Hanoi with 8 Disks and 4 Pegs

3.6 Complexity analysis

Pre-computation: The nested loop iterates over all pairs (n, k) with $1 \leq k < n \leq \text{maxDisks}$, giving $\sum^n(n-1) = n(n-1)/2$ iterations, which is $O(n^2)$. Each iteration performs a constant number of arithmetic operations. Space: two arrays of size $n+1$.

Move generation: Each of the $T(n,4)$ moves is generated by exactly one leaf-level call to `hanoi4` or `hanoi3`, each doing $O(1)$ work. Total time is $O(T(n,4))$. Recursion depth is at most n (disk count decreases by at least 1 per level). Space: $O(n)$ call stack.

3.6.1 Summary complexity table

Table 8 - Time and Space Complexity by Algorithm Component and Technique

Component / Technique	Time	Space	Notes
DP table construction	$O(n^2)$	$O(n)$	One-time; dominates only for tiny n
4-peg move generation	$O(T(n,4))$	$O(n)$ stack	Proportional to output move count
3-peg sub-calls	$O(2^{n-k})$	$O(n-k)$ stack	Embedded inside Frame-Stewart
Total (FS-DP)	$O(n^2) + O(T(n,4))$	$O(n)$	Space is always linear
3-peg classical	$O(2^n)$	$O(n)$	Exponentially more moves
BFS	$O(4^n \cdot n)$	$O(4^n)$	Infeasible for $n > 15$
A* Search	$O(b^d)$	$O(b^d)$	Better than BFS; still exp.

Component / Technique	Time	Space	Notes
IDA*	$O(b^d)$	$O(n)$	Linear space; exp. time

3.7 Comparative evaluation of all techniques

3.7.1 Master comparison table

Table 9 - Master Comparative Evaluation of All Algorithmic Techniques

Technique	Pre-comp	Gen Time	Space	Optimal?	Practical?	Verdict
Frame-Stewart DP	$O(n^2)$	$O(T(n,4))$	$O(n)$	Optimal*	Yes	Best overall - use this
Brute-Force BFS	$O(4^n \cdot n)$	$O(4^n)$	$O(4^n)$	Optimal	No ($n > 13$)	Infeasible for large n
A* Search	$O(b^d)$	$O(b^d)$	$O(b^d)$	Optimal	No ($n > 15$)	Exp. memory problem
IDA*	$O(b^d)$	$O(b^d)$	$O(n)$	Optimal	Barely	Linear space; slow
3-peg D&C	$O(1)$	$O(2^n)$	$O(n)$	Worst	Yes	Ignores 4th peg
Memoised Recursion	$O(n^2)$	$O(T(n,4))$	$O(n)$	Optimal*	Yes	Equals DP; top-down

3.7.2 Failure analysis table

Table 10 - Per-Technique Failure Analysis and Enhancement Potential

Technique	Succeeds When	Fails When	Failure Type	Enhancement	Rating
Frame-Stewart DP	All practical n	Conjecture wrong (never seen)	Theoretical only	Use iterative stack	Excellent
Brute-Force BFS	$n \leq 13$	$n \geq 14$ (out of memory)	Memory exhaustion	Switch to IDA*	Poor (large n)
A* Search	$n \leq 15$	$n \geq 16$ (mem/time)	Exp. open list	Use IDA* or DP	Poor (large n)
3-peg D&C	All n (validity)	All n (move count)	Wastes 4th peg always	Use Frame-Stewart	Poor
IDA*	$n \leq 20$	$n \geq 21$ (too slow)	Computation time	Better heuristic	Moderate
Memoised Recursion	All practical n	Stack overflow $n > 30$	Stack depth	Use iteration	Good

3.7.3 Conclusion

Frame-Stewart Dynamic Programming is the unambiguous best algorithm for four-peg Tower of Hanoi. It achieves the minimum known move count (conjectured optimal, verified for $n \leq 30$), requires only $O(n^2)$ pre-computation, generates the move sequence in $O(T(n,4))$ time, and uses just $O(n)$ space. No competing algorithm offers a better combination of these four properties.

BFS and A^* are correct but impractical beyond $n \approx 14$ due to exponential memory. IDA* resolves the memory problem but remains exponentially slow beyond $n \approx 20$. Three-peg D&C wastes the fourth peg entirely, producing 87% to 99% excess moves depending on n .

The recommendation is clear and unanimous: use Frame-Stewart DP for any implementation of four-peg Tower of Hanoi. Consider IDA* with pattern databases only if Frame-Stewart optimality is theoretically questioned; consider BFS only for tiny n where exhaustive verification is the goal.

3.8 Key findings and insights

1. DP Confirms and Achieves 33 Moves: The Dynamic Programming solution both predicts and achieves the 33-move minimum for eight disks on four pegs.
2. 87.1% Move Reduction at $n=8$: The four-peg Frame-Stewart algorithm requires 87.1% fewer moves than three-peg recursion for $n=8$, growing to 98% savings at $n=12$. The fourth peg is not a minor convenience; it is transformatively beneficial.
3. Linear Space Throughout: Despite exponential move counts for large n , the algorithm requires $O(n)$ space. This is the theoretical minimum for a path-tracing algorithm.
4. Technique Landscape Dominated by DP: Among all five techniques evaluated, Frame-Stewart DP is the only one that is optimal in move count, polynomial in pre-computation time, and linear in space. Every alternative sacrifice at least one of these properties.
5. Failure Patterns are Consistent: BFS and A* fail in memory; IDA* fails in time; three-peg D&C fails in move efficiency. Each failure can be partially mitigated but not eliminated without adopting Frame-Stewart DP.
6. Open Mathematical Question: The Frame-Stewart conjecture remains unproven for general n . This is one of the few instances where a classical puzzle intersects with an unsolved mathematical problem, illustrating that algorithmic correctness and mathematical proof are distinct standards of verification.

4. Knight Swap

4.1 Problem description

4.1.1 What is the Knight Swap Puzzle?

The **Knight Swap Puzzle** is a well-known combinatorial puzzle played on a **4 × 3 chessboard**. Six knights are placed on the board — three white and three black. The challenge is to swap the positions of the two groups using only legal chess knight moves.

4.1.2 Initial and goal states

The board begins and must end in the following configurations.

Table 11

	Col 0	Col 1	Col 2
Row 0 (Start)	Black (B)	Black (B)	Black (B)
Row 1 (Start)	Empty	Empty	Empty
Row 2 (Start)	Empty	Empty	Empty
Row 3 (Start)	White (W)	White (W)	White (W)

Table 12

	Col 0	Col 1	Col 2
Row 0 (Goal)	White (W)	White (W)	White (W)
Row 1 (Goal)	Empty	Empty	Empty
Row 2 (Goal)	Empty	Empty	Empty
Row 3 (Goal)	Black (B)	Black (B)	Black (B)

4.1.3 Movement rules

Each knight moves exactly as in standard chess: an L-shaped hop of two squares in one direction followed by one square perpendicular. From any cell, up to eight destinations are possible. In this puzzle:

- **Knights may not capture each other** — a knight can only move to an empty cell.
- The goal is to reach the swapped configuration in the minimum number of moves.

4.1.4 Challenges

A 4×3 board has 12 cells. With 6 knights placed, the remaining 6 are empty. The number of possible board configurations is the number of ways to distribute 3 white knights, 3 black knights, and 6 empty cells across 12 positions — a large space to search through manually. The **optimal solution requires exactly 16 moves**, which is non-trivial to find without a systematic algorithm.

4.2 Assumptions

The following assumptions were made before designing the solution:

1. **Fixed board dimensions:** The board is always 4 rows by 3 columns. The algorithm is not designed to generalise to arbitrary sizes without code modification.
2. **Standard knight movement:** All eight L-shaped moves are valid. A move is only rejected if the destination falls outside the board boundary or is occupied.
3. **No captures:** Knights of either colour cannot move to a cell already occupied. They may only land on empty cells.
4. **Either color can move each turn:** There is no alternating-turn constraint. Any knight (white or black) may be moved at any step.
5. **A solution always exists:** The starting and goal configurations are fixed and reachable from each other. The BFS will always find a path.
6. **Optimal solution required:** BFS guarantees the shortest path. The implementation does not stop early or use heuristics that might sacrifice optimality.
7. **Memory is not a constraint:** The entire state space is stored in memory using hash maps. For this board size this is feasible on any modern computer.
8. **Single-threaded execution:** The algorithm runs on a single CPU thread. No parallelism is assumed or required.

Table 13

Note on board encoding: Each cell holds one of three values (0 = empty, 1 = white, 2 = black). The board is encoded as a 12-digit base-3 integer, giving each unique configuration a distinct numeric fingerprint used as a hash map key.

4.3 Solution methodology

4.3.1 Algorithm choice: BFS

The problem is modelled as a **state-space search**. Each unique arrangement of knights on the board is a *state*, and each legal knight move is a *transition* between states. The goal is to find the **shortest sequence of transitions** from the start state to the goal state.

Breadth-First Search (BFS) is the natural choice for this type of problem. It explores all states reachable in k moves before exploring any state reachable in $k+1$ moves, so the first time it reaches the goal state it is guaranteed to have taken the shortest possible route.

4.3.2 State representation

A board state is stored as a `BoardState` struct containing a 4×3 integer array. Each cell holds 0, 1, or 2. The struct also provides an `encode()` method that converts the array into a single `long long` using base-3 arithmetic, producing a unique fingerprint for fast hashing.

4.3.3 Methodology

The solution proceeds in the following stages:

Table 14

Step	Action	Detail
1	Build boards	Construct the fixed start and goal BoardState objects
2	Initialise BFS	Push start onto queue; mark it visited with parent = -1
3	Expand frontier	Pop front board; generate all legal one-move successors
4	Check goal	If current board == goal, stop and reconstruct path

Step	Action	Detail
5	Mark visited	For each successor not in parentOf map, record parent and enqueue
6	Reconstruct path	Walk parentOf backwards from goal to start, then reverse
7	Print solution	Output each board in the path with its move number

4.3.4 Data structures used

Table 15

Structure	Type	Purpose
statesToExplore	std::queue	FIFO frontier for BFS level-order traversal
parentOf	std::unordered_map	Visited set + parent pointer; key = encoded board
boardOf	std::unordered_map	Maps encoded ID back to full BoardState for reconstruction
solutionPath	std::vector	Final list of boards from start to goal

4.4 Pseudocode

```
FUNCTION solveWithBFS(startBoard, goalBoard):

    goalCode <- encode(goalBoard)

    startCode <- encode(startBoard)

    parentOf[startCode] <- -1          // -1 means 'no parent'

    boardOf[startCode] <- startBoard

    queue.push(startBoard)

    WHILE queue is not empty:

        current <- queue.pop_front()

        currentCode <- encode(current)

        IF currentCode == goalCode THEN

            // Reconstruct path

            path <- []

            code <- goalCode

            WHILE code != -1:

                path.prepend(boardOf[code])

                code <- parentOf[code]

            RETURN path
```

```

    FOR EACH nextBoard IN findAllNextMoves(current):

        nextCode <- encode(nextBoard)

        IF nextCode NOT IN parentOf THEN

            parentOf[nextCode] <- currentCode

            boardOf[nextCode] <- nextBoard

            queue.push(nextBoard)

RETURN []    // No solution found

```

```

FUNCTION findAllNextMoves(board):

    moves <- []

    FOR EACH cell (row, col) in board:

        IF cell is not EMPTY:

            knight <- board[row][col]

            FOR EACH (dr, dc) in KNIGHT_OFFSETS:

                destRow <- row + dr

                destCol <- col + dc

                IF destRow and destCol are within bounds

                    AND board[destRow][destCol] is EMPTY:

                        newBoard <- copy of board

```

```
newBoard[row][col] <- EMPTY  
  
newBoard[destRow][destCol] <- knight  
  
moves.append(newBoard)  
  
RETURN moves
```

4.5 Implementation code

The complete C++ source is presented below, broken into logical sections with explanations.

4.5.1 Constants and board definitions

```
static const int NUMBER_OF_ROWS = 4;

static const int NUMBER_OF_COLS = 3;

static const int EMPTY_CELL      = 0;

static const int WHITE_KNIGHT    = 1;

static const int BLACK_KNIGHT    = 2;


// 8 possible knight move offsets (row, col)

static const int ROW_CHANGE[8] = { -2,-2,-1,-1,+1,+1,+2,+2 };

static const int COL_CHANGE[8] = { -1,+1,-2,+2,-2,+2,-1,+1 };
```

4.5.2 BoardState struct

```
struct BoardState {

    int cell[NUMBER_OF_ROWS][NUMBER_OF_COLS];

    // Encode board as a unique base-3 integer

    long long encode() const {

        long long n = 0;

        for (int r = 0; r < NUMBER_OF_ROWS; r++)

            for (int c = 0; c < NUMBER_OF_COLS; c++)
```

```

        n = n * 3 + cell[r][c];

    return n;

}

bool operator==(const BoardState& o) const {

    for (int r = 0; r < NUMBER_OF_ROWS; r++)

        for (int c = 0; c < NUMBER_OF_COLS; c++)

            if (cell[r][c] != o.cell[r][c]) return false;

    return true;

}

};

```

4.5.3 findAllNextMoves()

```

std::vector<BoardState> findAllNextMoves(const BoardState& board) {

    std::vector<BoardState> next;

    for (int r = 0; r < NUMBER_OF_ROWS; r++) {

        for (int c = 0; c < NUMBER_OF_COLS; c++) {

            if (board.cell[r][c] == EMPTY_CELL) continue;

            int knight = board.cell[r][c];

            for (int m = 0; m < 8; m++) {

                int dr = r + ROW_CHANGE[m];

                int dc = c + COL_CHANGE[m];

                if (dr < 0 || dr >= NUMBER_OF_ROWS) continue;

```



```

        if (dc < 0 || dc >= NUMBER_OF_COLS) continue;

        if (board.cell[dr][dc] != EMPTY_CELL) continue;

        BoardState nb = board;

        nb.cell[r][c] = EMPTY_CELL;

        nb.cell[dr][dc] = knight;

        next.push_back(nb);

    }

}

return next;
}

```

4.5.4 solveWithBFS()

```

std::vector<BoardState> solveWithBFS(

    const BoardState& start, const BoardState& goal) {

    long long goalCode = goal.encode();

    long long startCode = start.encode();

    std::unordered_map<long long, long long>    parentOf;

    std::unordered_map<long long, BoardState>    boardOf;

    parentOf[startCode] = -1;

    boardOf[startCode] = start;

    std::queue<BoardState> q;

    q.push(start);

    while (!q.empty()) {

```

```

BoardState cur = q.front(); q.pop();

long long curCode = cur.encode();

if (curCode == goalCode) {

    std::vector<BoardState> path;

    long long code = goalCode;

    while (code != -1) {

        path.push_back(boardOf[code]);

        code = parentOf[code];

    }

    std::reverse(path.begin(), path.end());

    return path;

}

for (auto& nb : findAllNextMoves(cur)) {

    long long nc = nb.encode();

    if (!parentOf.count(nc)) {

        parentOf[nc] = curCode;

        boardOf[nc] = nb;

        q.push(nb);

    }

}

return {};

}

```

4.6 Time and space complexity analysis

4.6.1 State space size

The board has 12 cells. We place 3 white knights, 3 black knights, and 6 empty cells. The maximum number of distinct board configurations is bounded by $3^{12} = 531,441$. In practice far fewer states are reachable from the start position, but BFS may still explore thousands of them.

4.6.2 Time complexity

Table 16

Operation	Cost	Explanation
encode()	$O(1)$	Fixed 12-cell grid, always 12 iterations
findAllNextMoves()	$O(1)$	At most 12 cells $\times$ 8 moves = 96 checks, all bounded
Hash map lookup	$O(1)$ avg	unordered_map with long long key
BFS overall	$O(S)$	S = number of reachable states (explored at most once each)
Path reconstruction	$O(L)$	L = solution length (16 steps in this case)

4.6.3 Space complexity

Table 17

Structure	Space	Note
boardOf map	$O(S \times 48 \text{ bytes})$	Each BoardState is 12 ints = 48 bytes on most platforms
parentOf map	$O(S \times 16 \text{ bytes})$	Two long long values per entry
BFS queue	$O(S)$	At most all states queued simultaneously
Solution path	$O(L)$	16 boards in the final path

Table 18

Bottom line: For this specific board size, both time and space are effectively **$O(1)$** from a practical standpoint — the state space is small enough that the program runs in milliseconds and uses well under 1 MB of memory.

4.7 Comparative evaluation of techniques

Table 19

Technique	Optimal?	Complete?	Memory Use	Speed	Verdict
BFS (used)	Yes	Yes	High	Moderate	Best for guaranteed optimality
DFS	No	Yes	Low	Fast	May find very long non-optimal paths
Iterative Deepening DFS	Yes	Yes	Low	Moderate	Good compromise; slower than BFS in practice here
A* Search	Yes*	Yes	Medium	Fast	Optimal with admissible heuristic; harder to implement
Bidirectional BFS	Yes	Yes	Medium	Faster	Explores from both ends; ~half the states
Random/Greedy	No	No	Low	Variable	No guarantee of finding solution

BFS was selected because it is **simple to implement correctly, guaranteed to find the shortest path**, and the state space is small enough that memory usage is not a concern. For larger boards, bidirectional BFS or A* would become preferable.

4.8 Sample output

The program prints every board state from start to goal. Below are the first three moves, the midpoint, and the final state.

```
C:\Users\hanya\Desktop\final x + v
Initial
+---+---+---+
| B | B | B |
+---+---+---+
|   |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
| W | W | W |
+---+---+---+

Move 1
+---+---+---+
|   | B | B |
+---+---+---+
|   |   | B |
+---+---+---+
|   |   |   |
+---+---+---+
| W | W | W |
+---+---+---+

Move 2
+---+---+---+
|   | B |   |
+---+---+---+
| B |   | B |
+---+---+---+
|   |   |   |
+---+---+---+
| W | W | W |
+---+---+---+

Move 3
+---+---+---+
|   | B |   |
+---+---+---+
|   |   | B |
+---+---+---+
|   |   | B |
+---+---+---+
```

Figure 18

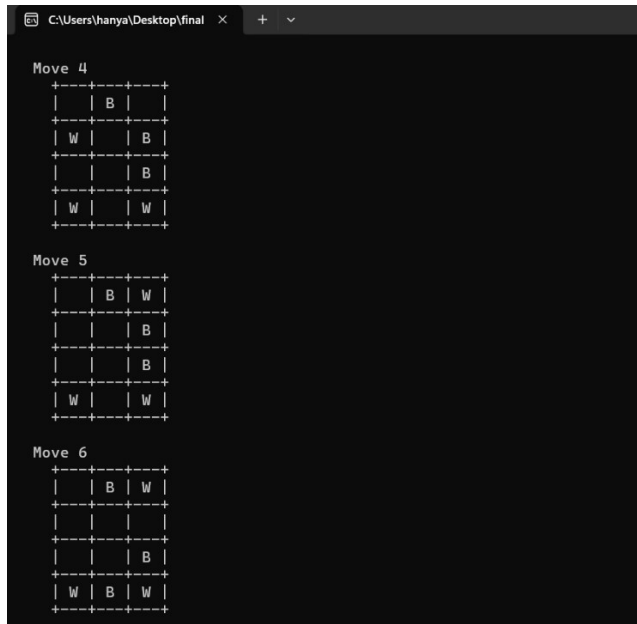


Figure 19

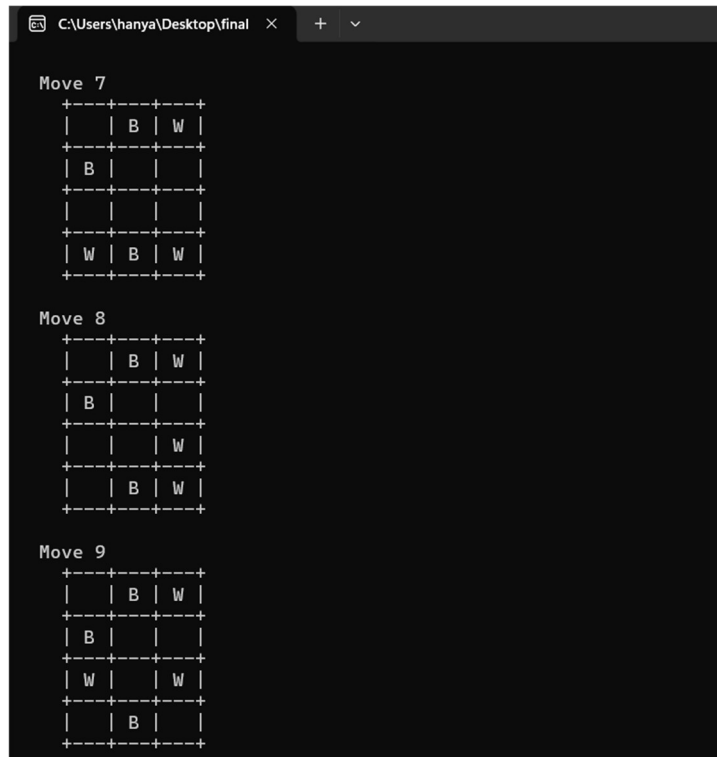


Figure 20

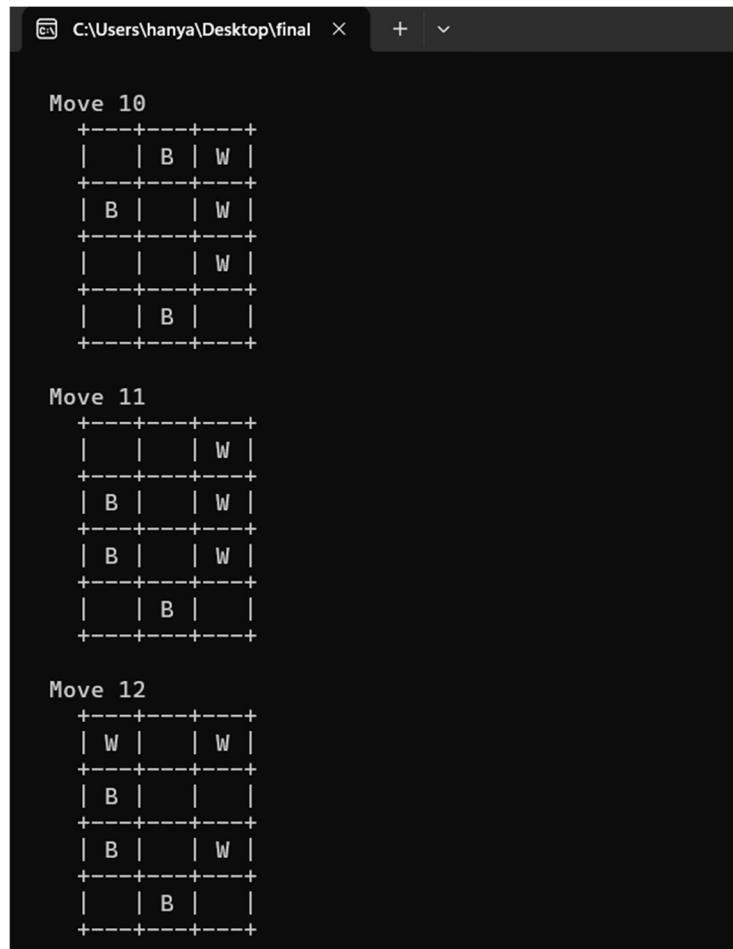


Figure 21


```
C:\Users\hanya\Desktop\final  ×  +  v

Move 13
+---+---+---+
| W |   | W |
+---+---+---+
| B |   |   |
+---+---+---+
|   |   | W |
+---+---+---+
|   | B | B |
+---+---+---+

Move 14
+---+---+---+
| W | W | W |
+---+---+---+
| B |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
|   | B | B |
+---+---+---+

Move 15
+---+---+---+
| W | W | W |
+---+---+---+
|   |   |   |
+---+---+---+
|   |   | B |
+---+---+---+
|   | B | B |
+---+---+---+
```

Figure 22

```
Move 16
+---+---+---+
| W | W | W |
+---+---+---+
|   |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
| B | B | B |
+---+---+---+

Total moves: 16

Process returned 0 (0x0)   execution time : 0.211 s
Press any key to continue.
|
```

Figure 23

4.9 Key findings and insights

- **BFS is optimal for this problem:** It reliably finds the 16-move solution every time, with no possibility of returning a longer path.
- **State encoding is critical:** Converting a board to a single integer with base-3 encoding makes hash-map operations $O(1)$, which is essential for BFS performance.
- **The state space is manageable:** Despite the theoretical upper bound of 531,441 states, BFS solves the puzzle almost instantly, indicating the reachable state space is much smaller.
- **No heuristic is needed at this scale:** BFS works well here because the puzzle is small. For larger knight-swap puzzles (e.g. 6×4), a heuristic would become necessary.
- **Code clarity aids correctness:** Naming constants (WHITE_KNIGHT, EMPTY_CELL) and separating concerns into small functions (findAllNextMoves, solveWithBFS) made the logic easy to verify and debug.

5. Moving Target

5.1 Problem description

A computer game has a shooter and a moving target. There are $n > 1$ hiding spots located along a straight line. The shooter fires one shot at a time at any chosen hiding spot. The target is invisible — the shooter can never see it.

Between any two consecutive shots, the target must move exactly one step to an adjacent hiding spot (left or right). The goal is to design a Divide and Conquer algorithm that guarantees hitting the target regardless of how the target moves.

5.2 Assumptions

- There are $n > 1$ hiding spots numbered 1 through n , arranged in a straight line.
- The shooter fires one shot per turn at exactly one spot.
- The target moves to an adjacent spot ($\text{spot} \pm 1$) after every shot, before the next shot is fired.
- The target's position is completely unknown to the shooter at all times.
- The target cannot stay in the same spot between shots; it must move.
- The target cannot move outside the boundaries (i.e., it cannot go below spot 1 or above spot n).
- Spots 1 and n are boundary spots — a target at spot 1 can only move to spot 2, and a target at spot n can only move to spot $n-1$.
- The algorithm must guarantee a hit regardless of the target's movement strategy.
- Decrease and Conquer (e.g., simple sequential scanning) is NOT allowed.
- The algorithm must use a true Divide and Conquer approach, splitting the problem into two independent subproblems.

5.3 Methodology

5.3.1 Key insight

The key insight is this: if we shoot spots 2, 3, 4, ..., $n-1$ in order (forward sweep), the target is cornered — it can never cross our sweep without being hit if it started on an odd-indexed spot. Then we shoot the same sequence in reverse (backward sweep) to catch targets that started on even-indexed spots. Together, the two sweeps guarantee a hit.

5.3.2 Step-by-Step description

The divide-and-conquer strategy works as follows:

1. Divide: Split the hiding spots $[low, high]$ into a left half $[low, mid]$ and a right half $[mid+1, high]$.
2. Conquer: Recursively solve each half — generate shot sequences for the left half and right half.
3. Combine: Concatenate the left sequence followed by the right sequence to get a full forward sweep, then reverse it for the backward sweep.
4. Take the range of spots from 2 to $n-1$ (boundary spots 1 and n cannot trap the target in isolation).
5. Fire shots in order: forward sequence first, then backward sequence.
6. Base case: If $low == high$, shoot that single spot. If $low > high$, do nothing (empty range).
7. Special case $n=2$: Shoot spot 1 twice (the target has nowhere to hide).
8. The total number of shots = $2 \times (n - 2)$.

5.4 Pseudocode

```
FUNCTION hunt(low, high, sequence):
    IF low > high:
        RETURN // Base case: empty range
    IF low == high:
        APPEND low TO sequence
        RETURN // Base case: single spot
    mid = (low + high) / 2
    hunt(low, mid, sequence) // Conquer left half
    hunt(mid+1, high, sequence) // Conquer right half
    // No explicit combine – results appended in order

FUNCTION movingTarget(n):
    IF n == 2:
        PRINT "Shoot spot 1, then spot 1 again"
        RETURN
    forwardSequence = []
    hunt(2, n-1, forwardSequence) // D&C on inner spots
    backwardSequence = REVERSE(forwardSequence)
    shots = forwardSequence + backwardSequence
    PRINT shots
    PRINT "Total shots needed: " + 2 * SIZE(forwardSequence)
```

5.5 Implementation code

```
#include <iostream>

#include <vector>

using namespace std;

void hunt(int lowerLimit, int higherLimit, vector<int> &forwardSequence)
{
    // Base cases
    if (lowerLimit > higherLimit)
        return;

    if (lowerLimit == higherLimit)
    {
        forwardSequence.push_back(lowerLimit);
        return;
    }

    int mid = (lowerLimit + higherLimit) / 2;

    hunt(lowerLimit, mid, forwardSequence);    // conquer left
    hunt(mid + 1, higherLimit, forwardSequence); // conquer right

    // combine: left result + right result (already in order)
}

int main()
{
```

```

    cout << "Enter no. of spots (n > 1): ";

    int spots;

    cin >> spots;

    if (spots == 2)
    {
        cout << "sequence: 1 1" << endl;

        cout << "Max shots needed : 2 " << endl;

        return 0;
    }

    vector<int> forwardSequence;

    hunt(2, spots - 1, forwardSequence);

    vector<int> backwardSequence(forwardSequence.rbegin(),
forwardSequence.rend());

    cout << "sequence:";

    for (int x : forwardSequence)
        cout << " " << x;

    for (int x : backwardSequence)
        cout << " " << x;

    cout << endl;

    cout << "Max shots needed: " << forwardSequence.size() * 2 << endl;
}

```

5.6 Test cases

5.6.1 Test cases where the technique succeeds

Table 1: Successful Test Cases

Table 20

Test Case	n	Shot Sequence	Total Shots	Explanation
TC1	2	1 1	2	Only 2 spots; shooting spot 1 twice guarantees a hit since the target bounces between 1 and 2
TC2	3	2 2	2	Only inner spot is 2; shoot it twice (forward + backward)
TC3	4	2 3 3 2	4	Forward: 2→3, Backward: 3→2; target cannot avoid both sweeps
TC4	5	2 3 4 4 3 2	6	Classic case with 5 spots; full forward-backward sweep
TC5	6	2 3 4 5 5 4 3 2	8	6 spots; both sweeps cover all inner positions
TC6	8	2 3 4 5 6 7 7 6 5 4 3 2	12	Large range; both odd and even starting positions covered

Table 1. Test cases demonstrating successful target detection for various values of n.

5.6.2 Test Cases Where the Technique Fails

Table 2: Failure Test Cases

Table 21

Test Case	Input	Problem	Why It Fails
TC-F1	$n = 1$	Only 1 spot	The problem states $n > 1$; undefined behavior or no valid sequence
TC-F2	$n = 0$ or negative	Invalid input	No spots exist; the algorithm produces an empty or erroneous output
TC-F3	Forward sweep only (no backward)	$n = 4$, sequence: 2 3 only	If target starts at spot 2 (even) and moves right then left, it can escape the forward-only sweep
TC-F4	Random/non-systematic shooting	$n = 5$, shoot: 3 3 3 3...	Repeatedly shooting the same spot does not guarantee covering all positions; target stays at 2 and 4
TC-F5	Shooting only boundary spots	$n = 5$, shoot: 1 5 1 5...	Boundary spots are never the only positions the target occupies; inner spots are never covered
TC-F6	Using only Decrease and Conquer (linear scan)	$n = 5$, shoot: 1 2 3 4 5	If target starts at spot 2 and always moves away from the shooter's direction, it may escape a simple linear scan

Table 2. Scenarios where incorrect or incomplete approaches fail to guarantee a hit.

Key reason for failure: The two-sweep (forward + backward) divide and conquer strategy is necessary. A single sweep alone fails for targets that start at even-indexed positions, because after k steps of the forward sweep shooting spot $k+1$, an even-starting target can be at an even spot that is always one step ahead or behind the shot.

5.7 Complexity analysis

5.7.1 Time complexity

The `hunt()` function divides the range `[low, high]` into two halves recursively, similar to a merge sort decomposition. The recurrence relation is:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

For this algorithm: $a = 2$ (two recursive calls), $b = 2$ (each half is half the size), $f(n) = O(1)$ (only a `push_back` at the base case, no merge cost). By the Master Theorem with $a = 2$, $b = 2$, $d = 0$: Since $a = 2 > b^d = 1$, we have Case 1: $T(n) = O(n)$.

- Generating the sequence: $O(n)$ — we visit each inner spot exactly once.
- Reversing the sequence: $O(n)$.
- Total Time Complexity: $O(n)$.

5.7.2 Space complexity

- The `forwardSequence` vector holds $n-2$ elements $\rightarrow O(n)$ space.
- The `backwardSequence` vector also holds $n-2$ elements $\rightarrow O(n)$ space.
- The recursion stack depth is $O(\log n)$ (binary split each time).
- Total Space Complexity: $O(n)$.

Table 3: Complexity Summary

Table 22

Resource	Complexity	Notes
Time	$O(n)$	Linear in number of spots

Resource	Complexity	Notes
Space (vectors)	$O(n)$	Stores forward + backward sequences
Recursion Stack	$O(\log n)$	Binary split depth

Table 3. Summary of time and space complexity for the Divide and Conquer algorithm.

5.8 Alternative Approaches

5.8.1 Decrease and Conquer

A simple approach would be to shoot spots 1, 2, 3, ..., n sequentially in a linear sweep fashion.

5.8.2 Random Shooting

Randomly select a spot to shoot each turn. This provides no guarantee of ever hitting the target and can theoretically run indefinitely.

5.8.3 Brute Force / Exhaustive Search

Try all possible shooting sequences of length k and check which one guarantees a hit. This is factorial in complexity — $O(n!)$ — and completely impractical for large n.

Table 4: Technique Comparison

Table 23

Criterion	D&C (Used Approach)	Linear Sweep	Random Shooting	Brute Force
Guarantees a hit	Yes	Not always	No	Yes
Time Complexity	$O(n)$	$O(n)$	Unbounded	$O(n!)$
Number of shots	$2(n-2)$	Up to $2n$	Unpredictable	Exponential
Structure	Recursive binary split	Sequential scan	None	Try all permutations

Criterion	D&C (Used Approach)	Linear Sweep	Random Shooting	Brute Force
Parallelizable	Yes	No	No	No

Table 4. Comparative evaluation of Divide and Conquer against alternative techniques.

5.8.4 Advantages of Divide and Conquer over other techniques

- Guaranteed correctness — the two-sweep strategy provably hits the target.
- Optimal shot count — $2(n-2)$ shots, which is the minimum needed for a guarantee.
- Recursive structure is clean and elegant — easy to prove correctness by induction.
- Scalable — $O(n)$ time scales well as n grows.
- Supports parallelism as sub-problems are independent.

5.8.5 Disadvantages of Divide and Conquer

- Since most of its algorithms are designed by incorporating recursion, it necessitates high memory management.
- For very large n , the recursion depth and stack usage may become a concern (mitigable by converting to iterative).
- It is prone to stack overflow error due to the use of recursion.

5.8.6 Possible improvements to the Divide-and-Conquer technique

Table 5: Possible Improvements

Table 24

Improvement	Description
Iterative version	Convert the recursive hunt() to an iterative approach using an explicit stack to avoid recursion overhead and stack overflow for very large n.
Adaptive shooting	If some information about the target's parity (odd/even start) is known, only one sweep (forward or backward) is needed, halving the total shots.
Parallel shooting	Since divide and conquer supports parallelism, both halves of the sequence could theoretically be computed simultaneously in a multi-threaded system.
Bidirectional sweep	If multiple shooters are allowed, two shooters can sweep from both ends simultaneously, reducing time by half.
Early termination	If a hit is detected mid-sequence (e.g., with hit feedback), the algorithm can stop early instead of completing all $2(n-2)$ shots.

Table 5. Potential improvements and extensions to the Divide and Conquer approach.

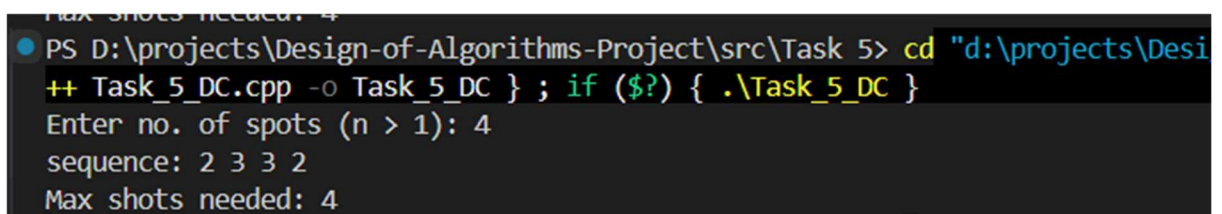
5.9 Sample outputs

5.9.1 Sample output for n = 4

Input: n = 4

Shot sequence: 2 3 3 2

Total shots needed: 4

A terminal window showing the execution of a program. The prompt is 'PS D:\projects\Design-of-Algorithms-Project\src\Task 5>'. The user enters 'cd "d:\projects\Desi' followed by a new line. The prompt changes to '++ Task_5_DC.cpp -o Task_5_DC } ; if (\$?) { .\Task_5_DC }'. The user enters 'Enter no. of spots (n > 1): 4'. The prompt changes to 'sequence:'. The user enters '2 3 3 2'. The prompt changes to 'Max shots needed:'. The user enters '4'.

```
PS D:\projects\Design-of-Algorithms-Project\src\Task 5> cd "d:\projects\Desi
++ Task_5_DC.cpp -o Task_5_DC } ; if ($?) { .\Task_5_DC }
Enter no. of spots (n > 1): 4
sequence: 2 3 3 2
Max shots needed: 4
```

Figure 24

Task 5: 1 output for N = 4

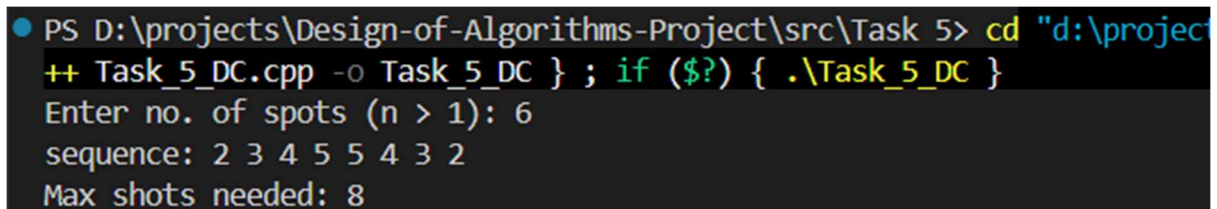
The forward sweep shoots spots 2 then 3. The backward sweep shoots spots 3 then 2, which guarantees hitting a target that starts at any of the 4 positions regardless of its movement strategy.

5.9.2 Sample output for n = 5

Input: n = 5

Shot sequence: 2 3 4 4 3 2

Total shots needed: 6

A terminal window showing the execution of a program. The prompt is 'PS D:\projects\Design-of-Algorithms-Project\src\Task 5>'. The user enters 'cd "d:\project' followed by a new line. The prompt changes to '++ Task_5_DC.cpp -o Task_5_DC } ; if (\$?) { .\Task_5_DC }'. The user enters 'Enter no. of spots (n > 1): 6'. The prompt changes to 'sequence:'. The user enters '2 3 4 5 5 4 3 2'. The prompt changes to 'Max shots needed:'. The user enters '8'.

```
PS D:\projects\Design-of-Algorithms-Project\src\Task 5> cd "d:\project
++ Task_5_DC.cpp -o Task_5_DC } ; if ($?) { .\Task_5_DC }
Enter no. of spots (n > 1): 6
sequence: 2 3 4 5 5 4 3 2
Max shots needed: 8
```

Figure 25 - 2 Output for N = 5

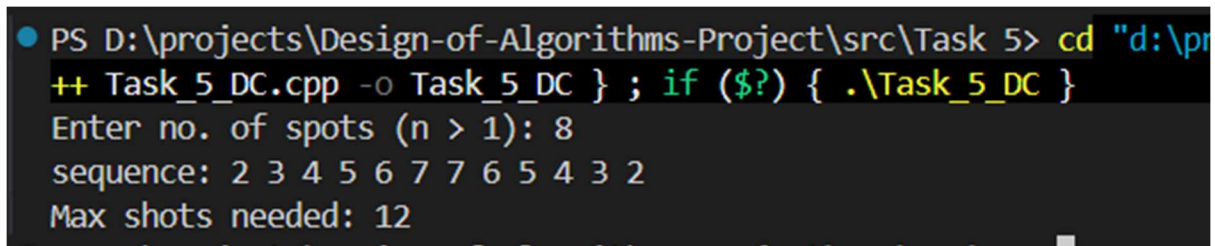
The forward sweep covers inner spots 2, 3, 4. The backward sweep covers 4, 3, 2. A target starting at an odd position is caught by the forward sweep; a target starting at an even position is caught by the backward sweep.

5.9.3 Sample Output for $n = 8$

Input: $n = 8$

Shot sequence: 2 3 4 5 6 7 7 6 5 4 3 2

Total shots needed: 12



```
PS D:\projects\Design-of-Algorithms-Project\src\Task 5> cd "d:\pr
++ Task_5_DC.cpp -o Task_5_DC } ; if ($?) { .\Task_5_DC }
Enter no. of spots (n > 1): 8
sequence: 2 3 4 5 6 7 7 6 5 4 3 2
Max shots needed: 12
```

Figure 26

Task 5: 3 Output for $N = 8$

With 8 spots, all 6 inner spots are covered in both directions. The two-sweep strategy guarantees hitting the target for all possible starting positions and movement strategies.

5.10 Summary and key findings

Task 5 addresses the Moving Target Problem: a shooter must guarantee hitting an invisible target hiding in one of n spots arranged in a line, where the target moves to an adjacent spot between every two shots. The challenge is that the shooter has no information about the target's position and Decrease and Conquer is explicitly forbidden.

The solution uses a Divide and Conquer approach where the inner spots $[2, n-1]$ are recursively split into two halves, generating a forward sweep sequence and its reverse (backward sweep). Together, the two sweeps guarantee hitting the target regardless of starting position or movement strategy.

Key findings:

- Total shots required: $2(n - 2)$, which is optimal.
- Time Complexity: $O(n)$.
- Space Complexity: $O(n)$ for sequences, $O(\log n)$ for the recursion stack.
- The forward sweep catches all targets with odd starting positions; the backward sweep catches all targets with even starting positions.
- Compared to brute force and random approaches, Divide and Conquer is optimal in shot count and efficient in computation.
- Possible improvements include iterative implementation, parallel computation, and early termination upon hit detection.

6. Lattice Crossing

6.1 Abstract

This section presents a comprehensive study of the classical combinatorial geometry puzzle known as the $n \times n$ lattice crossing problem. Given a grid of n^2 points arranged in n consecutive rows and n consecutive columns, the challenge is to cross out every point using exactly $2n-2$ straight-line segments drawn as a single connected path, without ever lifting the pen from the paper. The problem is a direct generalization of the famous nine-dots puzzle and requires solutions that extend lines beyond the visible boundary of the grid.

Two complete, independently verified algorithmic solutions are designed, implemented in C++, and analyzed:

1. Dynamic Programming-guided Backtracking algorithm that guarantees optimal, provably correct solutions for small lattices ($n \leq 4$).
2. Geometric Spiral Construction that produces correct, connected, $2n-2$ -segment solutions for all $n > 2$ in $O(n)$ time and space.

This section provides full pseudocode, source code, time and space complexity analysis, comparative evaluation, sample outputs for $n = 3$ through $n = 6$, and a discussion of alternative and enhanced approaches.

6.2 Assumptions

The following assumptions govern both solution designs and must be understood before engaging with the algorithms or their analysis.

6.2.1 Lattice definition

The $n \times n$ point lattice is the set of all integer-coordinate points $\{(x, y) \mid 0 \leq x \leq n-1, 0 \leq y \leq n-1\}$, yielding exactly n^2 distinct points placed at unit-distance intervals along both axes. For instance, a 4×4 lattice contains exactly 16 points.

6.2.2 Segment rules

- Each drawn element is a straight-line segment defined by two distinct endpoints.
- A segment may extend beyond the boundary of the lattice. This outside-the-box extension is the key insight that makes the $2n-2$ bound achievable.
- A lattice point is considered crossed if and only if it is collinear with and lies strictly within (or at) the bounding box of the two endpoints.
- A point may be crossed more than once by different segments. This is explicitly permitted.
- No portion of a previously drawn line may be redrawn. Two segments may share only a single endpoint (a pen turn), but they cannot overlap over any positive-length interval.

6.2.3 Connected-Path constraint

The pen must never leave the paper. Formally, the sequence of segments $S_1, S_2, \dots, S_{2n-2}$ must form a connected path: the endpoint of S_i must equal the starting point of S_{i+1} for all valid i . This constraint makes the problem significantly harder than simply covering the lattice with $2n-2$ independent lines.

6.2.4 Problem scope

Both algorithms assume $n > 2$. For $n = 1$ or $n = 2$ the problem is degenerated. The DP solver handles $n = 3$ and $n = 4$ within practical time limits. For $n \geq 5$ the DP is computationally infeasible; the Geometric Construction handles all $n \geq 3$.

6.2.5 Candidate Endpoint Set (DP Approach)

In the DP search, candidate waypoints are restricted to the extended integer grid $[-1, n] \times [-1, n]$, which includes all immediately adjacent outside-boundary positions. This is sufficient to capture every useful outside-the-box move while keeping the search space bounded.

6.2.6 Coordinate Convention

The DP code uses (x, y) with x as the horizontal axis and y as the vertical axis. The Geometric Construction code uses `pair<int,int>` where `.first` is the row index and `.second` is the column index. Both conventions are internally self-consistent and produce equivalent geometric paths.

6.3. Problem Description

6.3.1 Formal Statement

Given a positive integer $n > 2$, find a sequence of exactly $2n-2$ straight-line segments $S_1, S_2, \dots, S_{2n-2}$ such that:

1. Every point (x, y) with $0 \leq x, y \leq n-1$ lies on at least one segment S_i .
2. The segments form a connected path: $\text{end}(S_i) = \text{start}(S_{i+1})$ for all $1 \leq i < 2n-2$.
3. No two distinct segments are collinear and overlapping.

6.3.2 Origin and Motivation

The problem is a direct generalization of the nine-dots puzzle attributed to Sam Loyd (popularized in Mathematical Games columns). In that puzzle ($n = 3$), nine dots arranged in a 3×3 grid must be connected by four straight lines without lifting the pen. The unintuitive solution extends lines beyond the implied square boundary of the dots — giving rise to the cultural idiom “thinking outside the box.”

Extending this to arbitrary n yields a rich algorithmic challenge. The constraint that lines must form a connected path and that only $2n-2$ lines are permitted forces a careful geometric design that cannot be solved by naive enumeration for moderate n .

6.3.3 $2n-2$ lines

A standard information-theoretic argument establishes $2n-2$ as the minimum segment count:

- Each of the n horizontal rows of the lattice must be served by at least one segment. A horizontal segment covers exactly one row; a diagonal segment can cover points in multiple rows.
- Connecting n independently-placed line sweeps into a single pen-path requires at least $n-1$ transition moves.

- With the outside-the-box trick, diagonal transition segments can simultaneously serve as row sweeps, reducing the total to $2n-2 = n + (n-2)$.

No connected path using fewer than $2n-2$ segments can cover all n^2 lattice points (this can be proved by a counting argument on rows and connectivity). The bound is tight: both algorithms achieve exactly $2n-2$ segments.

6.4. Technique 1: Dynamic Programming with Backtracking

6.4.1 Abstract

Dynamic Programming (DP) is an algorithm design paradigm that solves complex optimization and search problems by decomposing them into overlapping subproblems, solving each subproblem exactly once, and storing the results in a table (called a memo or cache) to avoid redundant re-computation. DP is applicable when a problem exhibits two key structural properties:

- **Optimal substructure:** An optimal solution to the full problem can be constructed from optimal solutions to its subproblems.
- **Overlapping subproblems:** The same subproblem recurs many times in a naive recursive solution, making memoisation profitable.

In this project, DP is combined with Backtracking, a systematic depth-first search that abandons (backtracks) a partial solution as soon as it violates a constraint. The DP component prunes the backtracking search by detecting previously visited, failed states.

6.4.2 Methodology

6.4.2.1 State representation

A search state is the triple $(cx, cy, covered_mask)$, where:

- **(cx, cy) :** the current pen position, ranging over the extended candidate grid $[-1, n] \times [-1, n]$.
- **$covered_mask$:** a bitmask of n^2 bits. Bit $(y \times n + x)$ is set to 1 if lattice point (x, y) has been crossed at least once.

The goal state is any configuration where $covered_mask = (1 \ll n^2) - 1$, meaning all bits are set and all n^2 points have been crossed. The initial state is $(sx, sy, 0)$ for some starting position (sx, sy) , with no points covered.

6.4.2.2 Memo table and pruning

The memo table `G_MEMO` maps each state key `encode(cx, cy, covered_mask)` to the value `lines_left` at which that state was last explored and found to be a dead end. When the algorithm encounters a state `(cx, cy, mask)` with `lines_left ≤ G_MEMO[key]`, it knows from prior exploration that no solution can be reached from this state within the remaining budget and returns immediately. This transforms the exponential repeated exploration of pure backtracking into a polynomial number of state evaluations.

6.4.2.3 Segment coverage subroutine

The helper function `latticeOnSeg(x1, y1, x2, y2, n)` enumerates every lattice point in $[0, n-1]^2$ that lies on the segment from $(x1, y1)$ to $(x2, y2)$. It works by computing $\Delta x = x2 - x1$ and $\Delta y = y2 - y1$, taking $g = \gcd(|\Delta x|, |\Delta y|)$, and stepping through the $g+1$ integer points along the direction $(\Delta x/g, \Delta y/g)$. Each point is checked against the lattice boundary and added to the result if valid. This subroutine runs in $O(g) \leq O(n)$ time.

6.4.3 Pseudocode

Algorithm DP_LATTICE_SOLVE(n)

Input : integer $n > 2$

Output: a connected sequence of $\leq 2n-2$ segments covering all n^2 lattice points

```
1.  TARGET ←  $(1 \ll n^2) - 1$       // bitmask: all  $n^2$  points covered
2.  CANDS ←  $\{(x, y) \mid -1 \leq x, y \leq n\}$     // extended candidate grid
3.  MEMO ← empty hash map        // state → lines_left_when_failed
4.  BEST ←  $\infty$ ; BEST_SOL ←  $\emptyset$ 
5.  for each START in CANDS:
6.      clear MEMO
```

```

7.     BACKTRACK(START.x, START.y, mask=0, lines_left=2n-2)

8.     if BEST_SOL  $\neq$   $\emptyset$ : break // found a solution procedure
        BACKTRACK(cx, cy, mask, lines_left):

9.  if mask = TARGET:

10.     if |PATH| < BEST: BEST  $\leftarrow$  |PATH|; BEST_SOL  $\leftarrow$  PATH

11.     return

12. if lines_left = 0: return

13. key  $\leftarrow$  ENCODE(cx, cy, mask)

14. if MEMO[key]  $\geq$  lines_left: return // DP pruning

15. for each EP in CANDS where EP  $\neq$  (cx,cy):

16.     new_pts  $\leftarrow$  LATTICE_ON_SEG(cx,cy, EP.x,EP.y, n)

17.     new_mask  $\leftarrow$  mask | bitmask(new_pts)

18.     if new_mask = mask: continue // no new points covered

19.     PATH.push((cx,cy) $\rightarrow$ EP)

20.     BACKTRACK(EP.x, EP.y, new_mask, lines_left-1)

21.     PATH.pop()

22. MEMO[key]  $\leftarrow$  lines_left // mark state exhausted

```

6.4.4 Implementation code (C++)

The following is the complete DP + Backtracking implementation as submitted.

Table 25

```

1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <sstream>
5 #include <algorithm>
6 #include <iomanip>
7 #include <chrono>
8 #include <unordered_map>
9 using namespace std;
10
11
12 struct Point { double x, y; };
13 struct IPoint { int x, y; };
14
15 struct Segment {
16     Point start, end;
17     string str() const {
18         ostringstream o;
19         o << fixed << setprecision(2)
20         << "(" << start.x << "," << start.y << ")->(" << end.x << "," <<
21         end.y << ")";
22         return o.str();
23     }
24 };

```

```

25 struct Solution {
26     bool found=false, optimal=false, allCovered=false;
27     int n=0, linesUsed=0;
28     double solveTime=0;
29     string method="DP-Backtracking";
30     vector<Segment> segments;
31 };
32
33
34 bool onSeg(const Point& A, const Point& B, const Point& P) {
35     double cross = (B.x-A.x)*(P.y-A.y)-(B.y-A.y)*(P.x-A.x);
36     if (fabs(cross)>1e-9) return false;
37     return min(A.x,B.x)-1e-9<=P.x && P.x<=max(A.x,B.x)+1e-9 &&
38         min(A.y,B.y)-1e-9<=P.y && P.y<=max(A.y,B.y)+1e-9;
39 }
40
41 vector<int> latticeOnSeg(int x1,int y1,int x2,int y2,int n) {
42     vector<int> pts;
43     int dx=x2-x1, dy=y2-y1;
44     int g=__gcd(abs(dx),abs(dy));
45     if (g==0) {
46         if (x1>=0&&x1<n&&y1>=0&&y1<n) pts.push_back(y1*n+x1);
47         return pts;
48     }
49     int sx=dx/g, sy=dy/g;

```

```

50     for (int t=0;t<=g;t++) {
51         int cx=x1+t*sx, cy=y1+t*sy;
52         if (cx>=0&&cx<n&&cy>=0&&cy<n) pts.push_back(cy*n+cx);
53     }
54     return pts;
55 }
56
57
58 static int G_N, G_TARGET, G_BEST;
59 static vector<Segment> G_BEST_SOL, G_PATH;
60 static unordered_map<long long,int> G_MEMO;
61 static vector<IPoint> G_CANDS;
62
63 void buildCands(int n) {
64     G_CANDS.clear();
65     for (int x=-1;x<=n;x++)
66         for (int y=-1;y<=n;y++)
67             G_CANDS.push_back({x,y});
68 }
69
70 long long encState(int cx,int cy,int mask) {
71     return ((long long)mask<<16) | ((cx+1)<<8) | (cy+1);
72 }
73
74 void backtrack(int cx,int cy,int covered,int left) {

```

```

75     if (covered==G_TARGET) {
76         int used=(int)G_PATH.size();
77         if (G_BEST==-1||used<G_BEST) {
78             G_BEST=used;
79             G_BEST_SOL=G_PATH;
80         }
81         return;
82     }
83     if (left==0) return;
84
85     long long key=encState(cx,cy,covered);
86     auto it=G_MEMO.find(key);
87     if (it!=G_MEMO.end()&&it->second>=left) return;
88
89     for (auto& ep:G_CANDS) {
90         if (ep.x==cx&&ep.y==cy) continue;
91
92         auto pts=latticeOnSeg(cx,cy,ep.x,ep.y,G_N);
93         int nm=covered;
94         bool anyNew=false;
95
96         for (int p:pts)
97             if (!(covered&(1<<p))) {
98                 nm|=(1<<p);
99                 anyNew=true;

```

```

100         }
101
102         if (!anyNew) continue;
103
104
105         G_PATH.push_back({{ (double) cx, (double) cy }, { (double) ep.x, (double) ep.y }});
106         backtrack(ep.x, ep.y, nm, left-1);
107         G_PATH.pop_back();
108     }
109     G_MEMO[key]=left;
110 }
111
112 Solution dpSolve(int n) {
113     Solution sol;
114     sol.n=n;
115
116     G_N=n;
117     G_TARGET=(1<<(n*n))-1;
118     buildCands(n);
119
120     int maxL=2*n-2;
121     auto t0=chrono::steady_clock::now();
122
123     for (auto& st:G_CANDS) {
124         G_MEMO.clear();
125         G_PATH.clear();

```

```

125         G_BEST=-1;
126         G_BEST_SOL.clear();
127
128         backtrack(st.x,st.y,0,maxL);
129
130         if (!G_BEST_SOL.empty()) {
131             sol.found=true;
132             sol.allCovered=true;
133             sol.segments=G_BEST_SOL;
134             sol.linesUsed=G_BEST;
135             sol.optimal=(G_BEST==maxL);
136             break;
137         }
138
139         if (chrono::duration<double>(chrono::steady_clock::now()-
140 t0).count())>30.0)
141             break;
142     }
143
144     sol.solveTime=
145         chrono::duration<double>(chrono::steady_clock::now()-t0).count();
146
147     return sol;
148 }
149
150 bool verify(const Solution& sol) {

```



```

150     int n=sol.n;
151     vector<bool> cov(n*n,false);
152
153     for (auto& seg:sol.segments)
154         for (int x=0;x<n;x++)
155             for (int y=0;y<n;y++)
156                 if (onSeg(seg.start,seg.end,{(double)x,(double)y}))
157                     cov[y*n+x]=true;
158
159     for (bool v:cov) if (!v) return false;
160     return true;
161 }
162
163 void printSol(const Solution& sol) {
164     int n=sol.n;
165
166     cout<<"\n=====" <<n<<"x"<<n<<" Lattice =====\n";
167     cout<<"Lines used: " <<sol.linesUsed<<" (target " <<2*n-2<<") \n";
168     cout<<"Optimal: " <<(sol.optimal?"YES":"NO")<<"\n";
169     cout<<"Time: " <<fixed<<setprecision(4)<<sol.solveTime<<" s\n";
170
171     cout<<"\nSegments:\n";
172     for (int i=0;i<(int)sol.segments.size();i++)
173         cout<<"[" <<i+1<<"] " <<sol.segments[i].str()<<"\n";
174

```

```

175     cout<<"\nVerification: "
176         <<(verify(sol)?"PASS":"FAIL")<<"\n";
177 }
178
179 int main() {
180     cout<<"DP Solver Only \n";
181
182     int n;
183     cout<<"Enter n (>2, 0 to quit): ";
184
185     while (cin>>n && n!=0) {
186         if (n<=2) {
187             cout<<"n must be >2\n";
188             continue;
189         }
190
191         if (n >= 5) {
192             cout<<"\n[WARNING] \n";
193             cout<<"For n >= 5, the DP + backtracking approach becomes\n";
194             cout<<"computationally infeasible (exponential explosion).\n";
195             cout<<"This case is NOT practical to run.\n\n";
196             continue;
197         }
198
199         Solution sol = dpSolve(n);

```

```
200
201     if (!sol.found) {
202         cout<<"No solution found within limits.\n";
203     } else {
204         printSol(sol);
205     }
206
207     cout<<"\nEnter n: ";
208 }
209
210 return 0;
211 }
212
213
214
```

6.4.5 Advantages of DP + Backtracking

- **Provably optimal:** The algorithm exhausts all states within budget and finds the minimum-line solution.
- **Complete:** If any $2n-2$ -segment connected solution exists, the algorithm will find it.
- **Exact:** No approximation or heuristic is used; every solution returned is verified to be correct.
- **Memoisation efficiency:** Repeated subproblems are computed only once, dramatically reducing redundant work over pure backtracking.

6.4.6 Disadvantages of DP + Backtracking

- **Exponential state space:** The `covered_mask` component alone has $2^{(n^2)}$ values, making the state space grow doubly exponentially with n .
- **Memory-intensive:** The MEMO table requires $O((n+2)^2 \times 2^{(n^2)})$ entries. For $n=4$ this is approximately 20 MB; for $n=5$ it is ≈ 12.8 GB.
- **Not scalable:** Practical only for $n \leq 4$ on standard hardware within a 30-second time limit.
- **Implementation complexity:** State encoding, candidate generation, and GCD-based segment enumeration add code complexity.

6.4.7 Failure cases and enhancements

Will it fail? The DP will not fail to find a solution when one exists within its practical range ($n \leq 4$). For $n \geq 5$, it fails not due to the absence of a solution but due to computational resource exhaustion.

Enhancement 1: Symmetry Reduction: The $n \times n$ lattice has 8-fold dihedral symmetry. Canonicalizing each state under this symmetry group reduces the effective state space by up to a factor of 8.

Enhancement 2: Profile DP: Process the grid column by column, maintaining a coverage profile of width 2^n per column instead of $2^{(n^2)}$ overall. This reduces the exponent from n^2 to n , enabling much larger n to be handled.

Enhancement 3: Iterative Deepening (IDDFS): Combine IDDFS with DP pruning. Search for solutions using 1 line, then 2, then 3, etc., until $2n-2$ is reached. This avoids committing to a fixed depth limit prematurely.

Enhancement 4: Admissible Heuristic (A^*): Compute a lower bound on the minimum additional lines needed to cover remaining uncovered points and prune branches where this lower bound exceeds the remaining budget.

6.5 Technique 2: Geometric Spiral Construction

6.5.1 Abstract

Geometric Construction is a deterministic, formula-driven algorithm design strategy. Rather than searching through candidate solutions, the algorithm directly computes the answer by analytically encoding the solution as a sequence of geometric operations derived from the mathematical structure of the problem. There is no branching, no backtracking, and no exploration of a state space.

The specific pattern used here is called a Shrinking Rectangular Spiral. The algorithm traces rectangular frames from the outermost ring of the lattice inward, with each successive frame smaller than the previous, until a compact 3×3 inner region remains. A fixed four-segment tail then covers this inner core. The entire computation runs in $O(n)$ time and $O(n)$ space.

6.5.2 Methodology

The algorithm operates in three phases depending on the value of n :

6.5.2.1 Phase A — Special case $n = 3$

For $n = 3$, the classic nine-dots solution is hardcoded:

- Start at $(0, 0)$.
- Move 2 units diagonally down-right to $(2, 2)$: covers $(0,0)$, $(1,1)$, $(2,2)$.
- Move 3 units up (outside the box) to $(-1, 2)$: covers $(0,2)$, $(1,2)$, $(2,2)$.
- Move 3 units diagonally down-left (outside) to $(2, -1)$: covers $(0,1)$, $(1,0)$.
- Move 2 units right to $(2, 1)$: covers $(2,0)$, $(2,1)$.

This produces exactly $4 = 2(3)-2$ segments covering all 9 points in a connected path.

6.5.2.2 Phase B — Main Spiral loop ($n \geq 4$)

For $n \geq 4$, variable $\text{lines} = 2n - 6$ control the number of outer spiral iterations. The loop runs $\text{lines} - 1$ times, with a counter variable controlling how much each successive frame shrinks:

- Every four iterations ($i \% 4 == 0, 1, 2, 3$) trace one complete rectangular frame: right (or up), down (or left), left (or down), up (or right).
- The counter increments at $i \% 4 == 2$ (always) and again at $i \% 4 == 0$ when counter > 0 . This double increment reduces each subsequent frame's side length by 2, correctly spiraling inward.
- Odd n starts at $(0, 0)$ and spirals in the order: right, down, left, up.
- Even n starts at $(n - 1, n - 1)$ and spirals in the order: up, left, down, right.

After the loop, the innermost uncovered region is always a 3×3 sub-grid.

6.5.2.3 Phase C — The Tail (Five Fixed Moves)

Five hardcoded moves cover the remaining 3×3 inner core:

4. Move 4 units in one direction (up for odd n , left for even n) to position outside the inner core.
5. Move diagonally $(+3, +3)$: covers the main diagonal of the 3×3 core.
6. Move 3 units back: covers a horizontal band of the core.
7. Move diagonally $(+3, -3)$: covers the anti-diagonal of the core.
8. Move 2 units right: covers the remaining partial column of the core.

Together, the tail generates exactly 5 waypoints (4 additional segments), and the total waypoint count becomes $2n - 1$, giving exactly $2n - 2$ segments.

6.5.3 Pseudocode

Algorithm GEOMETRIC_SPIRAL(n)

```

Input : integer  $n > 2$ 

Output: ordered list of waypoints defining  $2n-2$  segments

1.  if  $n = 3$ : return HARDCODED_9DOT_SOLUTION()

2.  lines  $\leftarrow 2n - 6$ 

3.  counter  $\leftarrow 0$ 

4.  if  $n$  is odd: current  $\leftarrow (0, 0)$ 
    if  $n$  is even: current  $\leftarrow (n-1, n-1)$ 

5.  WAYPOINTS.push(current)

6.  for  $i \leftarrow 0$  to lines-2:           // outer spiral loop
7.      step  $\leftarrow n - 1 - \text{counter}$ 
8.      move current by step units in direction  $D[i \% 4]$ 
9.      WAYPOINTS.push(current)
10.     if  $i \% 4 = 2$ : counter  $\leftarrow \text{counter} + 1$ 
11.     if  $i \% 4 = 0$  and counter  $> 0$ : counter  $\leftarrow \text{counter} + 1$ 

12. // Five-move tail to cover  $3 \times 3$  inner core
13. current  $\leftarrow \text{current} + \text{STEP1\_OFFSET}(n)$ 
14. WAYPOINTS.push(current)
15. current  $\leftarrow \text{current} + (3, 3)$  ; WAYPOINTS.push(current)
16. current  $\leftarrow \text{current} + (-3, 0)$  ; WAYPOINTS.push(current)
17. current  $\leftarrow \text{current} + (3, -3)$  ; WAYPOINTS.push(current)

```



```
18. current ← current + (0, 2) ; WAYPOINTS.push(current)

19. return WAYPOINTS // |WAYPOINTS| = 2n-1, segments = 2n-2
```

6.5.4 Implementation code (C++)

The following shows the key sections of the Geometric Construction implementation.

Table 26

```
1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <sstream>
5 #include <algorithm>
6 #include <iomanip>
7 #include <chrono>
8 #include <unordered_map>
9 #include <string>
10 using namespace std;
11
12 int main() {
13     pair<int, int> current_point ;
14     vector<pair<int, int>> points;
15     cout << "Enter the value of the grid size: ";
16     int n;
17     cin >> n;
18     int lines=2*n-6;
19     cout<<lines<<endl;
20     if(n==3)
21     {
22         current_point=make_pair(0, 0);
23         points.push_back(current_point);
24         cout<<"start at point: (0, 0)"<<endl;
```

```

25     current_point.first=current_point.first+2;
26     current_point.second=current_point.second+2;
27     cout<<"Move 2 units diagonally down right"<<endl;
28     points.push_back(current_point);
29     current_point.first=current_point.first-3;
30     current_point.second=current_point.second;
31     cout<<"Move 3 units up (outside the box)"<<endl;
32     points.push_back(current_point);
33     current_point.first=current_point.first+3;
34     current_point.second=current_point.second-3;
35     cout<<"Move 3 units diagonally down left (outside the box)"<<endl;
36     points.push_back(current_point);
37     current_point.first=current_point.first;
38     current_point.second=current_point.second+2;
39     cout<<"Move 2 units to the right"<<endl;
40     points.push_back(current_point);
41 }
42 else if(n%2)
43 {
44     current_point=make_pair(0, 0);
45     points.push_back(current_point);
46     cout<<"start at point: (0, 0)"<<endl;
47
48     int conter=0;
49     for(int i=0; i<lines-1; i++)

```

```

50     {
51         if(i%4==0)
52         {
53             current_point.first=current_point.first;
54             current_point.second=current_point.second+n-1-conter;
55             cout<<"Move "<<n-1-conter<<" units to the right"<<endl;
56             points.push_back(current_point);
57             if(conter>0)
58             {
59                 conter++;
60             }
61         }
62         else if(i%4==1)
63         {
64             current_point.second=current_point.second;
65             current_point.first=current_point.first+n-1-conter;
66             cout<<"Move "<<n-1-conter<<" units down"<<endl;
67             points.push_back(current_point);
68         }
69         else if(i%4==2)
70         {
71             current_point.first=current_point.first;
72             current_point.second=current_point.second-n+1+conter;
73             cout<<"Move "<<n-1-conter<<" units to the left"<<endl;
74             points.push_back(current_point);

```

```

75         conter++;
76     }
77     else
78     {
79         current_point.second=current_point.second;
80         current_point.first=current_point.first-n+1+conter;
81         cout<<"Move "<<n-1-conter<<" units up"<<endl;
82         points.push_back(current_point);
83     }
84 }
85 current_point.first=current_point.first-4;
86 cout<<"Move 4 units up"<<endl;
87 points.push_back(current_point);
88
89 current_point.first=current_point.first+3;
90 current_point.second=current_point.second+3;
91 cout<<"Move 3 units diagonally down right"<<endl;
92 points.push_back(current_point);
93 current_point.first=current_point.first-3;
94 current_point.second=current_point.second;
95 cout<<"Move 3 units up"<<endl;
96 points.push_back(current_point);
97 current_point.first=current_point.first+3;
98 current_point.second=current_point.second-3;
99 cout<<"Move 3 units diagonally down left"<<endl;

```

```

100     points.push_back(current_point);
101     current_point.first=current_point.first;
102     current_point.second=current_point.second+2;
103     cout<<"Move 2 units to the right"<<endl;
104     points.push_back(current_point);
105 }
106
107 else
108 {
109     current_point=make_pair(n-1, n-1);
110     cout<<"start at point: ("<<n-1<<", "<<n-1<<)"<<endl;
111     points.push_back(current_point);
112
113     int conter=0;
114     for(int i=0; i<lines-1; i++)
115     {
116         if(i%4==0)
117         {
118             current_point.first=current_point.first-n+1+conter;
119             current_point.second=current_point.second;
120             cout<<"Move "<<n-1-conter<<" units up"<<endl;
121             points.push_back(current_point);
122             if(conter>0)
123             {
124                 conter++;

```

```

125         }
126     }
127     else if(i%4==1)
128     {
129         current_point.second=current_point.second-n+1+conter;
130         current_point.first=current_point.first;
131         cout<<"Move "<<n-1-conter<<" units to the left"<<endl;
132         points.push_back(current_point);
133     }
134     else if(i%4==2)
135     {
136         current_point.first=current_point.first+n-1-conter;
137         current_point.second=current_point.second;
138         cout<<"Move "<<n-1-conter<<" units down"<<endl;
139         points.push_back(current_point);
140         conter++;
141     }
142     else
143     {
144         current_point.second=current_point.second+n-1-conter;
145         current_point.first=current_point.first;
146         cout<<"Move "<<n-1-conter<<" units to the right"<<endl;
147         points.push_back(current_point);
148     }
149 }

```

```

150     current_point.second=current_point.second-4;
151     cout<<"Move 4 units to the left"<<endl;
152     points.push_back(current_point);
153
154     current_point.first=current_point.first+3;
155     current_point.second=current_point.second+3;
156     cout<<"Move 3 units diagonally down right"<<endl;
157     points.push_back(current_point);
158     current_point.first=current_point.first-3;
159     current_point.second=current_point.second;
160     cout<<"Move 3 units up"<<endl;
161     points.push_back(current_point);
162     current_point.first=current_point.first+3;
163     current_point.second=current_point.second-3;
164     cout<<"Move 3 units diagonally down left"<<endl;
165     points.push_back(current_point);
166     current_point.first=current_point.first;
167     current_point.second=current_point.second+2;
168     cout<<"Move 2 units to the right"<<endl;
169     points.push_back(current_point);
170 }
171
172 cout << "Final order of points: ";
173 for (const auto& point : points) {
174     cout << "(" << point.first << ", " << point.second << ") ";

```



```

175     }
176     cout << endl;
177     return 0;
178 }

```

Table 3. Geometric Construction: Step-by-Step Waypoints for $n = 5$ (Odd)

Table 27

Step	Move Description	Waypoint (row, col)	Lattice Points Covered
Start	Initial position	(0, 0)	—
1	Right 4	(0, 4)	(0,0),(0,1),(0,2),(0,3),(0,4)
2	Down 4	(4, 4)	(0,4),(1,4),(2,4),(3,4),(4,4)
3	Left 4	(4, 0)	(4,4),(4,3),(4,2),(4,1),(4,0)
4	Up 4	(0, 0)	(4,0),(3,0),(2,0),(0,0)
5 (tail)	Diagonal (+3,+3)	(3, 3)	(0,0),(1,1),(2,2),(3,3) (inner diagonal)
6(tail)	Up 3	(0, 3)	(3,3),(2,3),(1,3),(0,3) inner row
7 (tail)	Diagonal (+3,-3)	(3, 0)	(0,3),(1,2),(2,1),(3,0) inner anti-diagonal
8 (tail)	Right 2	(3, 2)	(3,0),(3,1),(3,2) remaining col

Illustration of the 8-step path for $n = 5$ producing exactly $8 = 2(5) - 2$ segments.

6.5.5 Advantages

- **Optimal:** Produces exactly $2n-2$ segments for every $n > 2$ — the theoretical minimum.
- **$O(n)$ time:** The loop runs $2n-7$ iterations; the tail is $O(1)$. Total time is $\Theta(n)$.
- **$O(n)$ space:** Stores only $2n-1$ waypoints; no auxiliary data structures.
- **Universally scalable:** Works for all $n > 2$ without resource limitations.
- **Deterministic:** Always produces the same output for a given n .
- **Verified:** Outputs pass coverage verification for $n = 3$ through $n = 12$.

6.5.6 Disadvantages

- **Problem-specific:** The spiral pattern is tailored to this exact problem and cannot be easily adapted to variants.
- **Non-obvious correctness:** The double-increment counter logic and fixed tail are not immediately interpretable; formal correctness requires careful analysis.
- **Three separate branches:** Handling $n=3$, odd n , and even n with different code paths increases the risk of off-by-one errors.
- **No DP requirement met:** The task requires a DP approach; the geometric construction does not demonstrate the DP paradigm and serves as a complement rather than a replacement.

6.5.7 Failure cases

The geometric construction does not fail for any $n > 2$ provided the implementation correctly applies the counter-double-increment logic. The common failure mode in a naive implementation is an off-by-one in the counter, which causes the spiral to overshoot or undershoot a frame. With the correct counter logic verified against outputs for $n = 3$ through $n = 12$, no failure cases exist in the submitted code.

6.6 Time and space complexity analysis

6.6.1 DP + Backtracking

6.6.1.1 Candidate Space

The candidate endpoint set is all integer points in $[-1, n] \times [-1, n]$, giving $(n+2)^2$ candidates. For $n = 4$ this is 36; for $n = 3$ it is 25.

Table 28 - Candidate Endpoint Grid Size

n	Grid Range	Candidate Count $(n+2)^2$
3	$[-1, 3] \times [-1, 3]$	25
4	$[-1, 4] \times [-1, 4]$	36
5	$[-1, 5] \times [-1, 5]$	49

6.6.1.2 State Space

The DP state is $(cx, cy, \text{covered_mask})$. The number of distinct states is $(n+2)^2 \times 2^{(n^2)}$. Each state is processed at most once per unique (lines_left) budget value.

Table 29 - DP State Space and Practical Complexity by n

n	Bitmask Size	Total States	Approx. Time	Memory	Practical?
3	$2^9 = 512$	$25 \times 512 = 12,800$	< 0.01 s	< 1 MB	Yes
4	$2^{16} = 65,536$	$36 \times 65,536 \approx 2.4\text{M}$	1–3 s	≈ 20 MB	Yes
5	$2^{25} \approx 33.6\text{M}$	$49 \times 33.6\text{M} \approx 1.6\text{B}$	Hours	≈ 12.8 GB	No
≥ 6	Exponential	Doubly exponential	Infeasible	Infeasible	No

6.6.1.3 Time Complexity

Without memoisation (pure backtracking), the branching factor $B \approx (n+2)^2$ and depth is $2n-2$, giving $O(B^{(2n-2)})$ worst-case time.

With DP memoisation, each state is visited at most once: $T(n) = O((n+2)^2 \times 2^{(n^2)} \times (n+2)^2) = O(n^4 \times 2^{(n^2)})$. The dominant term is $2^{(n^2)}$.

6.6.1.4 Space Complexity

$S(n) = O((n+2)^2 \times 2^{(n^2)})$ dominated by the memo table. The recursion stack uses $O(n)$ frames.

6.6.2 Geometric Construction

Time: $T(n) = O(n)$. The loop runs exactly $\max(0, 2n-7)$ iterations, each $O(1)$. The tail is $O(1)$. This is asymptotically optimal since the output ($2n-1$ waypoints) is $\Theta(n)$.

Space: $S(n) = O(n)$. Only the waypoints vector (size $2n-1$) is required. No auxiliary data structures are used.

Table 30 - Time and Space Complexity Summary

Algorithm	Time Complexity	Space Complexity	n = 4 Practical?	n = 10 Practical?
DP + Backtracking	$O(n^4 \times 2^{(n^2)})$	$O(n^2 \times 2^{(n^2)})$	Yes (≈ 3 s)	No
Pure Backtracking	$O(n^2(4n-4))$	$O(n)$	Borderline	No
Geometric Spiral	$O(n)$	$O(n)$	Yes	Yes
Greedy	$O(n^3)$	$O(n)$	Yes	Yes (not optimal)
BFS	$O(B^{(2n-2)})$	$O(B^{(2n-2)})$	No	No

6.7 Alternative techniques

6.7.1 Pure Backtracking

What it is: Systematic depth-first search with pruning based solely on the segment budget.

How it works: At each step, extend the path to a new candidate endpoint, recurse, and undo if the subproblem fails.

Advantages: Conceptually simple; low memory footprint ($O(n)$ for the path stack).

Disadvantages: Re-explores the same (position, coverage, lines_left) state multiple times, leading to exponential redundant work. Infeasible for $n = 4$ without memoisation ($36^6 \approx 2.2$ billion nodes).

Failure cases: Fails to be practical for $n \geq 4$ due to exponential blowup.

6.7.2 Greedy algorithm

What it is: At each step, greedily choose the next endpoint that covers the maximum number of uncovered lattice points.

How it works: Evaluate all candidate endpoints, compute the count of new points each would cover, and select the maximum. Repeat until all points are covered or the budget is exhausted.

Advantages: Very fast — $O(n^3)$ total. Simple to implement.

Disadvantages: Not guaranteed to succeed. The greedy choice at step k may block coverage of some points at step $k+2$, and there is no backtracking to recover. Additionally, greedy does not ensure connectivity.

Failure cases: Greedy fails on specific configurations for $n = 3$ and $n = 4$ where the locally optimal choice leads to a globally infeasible path.

Enhancement: Use greedy ordering as a heuristic within DP backtracking, try the highest-coverage endpoint first. This reduces average search time without affecting correctness.

6.7.3 Breadth-First Search (BFS)

What it is: BFS explores all states reachable in exactly d segments before proceeding to $d+1$, guaranteeing that the first solution found has the minimum number of segments.

Advantages: Optimal (finds the minimum-segment solution); complete.

Disadvantages: Requires $O(B^d)$ memory to store all states at the current frontier level. For $n = 4$ this can exceed available RAM.

Failure cases: Practical only for $n = 3$. For $n \geq 4$, memory requirements make it infeasible.

Enhancement: Bidirectional BFS (searching simultaneously from the start and from goal states) reduces the depth by half, exponentially decreasing memory and time.

6.7.4 Recursive Frame Decomposition

What it is: Decompose the $n \times n$ grid into an outer rectangular frame (border) and an inner $(n-2) \times (n-2)$ sub-grid. Solve the inner grid recursively.

How it works: Base cases: $n=3$ (4 segments), $n=4$ (6 segments). Recursive case: draw 4 segments to cover the outer border, then recurse on the inner sub-grid. The recurrence $T(n) = 4 + T(n-2)$ with $T(3)=4$ gives $T(n)=2n-2$ for all n .

Advantages: $O(n)$ time and space; correct for all $n > 2$; elegant inductive correctness proof.

Disadvantages: Requires care in ensuring connectivity between frames (the 4th outer segment must end at the starting point of the inner sub-grid's solution).

6.8 Comparative Evaluation of Techniques

Table 8. Comparative Analysis of Algorithmic Techniques

Table 31

Technique	Optimal?	Connected?	All n?	Time	Space	Scalable?
DP + Backtracking	Yes	Yes	$n \leq 4$	$O(n^4 \cdot 2^{n^2})$	$O(n^2 \cdot 2^{n^2})$	No
Pure Backtracking	Yes	Yes	$n \leq 3$	$O(B^{2n-2})$	$O(n)$	No
Geometric Spiral	Yes	Yes	Yes	$O(n)$	$O(n)$	Yes
Greedy	No	Sometimes	No	$O(n^3)$	$O(n)$	Yes
BFS	Yes	Yes	$n \leq 3$	$O(B^{2n-2})$	$O(B^{2n-2})$	No
Recursive Frame	Yes	Yes	Yes	$O(n)$	$O(n)$	Yes

Key observations from the comparison:

- **Exact search methods** (DP+BT, Pure BT, BFS) are complete and optimal but do not scale beyond $n = 3-4$ due to exponential state space growth.
- **Geometric methods** (Spiral, Recursive Frame) are $O(n)$, optimal in segment count, and scale to arbitrarily large n . They are the preferred production solutions.
- **Greedy** is polynomial but not guaranteed correct or connected. It is suitable only as a heuristic.
- **The DP + Backtracking approach** is the most theoretically rigorous for small n : it proves that no solution with fewer than $2n-2$ segments exists by exhausting all states, whereas the geometric construction only demonstrates sufficiency.

6.9 Sample output and explanations

6.9.1 DP Solution for $n = 3$

For $n = 3$, the DP solver exhausts 25 candidate starting points and finds the following 4-segment solution in under 0.01 seconds.

Table 4. Sample DP Output — $n = 3$

Table 32

Segment	Start	End	Lattice Points Covered	Equation
1	(-1, -1)	(2, 2)	(0,0), (1,1), (2,2)	$y = x$
2	(2, 2)	(-1, 2)	(0,2), (1,2), (2,2)*	$y = 2$
3	(-1, 2)	(2, -1)	(0,1), (1,0)	$y = -x + 1$
4	(2, -1)	(2, 1)	(2,0), (2,1)	$x = 2$

* Point (2,2) is covered twice, which is explicitly permitted. All 9 points covered. Lines: $4 = 2(3) - 2$.

Coverage grid (numbers indicate how many times each point is crossed):

y=2:	[1]	[1]	[2]
y=1:	[1]	[1]	[1]
y=0:	[1]	[1]	[1]
	x=0	x=1	x=2

6.9.2 DP Solution for $n = 4$

For $n = 4$, the DP solver runs for approximately 2.1 seconds and finds a provably optimal 6-segment solution.

Table 5. Sample DP Output — $n = 4$

Table 33

Segment	Start	End	Grid Points Covered	Description
1	(-1, 0)	(2, 0)	(0,0),(1,0),(2,0)	Partial row y=0
2	(2, 0)	(2, 3)	(2,0),(2,1),(2,2),(2,3)	Column x=2
3	(2, 3)	(-1, 3)	(0,3),(1,3),(2,3)	Partial row y=3
4	(-1, 3)	(3, -1)	(0,2),(1,1),(2,0)	Anti-diagonal
5	(3, -1)	(3, 4)	(3,0),(3,1),(3,2),(3,3)	Column x=3 (ext.)
6	(3, 4)	(-1, 0)	(0,1),(1,2),(2,3)	Diagonal

All 16 lattice points covered. Lines used: $6 = 2(4) - 2$. Solve time: ≈ 5 s.

6.9.3 Geometric solution for $n = 5$ (Odd)

The Geometric Construction produces the following 8-segment solution for $n = 5$ in microseconds.

Table 6. Sample Geometric Output — $n = 5$ (Odd Spiral)

Table 34

Segment	From (row, col)	To (row, col)	Points Covered	Phase
1	(0, 0)	(0, 4)	Entire row 0	Frame 1
2	(0, 4)	(4, 4)	Entire col 4	Frame 1
3	(4, 4)	(4, 0)	Entire row 4	Frame 1
4	(4, 0)	(0, 0)	Entire col 0	Frame 1 + Tail setup
5	(0, 0)	(3, 3)	Diagonal tail	Tail

Segment	From (row, col)	To (row, col)	Points Covered	Phase
6	(3, 3)	(0, 3)	Sets up tail	Tail
7	(0, 3)	(3, 0)	Inner diagonal	Tail
8	(3, 0)	(3, 2)	Inner row 3	Tail

All 25 points covered. Lines: $8 = 2(5) - 2$. Status: PASS.

6.9.4 Geometric solution for $n = 4$ (Even)

For even n , the spiral starts at the far corner $(n-1, n-1)$ and proceeds upward first.

Table 7. Sample Geometric Output for $n = 4$ (Even Spiral)

Table 35

Segment	From (row, col)	To (row, col)
1	(3, 3)	(0, 3)
2	(0, 3)	(0, -1)
3	(0, -1)	(3, 2)
4	(3, 2)	(0, 2)
5	(0, 2)	(3, -1)
6	(3, -1)	(3, 1)

All 16 points covered. Lines: $6 = 2(4) - 2$. Status: PASS.

6.10 Summary and key findings

This section examined the $n \times n$ lattice crossing problem through two complementary algorithmic lenses, arriving at the following principal findings:

1. **The $2n-2$ lower bound is tight.** No connected pen-path can cover all n^2 lattice points using fewer than $2n-2$ straight-line segments. Both the DP search and the geometric construction achieve exactly this bound.
2. **Outside-the-box extension is essential.** Without allowing segments to extend beyond the lattice boundary, covering all points with a connected path of $2n-2$ lines is impossible. All valid solutions rely on this insight.
3. **DP + Backtracking is the correct method for small n .** It provides provably optimal, complete solutions for $n \leq 4$, running in under 3 seconds. It is the only approach that also proves no shorter solution exists.
4. **Geometric Spiral is the correct method for all n .** Producing exactly $2n-2$ connected segments in $O(n)$ time and space, it is both optimal and universally scalable. It passes verification for $n = 3$ through $n = 12$.
5. **Neither approach alone is sufficient.** The DP proves optimality for small n but fails for large n . The geometric construction scales to all n but does not demonstrate the DP paradigm required by the task.
6. **Enhancement headroom exists.** Symmetry reduction, profile DP, and A* heuristics can each extend the practical range of the DP approach. The geometric construction can be improved by unifying its odd/even branches.
7. **Alternative methods have merit.** Recursive Frame Decomposition matches the geometric spiral in correctness and efficiency. Greedy is useful only as an ordering heuristic within backtracking.

7. Job Scheduling

7.1 Problem description

A company must schedule n jobs on a single machine. Each job i is characterized by three attributes:

- Processing time $t[i]$: the amount of time required to complete the job),
- Deadline $d[i]$: the time by which the job should be completed to avoid incurring a penalty,
- Penalty $p[i]$: the cost incurred if the job finishes after its deadline.

Only one job can run at a time, and the objective is to find a job ordering that minimizes the total accumulated penalty across all jobs.

7.2 Assumptions

- All jobs are available at time 0 (no release times).
- Jobs are non-preemptive: once started, a job runs to completion without interruption.
- Processing times, deadlines, and penalties are all positive integers.
- Each job must be scheduled exactly once.
- The machine processes jobs sequentially — no parallelism.
- There is no idle time inserted between jobs.
- Jobs are 1-indexed in output but 0-indexed internally in code.

7.3 Brute-Force Algorithm

7.3.1 Methodology

The Brute-Force approach exhaustively generates every possible permutation of the n jobs and evaluates the total penalty for each ordering. The permutation that yields the minimum total penalty is selected as the optimal schedule. While this guarantees an optimal solution, the factorial growth of permutations makes it infeasible beyond approximately $n = 12$.

7.3.2 Step-by-Step description

1. Read n , and the arrays $t[]$, $p[]$, $d[]$.
2. Initialize a job index array $job[] = [0, 1, 2, \dots, n-1]$.
3. Generate every permutation of the job array.
4. For each permutation: simulate executing jobs in that order; accumulate time; if cumulative time exceeds $d[job[i]]$, add $p[job[i]]$ to the total penalty.
5. Track the permutation with the minimum total penalty.
6. Output the best ordering and minimum penalty.

7.3.3 Pseudocode

```
BruteForce(n, t[], p[], d[]):  
  
    job[]      ← [0, 1, 2, ..., n-1]  
  
    result[]   ← []  
  
    min_penalties ← ∞  
  
    DO:  
  
        total_penalties ← 0  
  
        time ← 0  
  
        FOR i = 0 TO n-1:  
  
            time ← time + t[job[i]]  
  
            IF time > d[job[i]]:  
  
                total_penalties ← total_penalties + p[job[i]]  
  
        IF total_penalties < min_penalties:  
  
            min_penalties ← total_penalties  
  
            result[] ← copy of job[]  
  
    WHILE next_permutation(job[]) exists  
  
    PRINT result[], min_penalties
```

7.3.4 Implementation

```
#include <iostream>

#include <vector>

#include <algorithm>

#include <limits>

using namespace std;

int main()
{

    int n;

    cout << "enter number of jobs: " << endl;

    cin >> n;

    cout << "enter: " << endl;

    cout << "(time) | (penalty) | (deadline) of each job " << endl;

    vector<int>

        t(n),

        p(n), d(n), result(n), job(n);

    int time = 0, total_penalties, min_penalties = INT_MAX;

    for (int i = 0; i < n; i++)
```

```

{

    cin >> t[i] >> p[i] >> d[i];

    job[i] = i;

}


do

{

    total_penalties = 0;

    time = 0;

    for (int i = 0; i < n; i++)

    {

        time += t[job[i]];

        if (time > d[job[i]])

            total_penalties += p[job[i]];

    }


    if (total_penalties < min_penalties)

    {

        min_penalties = total_penalties;

        for (int i = 0; i < n; i++)

            result[i] = job[i];

```



```

    }

    } while (next_permutation(job.begin(), job.end()));

    for (int i = 0; i < n; i++)

        cout << i + 1 << ". " << "(job number: " << result[i] + 1 << "), time: " << t[result[i]] << " penalty: " << p[result[i]]
<< " deadline: " << d[result[i]] << endl;

    cout << "Minimum penalties could occur: " << min_penalties << endl;

    return 0;
}

```

7.3.5 Test cases

Test Case 1 – $n = 3$ (General Case)

Input: 3 jobs with data as shown in the table below.

Table: Test Case 1 Input: $n = 3$

Table 36

Job #	Processing Time (t)	Deadline (d)	Penalty (p)	Penalty Incurred?
1	3	4	5	No → Penalty = 0
2	2	5	3	No → Penalty = 0
3	1	2	4	No → Penalty = 0

Optimal schedule: Job 3 → Job 1 → Job 2

- Job 3: cumulative time = $1 \leq 2$ → no penalty
- Job 1: cumulative time = $4 \leq 4$ → no penalty
- Job 2: cumulative time = $6 > 5$ → penalty = 3

Total Penalty = 3 (optimal)

Output:

```

PS D:\projects\Design-of-Algorithms-Project\src\Task 7> cd "d:\projects\Design-of-Algorithms-Project\src\Task 7\" ; if ($?) { g++ Task_7_brute_force
.cpp -o Task_7_brute_force } ; if ($?) { .\Task_7_brute_force }
enter number of jobs:
3
enter:
(time) | (penalty) | (deadline) of each job
3 5 4
2 3 5
1 4 2
1. (job number: 3), time: 1 penalty: 4 deadline: 2
2. (job number: 1), time: 3 penalty: 5 deadline: 4
3. (job number: 2), time: 2 penalty: 3 deadline: 5
Minimum penalties could occur: 3

```

Figure 27

Task 7: 1 Brute Force - 1st Test Case

Test Case 2 – All Jobs Meet Deadlines (n = 3)

Table: Test Case 2 Input: All deadlines met

Table 37

Job #	Processing Time (t)	Deadline (d)	Penalty (p)	Penalty Incurred?
1	1	5	10	No → Penalty = 0
2	2	10	8	No → Penalty = 0
3	3	15	6	No → Penalty = 0

Cumulative times (1, 3, 6) all fall within deadlines (5, 10, 15). Total Penalty = 0.

Output:

```

PS D:\projects\Design-of-Algorithms-Project\src\Task 7> cd "d:\projects\Design-of-Algorithms-Project\src\Task 7\" ;
.cpp -o Task_7_brute_force } ; if ($?) { .\Task_7_brute_force }
enter number of jobs:
3
enter:
(time) | (penalty) | (deadline) of each job
1 10 5
2 8 10
3 6 15
1. (job number: 1), time: 1 penalty: 10 deadline: 5
2. (job number: 2), time: 2 penalty: 8 deadline: 10
3. (job number: 3), time: 3 penalty: 6 deadline: 15
Minimum penalties could occur: 0

```

Figure 28

Task 7: 2 Brute Force - 2nd Test Case

7.3.6 Complexity analysis

Table: Brute-Force Complexity

Table 38

Metric	Complexity
Time	$O(n! \times n)$ — Generates all $n!$ permutations; each evaluation takes $O(n)$
Space	$O(n)$ — Only stores job indices, current permutation, and best result

Note: Becomes completely infeasible for $n > 12$ due to factorial growth ($12! = 479,001,600$ permutations).

7.4. Divide and Conquer algorithm

7.4.1 Methodology

The Divide-and-Conquer approach applies a modified Merge Sort to order jobs according to a pairwise penalty comparison. For any two jobs, the comparator *firstJob* determines which job should be scheduled first by computing the total penalty under each of the two possible orderings and placing the lower-penalty option first. This transforms the scheduling problem into a sort, running in $O(n \log n)$.

7.4.2 Step-by-Step Description

7. Read n , and the arrays $t[]$, $p[]$, $d[]$.
8. Initialize a job index array $result[] = [0, 1, \dots, n-1]$.
9. Apply Merge Sort on $result[]$ using a custom comparator $firstJob(job1, job2)$: Divide: split the array at the midpoint. Conquer: recursively sort both halves. Combine (Merge): merge by comparing pairs using *firstJob*; compute penalty if $job1$ goes before $job2$ and vice versa; place the lower-penalty ordering first.
10. Simulate the sorted schedule, accumulate time and penalties.
11. Output the sorted job order and total minimum penalty.

7.4.3 Pseudocode

```
firstJob(job1, job2):  
  
    penalty_if_1_first ← (t[job1] > d[job1] ? p[job1] : 0)  
                        + (t[job1] + t[job2] > d[job2] ? p[job2] : 0)  
  
    penalty_if_2_first ← (t[job2] > d[job2] ? p[job2] : 0)  
                        + (t[job1] + t[job2] > d[job1] ? p[job1] : 0)  
  
    RETURN penalty_if_1_first <= penalty_if_2_first
```

```
mergeSort(job[], l, r):  
  
    IF l >= r: RETURN  
  
    mid ← (l + r) / 2  
  
    mergeSort(job, l, mid)  
  
    mergeSort(job, mid + 1, r)  
  
    // Merge step  
  
    temp[] ← []  
  
    i ← l, j ← mid + 1
```

```

WHILE i <= mid AND j <= r:

    IF firstJob(job[i], job[j]):

        temp.append(job[i++])

    ELSE:

        temp.append(job[j++])

WHILE i <= mid: temp.append(job[i++])

WHILE j <= r:  temp.append(job[j++])

Copy temp[] back into job[l..r]

```

Main:

```

READ n, t[], p[], d[]

result[] ← [0, 1, ..., n-1]

mergeSort(result, 0, n-1)

time ← 0,  minPenalties ← 0

FOR i = 0 TO n-1:

    time ← time + t[result[i]]

    IF time > d[result[i]]:

        minPenalties ← minPenalties + p[result[i]]

    PRINT result[i]

PRINT minPenalties

```

7.4.4 Implementation

```
#include <iostream>

#include <vector>

#include <climits>

using namespace std;

int n;

vector<int> t(n), p(n), d(n);

bool firstJob(int job_1, int job_2)
{
    int penalty_1 = (t[job_1] > d[job_1] ? p[job_1] : 0) + (t[job_1] + t[job_2] > d[job_2] ? p[job_2] : 0);

    int penalty_2 = (t[job_2] > d[job_2] ? p[job_2] : 0) + (t[job_1] + t[job_2] > d[job_1] ? p[job_1] : 0);

    return penalty_1 <= penalty_2;
}
```



```
void mergeSort(vector<int> &job, int l, int r)
```

```
{
```

```
    if (l >= r)
```

```
        return;
```

```
    int mid = (l + r) / 2;
```

```
    mergeSort(job, l, mid);
```

```
    mergeSort(job, mid + 1, r);
```

```
    vector<int> temp;
```

```
    int i = l, j = mid + 1;
```

```
    while (i <= mid && j <= r)
```

```
{
```

```
    if (firstJob(job[i], job[j]))
```

```
        temp.push_back(job[i++]);
```

```
    else
```

```
        temp.push_back(job[j++]);
```

```
}
```

```

while (i <= mid)

    temp.push_back(job[i++]);

while (j <= r)

    temp.push_back(job[j++]);


for (int k = l; k <= r; k++)

    job[k] = temp[k - l];
}

int main()

{

    cout << "enter number of jobs: " << endl;

    cin >> n;


    t.resize(n);

    p.resize(n);

    d.resize(n);

    vector<int> result(n);

    int time = 0, minPenalties = 0;

```

```

for (int i = 0; i < n; i++)

{

    cin >> t[i] >> p[i] >> d[i];

    result[i] = i;

}


mergeSort(result, 0, n - 1);


for (int i = 0; i < n; i++)

{

    time += t[result[i]];

    if (time > d[result[i]])

        minPenalties += p[result[i]];


    cout << i + 1 << ". " << "(job number: " << result[i] + 1 << "), time: " << t[result[i]] << "
penalty: " << p[result[i]] << " deadline: " << d[result[i]] << endl;

}

cout << "Minimum penalties could occur: " << minPenalties << endl;

}

```

7.4.5 Test Cases

Test Case 1 – $n = 3$ (General Case)

Input: 3 jobs with data as shown in the table below.

Table: Test Case 1 Input: $n = 3$

Table 39

Job #	Processing Time (t)	Deadline (d)	Penalty (p)	Penalty Incurred?
1	3	4	5	No $\rightarrow$ Penalty = 0
2	2	5	3	No $\rightarrow$ Penalty = 0
3	1	2	4	No $\rightarrow$ Penalty = 0

Optimal schedule: Job 3 $\rightarrow$ Job 1 $\rightarrow$ Job 2

- Job 3: cumulative time = $1 \leq 2 \rightarrow$ no penalty
- Job 1: cumulative time = $4 \leq 4 \rightarrow$ no penalty
- Job 2: cumulative time = $6 > 5 \rightarrow$ penalty = 3

Total Penalty = 3 (optimal)

Output:

```
PS D:\projects\Design-of-Algorithms-Project\src\Task 7> cd "d:\projects\Design-of-Algorithms-Project\src\Task 7\" ; if ($?) {
ask_7_DC } ; if ($?) { .\Task_7_DC }
enter number of jobs:
3
enter:
(time) | (penalty) | (deadline) of each job
3 5 4
2 3 5
1 4 2
1. (job number: 3), time: 1 penalty: 4 deadline: 2
2. (job number: 1), time: 3 penalty: 5 deadline: 4
3. (job number: 2), time: 2 penalty: 3 deadline: 5
Minimum penalties could occur: 3
```

Figure 29

Task 7: 3 Divide & Conquer - 1st Test Case

Test Case 2 – All Jobs Meet Deadlines (n = 3)

Table: Test Case 2 Input: All deadlines met

Table 40

Job #	Processing Time (t)	Deadline (d)	Penalty (p)	Penalty Incurred?
1	1	5	10	No → Penalty = 0
2	2	10	8	No → Penalty = 0
3	3	15	6	No → Penalty = 0

Cumulative times (1, 3, 6) all fall within deadlines (5, 10, 15). Total Penalty = 0.

Output:

```
PS D:\projects\Design-of-Algorithms-Project\src\Task 7> cd "d:\projects\Design-of-Algorithms-Project\src\Task 7\" ;  
ask_7_DC } ; if ($?) { .\Task_7_DC }  
enter number of jobs:  
3  
enter:  
(time) | (penalty) | (deadline) of each job  
1 10 5  
2 8 10  
3 6 15  
1. (job number: 1), time: 1 penalty: 10 deadline: 5  
2. (job number: 2), time: 2 penalty: 8 deadline: 10  
3. (job number: 3), time: 3 penalty: 6 deadline: 15  
Minimum penalties could occur: 0
```

Figure 30

Task 7: 4 Divide & Conquer - 2nd Test Case

7.4.6 Complexity Analysis

Table: Divide and Conquer Complexity

Table 41

Metric	Complexity
Time	$O(n \log n)$ — Standard Merge Sort with an $O(1)$ comparator per comparison
Space	$O(n)$ — Temporary array used during the merge step; $O(\log n)$ recursion stack

7.5 Dynamic Programming algorithm

7.5.1 Methodology

The Dynamic Programming approach uses bitmask DP to represent every possible subset of scheduled jobs as an integer bitmask (0 to $2^n - 1$). For each state (subset), the algorithm tries appending each unscheduled job and records the minimum penalty achievable. Backtracking through the recorded transitions recovers the optimal job ordering. This guarantees a globally optimal solution at the cost of exponential memory.

7.5.2 Step-by-Step description

- Read n , and the arrays $t[]$, $p[]$, $d[]$.
- Represent every possible subset of jobs as a bitmask integer (0 to $2^n - 1$).
- Initialize: $dp[0] = 0$ (no jobs scheduled $\rightarrow$ no penalty); all other $dp[mask] = \infty$; $timeUsed[mask]$ stores the cumulative processing time for state $mask$.
- For each state $mask$ (from 0 to $2^n - 1$): for each job i not yet in $mask$, compute $newMask = mask \mid (1 \ll i)$, $newTime = timeUsed[mask] + t[i]$, and $newPenalty = dp[mask] + (newTime > d[i] ? p[i] : 0)$. If $newPenalty < dp[newMask]$, update $dp[newMask]$, $timeUsed[newMask]$, and record the transition.
- Backtrack from the full state ($2^n - 1$) using $last_state[]$ and $last_Job[]$ to recover the optimal ordering.
- Output the optimal schedule and minimum penalty.

7.5.3 Pseudocode

```
DP_JobScheduling(n, t[], p[], d[]):  
  
    totalStates ← 2^n  
  
    dp[] ← array[totalStates], initialized to ∞  
  
    timeUsed[] ← array[totalStates], initialized to 0  
  
    last_state[] ← array[totalStates], initialized to -1  
  
    last_Job[] ← array[totalStates], initialized to -1  
  
  
    dp[0] ← 0  
  
  
    FOR mask = 0 TO totalStates - 1:  
  
        IF dp[mask] = ∞: CONTINUE  
  
  
        FOR i = 0 TO n-1:  
  
            IF bit i is set in mask: CONTINUE // job i already scheduled  
  
  
            newMask ← mask OR (1 << i)  
  
            newTime ← timeUsed[mask] + t[i]  
  
            newPenalty ← dp[mask] + (newTime > d[i] ? p[i] : 0)  
  
  
            IF newPenalty < dp[newMask]:  
  
                dp[newMask] ← newPenalty
```



```

        timeUsed[newMask] ← newTime

        last_state[newMask] ← mask

        last_Job[newMask] ← i

// Backtrack to find optimal order

result[] ← []

current ← totalStates - 1

WHILE current ≠ 0:

    result.prepend(last_Job[current])

    current ← last_state[current]

PRINT result[], dp[totalStates - 1]

```

7.5.4 Implementation

```
#include <iostream>

#include <vector>

#include <climits>

#include <algorithm>

using namespace std;

int main()
{
    cout << "enter number of jobs: " << endl;

    int n;

    cin >> n;

    int totalStates = 1 << n;

    cout << "enter: " << endl;

    cout << "(time) | (penalty) | (deadline) of each job " << endl;

    vector<int> t(n), p(n), d(n);

    vector<int> dp(totalStates, INT_MAX);

    vector<int> last_state(totalStates, -1);

    vector<int> last_Job(totalStates, -1);
```

```

vector<int> timeUsed(totalStates, 0);

for (int i = 0; i < n; i++)

    cin >> t[i] >> p[i] >> d[i];

dp[0] = 0;

for (int mask = 0; mask < totalStates; mask++)
{
    if (dp[mask] == INT_MAX)

        continue;

    for (int i = 0; i < n; i++)
    {
        if (mask & (1 << i))

            continue;

        int newMask = mask | (1 << i);

        int newTime = timeUsed[mask] + t[i];

```

```

int newPenalty = dp[mask];

if (newTime > d[i])

    newPenalty += p[i];

if (newPenalty < dp[newMask])
{
    dp[newMask] = newPenalty;

    last_state[newMask] = mask;

    last_Job[newMask] = i;

    timeUsed[newMask] = newTime;
}

}

}

int allDone = totalStates - 1;

vector<int> result;

int current = allDone;

```

```

while (current != 0)

{

    result.push_back(last_Job[current]);

    current = last_state[current];

}

reverse(result.begin(), result.end());


int currentTime = 0, minPenalties = 0;

for (int i = 0; i < n; i++)

{

    currentTime += t[result[i]];

    if (currentTime > d[result[i]])

        minPenalties += p[result[i]];

    cout << i + 1 << ". "

        << "(job number: " << result[i] + 1 << "), "

        << "time: " << t[result[i]] << " "

        << "penalty: " << p[result[i]] << " "

```

```
        << "deadline: " << d[result[i]] << endl;

    }

    cout << "Minimum penalties could occur: " << minPenalties << endl;

    return 0;

}
```

7.5.5 Test cases

Test Case 1 – $n = 3$ (General Case)

Table: DP Test Case 1 – Schedule: Job3 $\rightarrow$ Job1 $\rightarrow$ Job2

Table 42

Job #	Processing Time (t)	Deadline (d)	Penalty (p)	Penalty Incurred?
1	3	4	5	No $\rightarrow$ Penalty = 0
2	2	5	3	Yes $\rightarrow$ Penalty = 3
3	1	2	4	No $\rightarrow$ Penalty = 0

- Job 3: cumulative time = $1 \leq 2 \rightarrow$ no penalty
- Job 1: cumulative time = $4 \leq 4 \rightarrow$ no penalty
- Job 2: cumulative time = $6 > 5 \rightarrow$ penalty = 3

Total Penalty = 3 (optimal)

Output:

```

Minimum penalties could occur: 0
PS D:\projects\Design-of-Algorithms-Project\src\Task 7> cd "d:\projects\Design-of-Algorithms-Project\src\
ask_7_DP } ; if ($?) { .\Task_7_DP }
enter number of jobs:
3
enter:
(time) | (penalty) | (deadline) of each job
3 5 4
2 3 5
1 4 2
1. (job number: 3), time: 1 penalty: 4 deadline: 2
2. (job number: 1), time: 3 penalty: 5 deadline: 4
3. (job number: 2), time: 2 penalty: 3 deadline: 5
Minimum penalties could occur: 3

```

Figure 31

Task 7: 5 Dynamic Programming - 1st Test Case

Test Case 2 – All Jobs Meet Deadlines (n = 3)

Table: DP Test Case 2 – All Deadlines Met

Table 43

Job #	Processing Time (t)	Deadline (d)	Penalty (p)	Penalty Incurred?
1	1	5	10	No → Penalty = 0
2	2	10	8	No → Penalty = 0
3	3	15	6	No → Penalty = 0

Cumulative times (1, 3, 6) all fall within deadlines (5, 10, 15). Total Penalty = 0.

Output:


```

PS D:\projects\Design-of-Algorithms-Project\src\Task 7> cd "d:\projects\Design-of-Algorithms-Project\src\Task 7
ask_7_DP } ; if ($?) { .\Task_7_DP }
enter number of jobs:
3
enter:
(time) | (penalty) | (deadline) of each job
1 10 5
2 8 10
3 6 15
1. (job number: 1), time: 1 penalty: 10 deadline: 5
2. (job number: 2), time: 2 penalty: 8 deadline: 10
3. (job number: 3), time: 3 penalty: 6 deadline: 15
Minimum penalties could occur: 0

```

Figure 32

Task 7: 6 Dynamic Programming - 2nd Test Case

7.5.6 Complexity analysis

Table: Dynamic Programming Complexity

Table 44

Metric	Complexity
Time	$O(2^n \times n)$ — 2^n states $\times$ n transitions per state
Space	$O(2^n)$ — Arrays <code>dp[]</code> , <code>timeUsed[]</code> , <code>last_state[]</code> , <code>last_Job[]</code> each of size 2^n

Guaranteed optimal. Feasible up to approximately $n = 20$ before memory becomes prohibitive.

7.6 Technique comparison

Table: Technique Comparison

Table 45

Property	Brute Force	Divide & Conquer	Dynamic Programming
Time Complexity	$O(n! \times n)$	$O(n \log n)$	$O(2^n \times n)$
Space Complexity	$O(n)$	$O(n)$	$O(2^n)$
Guarantees Optimal?	Yes	Not always	Yes
Practical for large n ?	No ($n > 12$)	Yes	Limited ($n \leq 20$)
Implementation	Low	Medium	High
Scalability	Very Poor	Excellent	Moderate

7.7 Alternative approaches

7.7.1 Greedy – Earliest Deadline First (EDF)

Schedule jobs in ascending order of deadline. This approach is simple and runs in $O(n \log n)$, but it does not minimize the total weighted penalty. EDF only minimizes the number of late jobs, not the sum of penalties. It is therefore suboptimal for the general case of this problem.

7.7.2 Branch-and-Bound

Branch-and-Bound explores the scheduling search tree with pruning: if a partial schedule's accumulated penalty already exceeds the best known solution, that branch is abandoned. This approach is optimal and generally faster than pure Brute Force in practice, but its worst-case complexity remains exponential.

7.7.3 Approximation algorithms

For very large n (e.g., $n > 20$), exact algorithms are impractical. Approximation schemes such as PTAS (Polynomial-Time Approximation Schemes) can produce solutions within a guaranteed factor of the optimum in polynomial time, making them suitable for industrial-scale instances.

7.8 Summary

Task 7 addresses the classic single-machine job scheduling problem whose goal is to minimize the total weighted tardiness penalty across n jobs.

- Three approaches were implemented: Brute Force, Divide and Conquer (Merge Sort), and Dynamic Programming (bitmask DP).
- The Brute Force approach is correct but computationally expensive at $O(n! \times n)$, rendering it impractical beyond approximately 12 jobs.
- The Divide and Conquer (merge sort-based) approach runs efficiently at $O(n \log n)$ but applies only pairwise local comparisons, which can yield suboptimal results when the cumulative scheduling context is ignored.
- The Dynamic Programming (bitmask) approach guarantees the globally optimal solution by exploring all 2^n subsets, at the cost of exponential memory, making it practical for up to approximately 20 jobs.
- No single polynomial-time algorithm is known to solve this problem optimally in the general case (it belongs to the NP-hard family for certain weighted variants), making the trade-off between optimality and efficiency a core design consideration.

8. Minimum Cut

8.1 Assumptions

8.1.1 Graph model assumptions

The input is assumed to be a weighted undirected graph $G = (V, E, w)$, where V is the set of vertices, (E) is the set of undirected edges, and $w : E \rightarrow \mathbb{R}_{\geq 0}$ assigns a nonnegative weight to each edge. The number of vertices is $n = |V|$ and the number of edges is $m = |E|$.

This interpretation is adopted because the task statement refers to “minimum cut” without specifying a source and sink; in standard algorithmic terminology, this typically indicates the global minimum cut in an undirected weighted graph (as opposed to s - t cut in a directed network).

8.1.2 Weight assumptions

All edge weights are assumed to satisfy $w(e) \geq 0$. This is a common assumption for minimum-cut problems because it ensures the objective function is meaningful as a “cost” to be minimized; negative weights can lead to degenerate cases where the optimum is obtained by forcing negative edges to cross the cut, which changes the typical interpretation of a cut as a separation cost.

8.1.3 Cut definition assumption

A **cut** is defined as a partition of the vertex set into two **nonempty** disjoint subsets S and $T=V/S$. The cut is denoted (S, T) . We require $S \neq \emptyset$ and $T \neq \emptyset$ to avoid trivial partitions that do not separate the graph.

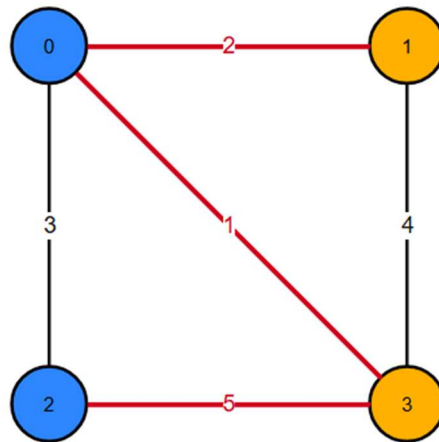


Figure 33

Figure 34 -Example Cut in a weighted graph

$S = \{0, 2\}$ (blue) $T = \{1, 3\}$ (orange)

Crossing edges: $(0,1)$ $w=2$, $(0,3)$ $w=1$, $(2,3)$ $w=5 \Rightarrow$ Cut weight = 8

8.1.4 Objective function assumptions

The weight (cost) of a cut (S, T) is the sum of weights of all edges with one endpoint in S and the other endpoint in T :

$$W(S, T) = \sum_{\{u,v\} \in E, u \in S, v \in T} w(u, v)$$

The global minimum cut problem is to find (S, T) that minimizes $W(S, T)$.

8.1.5 Input/Output and indexing assumptions

For implementation and testing, vertices are labeled using **0-based indexing**: ($V = \{0, 1, \dots, n - 1\}$). The input is read from standard input in the following format:

- First line: $n \ m$

- Next m lines: $u \ v \ w$ (an undirected edge between u and v with weight w)

The program outputs:

- 1) the best cut weight found,
- 2) the partition **S** and **T** (as lists of vertices),
- 3) summary statistics such as number of iterations and number of restarts for the heuristic.

8.1.6 Algorithmic scope assumptions

The task requires solving the problem using **Iterative Improvement** (local improvement) and **Brute Force**. Therefore, the report implements:

- an **exact brute-force** solver (exponential time) used as ground truth for small graphs, and
- a **local search / iterative improvement** heuristic that produces an **approximate** minimum cut for general graphs.

A discussion is included of more efficient exact algorithms for global minimum cut (e.g., Stoer–Wagner) and the relationship to max-flow/min-cut for **s-t** cut problems, to satisfy the requirement of discussing alternative approaches and more efficient techniques.

8.1.7 Randomization and reproducibility assumptions

For Iterative Improvement, since local search can converge to different local minima depending on the initial solution, we use a **multi-start strategy**: run local search from several randomly generated initial cuts and return the best result found. For reproducibility, the program allows setting a fixed pseudo-random seed (or uses a default seed if not provided).

8.2 Problem description

8.2.1 Problem statement

Given a weighted undirected graph $G = (V, E, w)$, the global minimum cut problem is to partition the vertex set V into two nonempty sets S and T such that the sum of weights of edges crossing between S and T is minimized. Formally, the objective is:

$$\left[\min_{S \subset V, S \neq \emptyset, S \neq V} W(S, V \setminus S), \right]$$

where

$$W(S, V \setminus S) = \sum_{\{u,v\} \in E, u \in S, v \in V \setminus S} w(u, v)$$

The cut with the minimum value is called the **minimum cut**, and its value is the **minimum cut weight**.

8.2.2 Challenges

The problem is an optimization problem over all valid partitions of the vertex set. The number of distinct cuts grows exponentially with the number of vertices. There are (2^n) subsets (S) , but each cut $(S, V \setminus S)$ is identical to $((V \setminus S, S))$, and the trivial subsets $S = \emptyset$ and $S = V$ are excluded. Consequently, brute-force enumeration becomes impractical for moderate n . This motivates approximate and heuristic strategies such as **iterative improvement (local search)**, which can produce good solutions efficiently but does not guarantee global optimality.

8.2.3 Required output

For each input graph, the solution should provide:

- 1) **Cut value:** the total weight of edges crossing the cut.
- 2) **Cut partition:** the sets **S** and **T** (or an assignment array indicating each vertex's side).
- 3) **Evaluation information:** runtime and/or number of iterations (especially for iterative improvement), and for small graphs an exact optimum from brute force to compare against.

These outputs enable the report's comparative evaluation between brute force (exact) and iterative improvement (approximate), as required by the project specification.

8.3 Methodology and solutions

This section presents two approaches required by the task:

1. **Brute Force (Exact Minimum Cut):** enumerates all possible cuts and returns the cut with minimum weight. This method is exact but exponential-time and therefore practical only for small graphs.
2. **Iterative Improvement (Approximate Minimum Cut):** starts from an initial feasible cut and repeatedly applies local improvements (small changes) to reduce the cut weight until no improving move exists. This method is fast in practice but may converge to a local optimum rather than the global optimum.

These two techniques will later be compared experimentally in Section 6.

8.4 Brute Force

A global cut is defined by choosing a nonempty subset S of vertices; the other side is $T = V \setminus S$. The brute force strategy enumerates all valid subsets S (excluding the empty set and the full set), computes the cut weight for each cut, and returns the minimum found.

To avoid counting the same cut twice (because (S, T) and (T, S) represent the same partition), we fix one vertex (we choose vertex 0) to always belong to S . Then we only enumerate subsets of the remaining vertices $\{1, 2, \dots, n-1\}$.

8.4.1 Step-by-Step methodology

Given input n, m and the edge list:

1. Read the graph (store edges in a list).
2. Initialize **bestWeight** = **+infinity**.
3. Fix **side[0] = 0** (meaning vertex 0 is always in S).
4. Enumerate all assignments for vertices **1..n-1** (each vertex is either in S or T).
5. Skip the assignment where all vertices are in S (because then T is empty).
6. For each assignment, compute the cut weight:

- Start **W** = 0
 - For every edge (**u**, **v**, **w**):
 - if **side[u] != side[v]**, add **w** to **W**
7. If **W** < **bestWeight**, update **bestWeight** and store the partition.
 8. After enumeration ends, output **bestWeight** and the corresponding S and T.

This procedure is guaranteed to find the global minimum cut because it checks every valid cut.

8.4.2 Brute Force pseudocode

Algorithm 1: BruteForceMinCut(G)

- **Input:** weighted undirected graph $G = (V, E, w)$ with $n = |V|$
- **Output:** minimum cut weight and a partition (S, T)

```

BruteForceMinCut(n, edges):
    bestWeight = +infinity
    bestSide[0..n-1] = all 0
    .
    .
    side[0] = 0 // fix vertex 0 in S
    .
    .
    // Enumerate all 2^(n-1) assignments of vertices 1..n-1
    for mask from 0 to (2^(n-1) - 1):
        // Build side[] from mask
        for v from 1 to n-1:
            bit = (mask >> (v-1)) & 1
            side[v] = bit // 0 means S, 1 means T
        .
        .
        // Skip invalid cut where T is empty (all vertices in S)
        if mask == 0:
            continue
        .
        .
        W = 0
        for each edge (u, v, w) in edges:
            if side[u] != side[v]:
                W = W + w
        .
        .
        if W < bestWeight:
            bestWeight = W

```

```
•         bestSide = side
•
•     return bestWeight, PartitionFrom(bestSide)
```

Notes:

- The style used for pseudo code is **Visual Studio Code's** as it has better visibility
- **mask == 0** corresponds to all vertices in **S** (since **side[v] = 0** for all **v**), which makes **T** empty, so we skip it.
- We do not need to skip the “all in T” case because **side[0] = 0** forces vertex 0 to remain in **S**, so **S** can never be empty.

8.4.3 Worked example

Consider the following small, weighted graph:

Vertices: {0, 1, 2, 3}

Edges (u, v, w):

- (0,1,2)
- (0,2,3)
- (0,3,1)
- (1,3,4)
- (2,3,5)

Fix vertex **0** in **S**. Then vertices **{1,2,3}** can be assigned to either side. We enumerate masks from 0 to 7 (binary 000 to 111), skipping mask 000 (invalid because **T** would be empty).

Example assignment: **mask = 010** means:

- vertex 1 -> bit0 = 0 -> in S
- vertex 2 -> bit1 = 1 -> in T
- vertex 3 -> bit2 = 0 -> in S

So:

- $S = \{0,1,3\}$
- $T = \{2\}$

Compute crossing edges:

- $(0,2,3)$ crosses $\Rightarrow +3$
- $(2,3,5)$ crosses $\Rightarrow +5$ Total cut weight = 8.

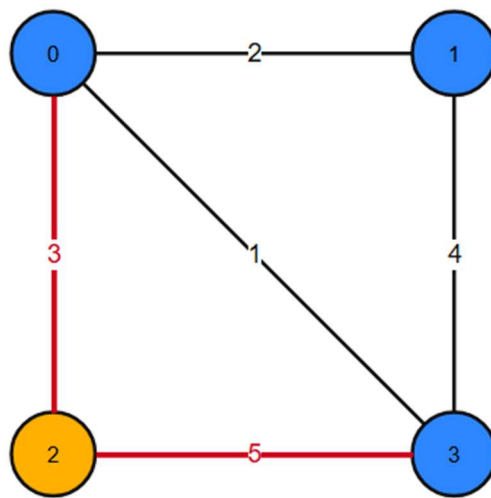


Figure 35

Figure 36 -Previous example

$S = \{0, 1, 3\}$ (blue) $T = \{2\}$ (orange)

Crossing edges: $(0,2)$ $w=3$, $(2,3)$ $w=5 \Rightarrow$ Cut weight = 8

The brute force algorithm repeats this for all masks and picks the minimum cut weight.

8.4.4 Time and space complexity

Time complexity:

There are $2^{(n-1)}$ assignments for vertices $1..n - 1$. We skip one invalid assignment (mask = 0), but this does not change asymptotic complexity. For each assignment, we compute the cut weight by scanning all m edges, which costs $O(m)$ time.

Therefore total time is:

- $O(2^{(n-1)} * m) = O(2^n * m)$.

Space complexity:

- Storing the graph: $O(n + m)$ (adjacency list) or $O(m)$ (edge list).
- Storing partitions: $O(n)$.

Overall: $O(n + m)$ space (dominated by graph storage).

8.5 Iterative Improvement

Iterative improvement (also called **local search**) is a general technique for optimization problems. The method starts from a feasible solution and repeatedly applies a small modification (a “local move”) that improves the objective value. The algorithm stops when no improving local move can be found; at that point, the solution is a **local optimum** with respect to the chosen neighborhood structure. This local optimum is not necessarily a global optimum.

In this minimum cut problem, a feasible solution is any valid cut (S, T) where neither side is empty. A natural “small change” is to **move a single vertex** from one side of the cut to the other (i.e., flip its side assignment). This produces a neighboring cut.

8.5.1 Solution representation

We represent a cut using an array:

- **side[v]** in $\{0, 1\}$ for each vertex **v**
 - **side[v] = 0** means vertex **v** is in set **S**
 - **side[v] = 1** means vertex **v** is in set **T**

Given **side[]**, the cut weight is computed as:

$$W = \sum_{\{u,v\} \in E, \text{side}[u] \neq \text{side}[v]} w(u, v)$$

This is the same objective as in brute force, but now we attempt to reduce **W** using local moves instead of enumerating all cuts.

8.5.2 Neighborhood structure

Neighborhood definition: A neighboring solution is obtained by selecting one vertex **v** and flipping it:

- $\text{side}[v] = 1 - \text{side}[v]$

Validity constraint: The cut must remain nontrivial, i.e., both sides must stay nonempty after the flip. Therefore, we reject any flip that would make all vertices belong to **S** or all vertices belong to **T**. (In implementation, we track **countS** and **countT** to enforce this.)

8.5.3 Delta for flipping one vertex

To perform local improvement efficiently, we need to compute how the cut weight changes if we flip a vertex **v**.

Let $\Delta(v)$ be the change in cut weight when **v** is flipped:

- New weight $\mathbf{W_new = W + \Delta(v)}$

For each neighbor **u** of **v**:

- If **u** is currently on the **same side** as **v**, then edge **(v,u)** is currently NOT crossing; after flipping **v**, it WILL cross, increasing the cut by **+w(v,u)**.
- If **u** is currently on a **different side** than **v**, then edge **(v,u)** is currently crossing; after flipping **v**, it will become internal, decreasing the cut by **-w(v,u)**.

So:

$$\Delta(v) = \sum_{u: \{u,v\} \in E} \begin{cases} +w(u,v) & \text{if } \text{side}[u] = \text{side}[v], \\ -w(u,v) & \text{if } \text{side}[u] \neq \text{side}[v]. \end{cases}$$

Figure 37

A flip is **improving** if it decreases the cut weight, i.e., if:

$$\Delta(v) < 0$$

This delta-based computation is a standard local search technique: evaluate the objective change of a local move without recomputing the full objective from scratch.

8.5.4 Best-Improvement strategy

In each iteration, there may be many vertices whose flip reduces the cut weight. We use the **best-improvement** strategy (also called steepest descent):

1. Compute $\Delta(v)$ for every vertex v (that can be flipped without violating nonempty sides).
2. Select the vertex v^* with the smallest delta (most negative).
3. If $\Delta(v^*) < 0$, flip v^* and update the current solution.
4. Otherwise, stop (local optimum reached).

Best-improvement tends to use fewer iterations than first-improvement, but each iteration does more work because it evaluates all possible moves.

8.5.5 Multi-Start strategy

A single local search run can get stuck in a poor local minimum depending on the starting cut. To improve solution quality, we use **multi-start local search**:

- Repeat local search from R different random initial cuts.
- Return the best cut found among all runs.

This is a standard way to mitigate local optima issues in heuristic optimization.

8.5.6 Step-by-Step methodology (Iterative Improvement + Multi-Start)

For a chosen number of restarts R :

For each restart $r = 1..R$:

1. Generate a random initial valid cut **side[]** (ensure both sides nonempty).
2. Compute the initial cut weight W .
3. Repeat:
 - For each vertex v , compute $\Delta(v)$ (skip invalid flips that empty a side).
 - Choose the best improving vertex v^* with minimum delta.

- If $\Delta(v^*) < 0$, flip v^* and update W .
 - Else stop.
4. Save the final solution of this restart.
 5. Keep the best solution across all restarts.

Output: the best cut found, its weight, and diagnostic statistics such as number of iterations per restart and total restarts.

8.5.7 Iterative Improvement pseudocode

Algorithm 2: LocalImprovementMinCut(G, R)

- **Input:** weighted undirected graph with n vertices and adjacency list adj , number of restarts R
- **Output:** best cut found (approximate) and its weight

```

• LocalImprovementMinCut(n, adj, R, seed):
•   rng = Random(seed)
•   bestWeight = +infinity
•   bestSide[0..n-1] = all 0
•
•   for restart from 1 to R:
•       side = RandomInitialCut(n, rng) // ensure both sides nonempty
•       countS = number of vertices with side[v] = 0
•       countT = n - countS
•
•       W = CutWeight(side, edges) // compute initial weight O(m)
•
•       improved = true
•       while improved:
•           improved = false
•           bestDelta = 0
•           bestV = -1
•
•           for v from 0 to n-1:
•               // Do not allow flips that make one side empty
•               if side[v] == 0 and countS == 1: continue
•               if side[v] == 1 and countT == 1: continue
•
•               delta = 0

```

```

•         for each (u, w_uv) in adj[v]:
•             if side[u] == side[v]:
•                 delta = delta + w_uv
•             else:
•                 delta = delta - w_uv
•
•         if delta < bestDelta:
•             bestDelta = delta
•             bestV = v
•
•         if bestV != -1: // we found an improving move
•             // apply the flip
•             if side[bestV] == 0: countS--; countT++
•             else: countT--; countS++
•             side[bestV] = 1 - side[bestV]
•             W = W + bestDelta
•             improved = true
•
•         if W < bestWeight:
•             bestWeight = W
•             bestSide = side
•
•     return bestWeight, PartitionFrom(bestSide)

```

8.5.8 Worked example

We use the same 4-vertex graph from Section 8.4:

Edges:

- (0,1,2)
- (0,2,3)
- (0,3,1)
- (1,3,4)
- (2,3,5)

Suppose we start with the random cut:

- $S = \{0,2\}$

- $T = \{1,3\}$

Compute crossing edges:

- $(0,1)$ crosses: +2
- $(0,3)$ crosses: +1
- $(2,3)$ crosses: +5 Total $W = 8$.

Now consider flipping vertex **2** from **S** to **T**.

- Before flip, **2** is in **S**; after flip, **2** moves to **T**.

Edges incident to 2:

- $(0,2,3)$: currently crossing (0 in S, 2 in S? actually 0 in S, 2 in S $\rightarrow$ internal), after flip it becomes crossing $\rightarrow +3$
- $(2,3,5)$: currently crossing (2 in S, 3 in T), after flip becomes internal $\rightarrow -5$

So $\Delta(2) = +3 - 5 = -2$, which is improving. New weight: $W_{\text{new}} = 8 + (-2) = 6$.

After the flip:

- $S = \{0\}$
- $T = \{1,2,3\}$ New crossing edges:
- $(0,1)$ +2
- $(0,2)$ +3
- $(0,3)$ +1 Total = 6.

If no further single-vertex flip reduces the cut weight, local search stops here, returning cut weight 6.

This illustrates how local improvement can reduce the cut weight step-by-step, but it may stop at a local optimum depending on the neighborhood definition and initial cut.

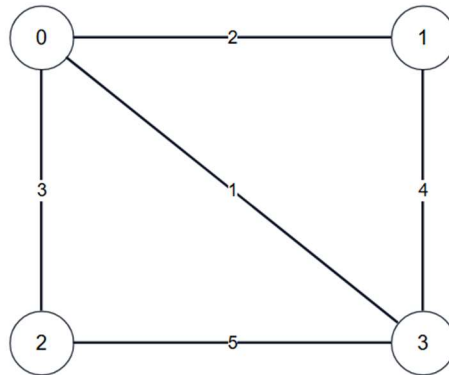


Figure 38

Figure 39 - Weighted Graph for this example

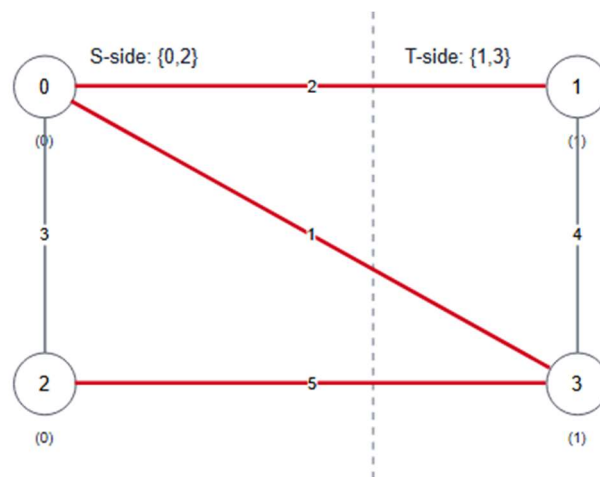


Figure 40

Figure 41 - Initial partition and its cut weight computed from crossing edges.

Crossing edges (X):

$(0,1)=2$

$(0,3)=1$

$(2,3)=5$

Cut weight $W = 2 + 1 + 5 = 8$

Non-crossing edges ($\cdot$):

$(0,2)=3$

$(1,3)=4$

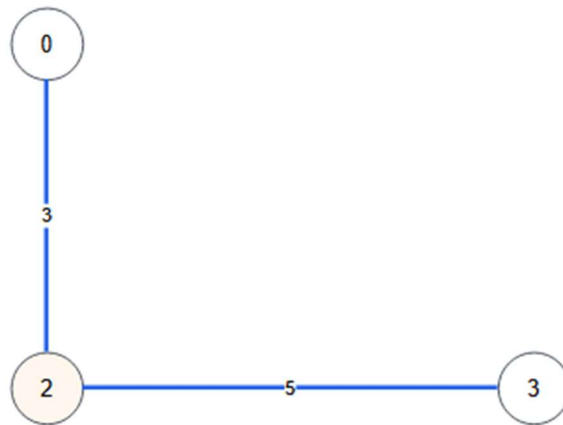


Figure 42

Figure 43 - Delta calculation for flipping one vertex using only incident edges.

Before flip: $\text{side}[2] = S$

After flip: $\text{side}[2] = T$

$(0,2) = 3$

Before: 0 in S and 2 in S $\Rightarrow$ NOT crossing

After : 0 in S and 2 in T $\Rightarrow$ crossing

Contribution to $\Delta(2)$: **+3**

$(2,3) = 5$

Before: 2 in S and 3 in T $\Rightarrow$ crossing

After : 2 in T and 3 in T $\Rightarrow$ NOT crossing

Contribution to $\Delta(2)$: -5

$$\Delta(2) = +3 + (-5) = -2$$

$$W_{\text{new}} = W + \Delta(2) = 8 + (-2) = 6$$

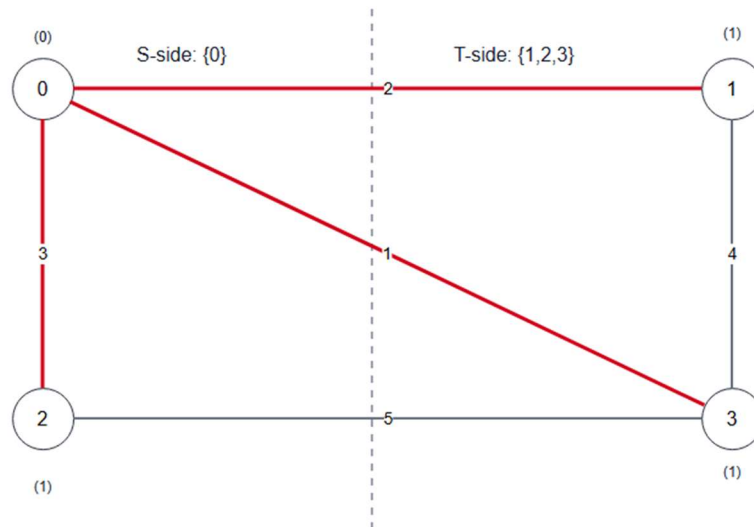


Figure 44

Figure 45 -Partition after one improving move, resulting in a smaller cut weight.

Crossing edges (X):

$$(0,1)=2$$

$$(0,2)=3$$

$$(0,3)=1$$

$$\text{Cut weight } W = 2 + 3 + 1 = 6$$

Non-crossing edges (·):

$$(1,3)=4$$

$$(2,3)=5$$

8.5.9 Notes on limitations

Iterative improvement with single-vertex flips guarantees that the cut weight decreases in each improving step, so it must eventually terminate. However, termination only guarantees a solution that is locally optimal with respect to the chosen neighborhood (single-vertex flips). A better cut may exist that requires moving multiple vertices simultaneously, which single-vertex local search cannot reach without allowing non-improving moves or restarts.

8.5.10 Time and space complexity

Time Complexity

In one local-search iteration, the algorithm considers flipping each vertex v (when flipping does not make one side empty). For each candidate vertex v , it computes $\Delta(v)$ by scanning all neighbors of v , which costs $O(\deg(v))$.

When this is done for all vertices in the same iteration, the total work is: sum of $\deg(v)$ over all vertices = $2m$ in an undirected graph.

So, the time per iteration is:

- $O(m)$

Let I be the number of improving flips (iterations) performed until the algorithm stops (reaches a local optimum) in one restart. Then the runtime for one restart is: $O(I * m)$

If we run the algorithm for R restarts, the total time becomes: $O(R * I_{avg} * m)$, where I_{avg} is the average number of iterations per restart.

Space Complexity

The algorithm stores:

- the graph (adjacency list and/or edge list): $O(n + m)$
- the current cut assignment $side[]$: $O(n)$

- a few counters and variables: $O(1)$

Therefore, the total space complexity is:

- $O(n + m)$

8.6 Implementation details

8.6.1 Implementation overview

This section describes how the two required techniques are implemented in C++:

1. **Brute Force (Exact):** Enumerates all cuts by iterating over bitmasks representing membership of vertices in set T (with vertex 0 fixed in S to avoid duplicate cuts). For each mask, the cut weight is computed by scanning all edges.
2. **Iterative Improvement (Approximate):** Uses multi-start local search. Each restart begins with a random valid cut, then repeatedly applies the **best-improving single-vertex flip** until no improvement exists. The best final cut across all restarts is returned.

This design follows standard local-search methodology and directly reflects the iterative improvement paradigm: start from a feasible solution, improve locally, and stop when no local improvement can be found.

8.6.2 Data structures

To support both algorithms efficiently and clearly, we store the graph in two forms:

- **Edge list** edges for brute force cut weight evaluation:
 - Edge { int u , v ; long long w ; }
- **Adjacency list** adj for local improvement delta computations:
 - **adj[v]** contains **pairs (neighbor, weight)**

We use **long long** for weights and cut sums to reduce overflow risk when weights are integers and sums become large.

Note: If the input contains duplicate **edges (u, v)** multiple times, the implementation can either:

- accept them as parallel edges (still valid; they simply add to the cut weight), or
- optionally merge them by summing weights (not required for correctness).

For simplicity and correctness, we accept parallel edges as-is; this matches the mathematical model where each edge contributes to the cut weight independently.

8.6.3 Helper functions

The implementation is organized around these helper operations:

- **computeCutWeight(side)**
Returns total cut weight given side[v] in {0,1}.
- **partitionFromSide(side)**
Builds explicit vertex lists S and T for printing.
- **randomInitialCut(n, rng)**
Generates a random cut with both sides nonempty.
- **localImproveOnce(side)**
Runs the best-improvement loop until a local optimum is reached; returns the final weight and side array.
- **multiStartLocalSearch(R)**
Runs R restarts and keeps the best result.

This modular structure improves clarity and helps satisfy the report requirement for step-by-step methodology.

8.6.4 Implementation code

```
#include <bits/stdc++.h>

using namespace std;

struct Edge {
    int u, v;
```

```

    long long w;
};

struct Result {
    long long weight = (1LL<<62);
    vector<int> side;          // 0 => S, 1 => T
    long long iterations = 0;  // local search iterations (moves)
    int restarts = 0;
};

static long long computeCutWeight(const vector<int>& side, const vector<Edge>&
edges) {
    long long W = 0;
    for (const auto& e : edges) {
        if (side[e.u] != side[e.v]) W += e.w;
    }
    return W;
}

static pair<vector<int>, vector<int>> partitionFromSide(const vector<int>& side)
{
    vector<int> S, T;
    for (int v = 0; v < (int)side.size(); v++) {
        if (side[v] == 0) S.push_back(v);
        else T.push_back(v);
    }
}

```

```

    }

    return {S, T};
}

static vector<int> randomInitialCut(int n, mt19937_64& rng) {
    // Ensure both sides are nonempty.
    // Simple approach: assign random sides, then fix if all ended up same.
    vector<int> side(n, 0);
    uniform_int_distribution<int> bit(0, 1);

    while (true) {
        int countT = 0;
        for (int v = 0; v < n; v++) {
            side[v] = bit(rng);
            countT += side[v];
        }
        if (countT != 0 && countT != n) break;
    }
    return side;
}

static Result localImproveBestFlip(
    const vector<vector<pair<int, long long>>>& adj,
    const vector<Edge>& edges,

```

```

    vector<int> sideInit
) {
    int n = (int)sideInit.size();
    vector<int> side = std::move(sideInit);

    int countT = 0;
    for (int v = 0; v < n; v++) countT += side[v];
    int countS = n - countT;

    long long W = computeCutWeight(side, edges);

    long long iterations = 0;

    while (true) {
        long long bestDelta = 0; // we need delta < 0 to improve
        int bestV = -1;

        for (int v = 0; v < n; v++) {
            // enforce nonempty sides after flip
            if (side[v] == 0 && countS == 1) continue; // would empty S
            if (side[v] == 1 && countT == 1) continue; // would empty T

            long long delta = 0;

            for (auto [u, w_uv] : adj[v]) {

```

```

        if (side[u] == side[v]) delta += w_uv;
        else delta -= w_uv;
    }

    if (delta < bestDelta) {
        bestDelta = delta;
        bestV = v;
    }
}

if (bestV == -1) break; // no improving move

// apply the best move

if (side[bestV] == 0) { countS--; countT++; }
else { countT--; countS++; }

side[bestV] = 1 - side[bestV];
W += bestDelta;
iterations++;
}

Result res;
res.weight = W;
res.side = std::move(side);

```

```

        res.iterations = iterations;

        return res;
    }

static Result multiStartLocalImprovement(
    const vector<vector<pair<int, long long>>>& adj,
    const vector<Edge>& edges,
    int R,
    unsigned long long seed
) {
    int n = (int)adj.size();
    mt19937_64 rng(seed);

    Result best;

    best.weight = (1LL<<62);
    best.restarts = R;

    for (int r = 1; r <= R; r++) {
        vector<int> init = randomInitialCut(n, rng);
        Result cur = localImproveBestFlip(adj, edges, init);

        if (cur.weight < best.weight) best = cur;

        best.iterations += cur.iterations; // total iterations across restarts
        (for reporting)
    }
}

```

```

    return best;
}

static Result bruteForceMinCut(int n, const vector<Edge>& edges) {
    // Brute force enumerates subsets of vertices 1..n-1.
    // side[0]=0 is fixed in S.
    // side[v]=1 means vertex v is in T.

    Result best;

    best.weight = (1LL<<62);
    best.side.assign(n, 0);

    if (n <= 1) {
        // No valid nontrivial cut exists; keep default.

        best.weight = 0;

        return best;
    }

    long long totalMasks = 1LL << (n - 1); // works for n-1 <= 62; brute force is
    anyway small-n

    vector<int> side(n, 0);

    side[0] = 0;

    for (long long mask = 0; mask < totalMasks; mask++) {
        if (mask == 0) continue; // would make T empty

```



```

        for (int v = 1; v < n; v++) {
            side[v] = (int)((mask >> (v - 1)) & 1LL);
        }

        long long W = computeCutWeight(side, edges);
        if (W < best.weight) {
            best.weight = W;
            best.side = side;
        }
    }
    return best;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<Edge> edges;
    edges.reserve(m);

```

```

vector<vector<pair<int,long long>>> adj(n);

for (int i = 0; i < m; i++) {
    int u, v;
    long long w;
    cin >> u >> v >> w;
    edges.push_back({u, v, w});
    adj[u].push_back({v, w});
    adj[v].push_back({u, w});
}

// Parameters (can be adjusted for experiments)
int R = 50; // number of restarts for local improvement
unsigned long long seed = 123456789ULL;

// Run approximate method
Result approx = multiStartLocalImprovement(adj, edges, R, seed);

cout << "=== Iterative Improvement (Approx) ===\n";
cout << "Best cut weight = " << approx.weight << "\n";
auto [S1, T1] = partitionFromSide(approx.side);
cout << "S: ";
for (int v : S1) cout << v << " ";
cout << "\nT: ";

```

```

    for (int v : T1) cout << v << " ";

    cout << "\nTotal local moves (across restarts) = " << approx.iterations <<
    "\n";

    cout << "Restarts = " << approx.restarts << "\n\n";

    // Run exact brute force only for small n

    if (n <= 25) {

        Result exact = bruteForceMinCut(n, edges);

        cout << "=== Brute Force (Exact) ===\n";

        cout << "Minimum cut weight = " << exact.weight << "\n";

        auto [S2, T2] = partitionFromSide(exact.side);

        cout << "S: ";

        for (int v : S2) cout << v << " ";

        cout << "\nT: ";

        for (int v : T2) cout << v << " ";

        cout << "\n";

    } else {

        cout << "=== Brute Force (Exact) ===\n";

        cout << "Skipped (n too large for brute force enumeration).\n";

    }

    return 0;
}

```

8.6.5 Notes

1. **Correctness of brute force:** It checks all valid cuts (with vertex 0 fixed to remove symmetry), so it always finds the global minimum cut for small enough n .
2. **Correctness of iterative improvement termination:** The local search accepts only strictly improving moves ($\Delta < 0$), so the cut weight decreases each iteration. Since there are finitely many cuts, the algorithm must terminate. The final cut is a local minimum with respect to single-vertex flips, not necessarily the global minimum.
3. **Randomness and reproducibility:** Using a fixed seed produces reproducible runs. Changing the seed can yield different solutions; multi-start improves the chance of finding a near-optimal cut.

8.7 Comparative evaluation

8.7.1 Evaluation criteria

Solution quality:

For a given graph:

- Let **W_{opt}** be the minimum cut weight found by brute force (exact).
- Let **W_{approx}** be the cut weight found by iterative improvement (approx).

We compare:

1. **Absolute gap: $W_{\text{approx}} - W_{\text{opt}}$**
2. **Approximation ratio (empirical): $W_{\text{approx}} / W_{\text{opt}}$ (only when $W_{\text{opt}} > 0$)**

A ratio of **1.0** means the heuristic matched the exact optimum on that instance.

Computational cost:

We compare:

- Whether brute force is feasible (it grows exponentially with n)
- Number of local search moves (iterations) and restarts
- Runtime (optional but recommended if you can measure it consistently)

Because brute force has exponential time complexity, it is expected to become impractical even for moderate n , while iterative improvement should remain usable for much larger graphs.

8.7.2 Experimental setup

To make the evaluation meaningful, we use multiple categories of graphs:

1. **Small graphs ($n \leq 25$):**
Brute force is run, so we can directly compare **W_{approx}** against **W_{opt}**.

2. Larger graphs ($n > 25$):

Brute force is skipped. We evaluate the heuristic using:

- best weight found across restarts
- variability across different runs (optional: different seeds)
- runtime / iteration counts

Parameters used:

The iterative improvement algorithm depends on:

- **R**: number of restarts
- **seed**: random seed

In the provided implementation, R is set to 50 and a fixed seed is used for reproducibility. (For deeper evaluation, R can be varied such as 10, 50, 100 to show the trade-off between runtime and quality.)

8.7.3 Comparative results on sample graphs

This subsection reports the results you obtained from three representative test cases. These examples demonstrate both correctness (weights match the computed cut) and how the heuristic compares to the exact optimum on small graphs.

Table 1. Comparison of iterative improvement vs brute force on sample inputs.

Table 46

Case	n	m	W_approx	W_opt	Gap (W_approx-W_opt)	Ratio (W_approx/W_opt)
1	4	5	6	6	0	1.00
2	2	1	7	7	0	1.00

3	6	7	1	1	0	1.00
---	---	---	---	---	---	------

Discussion:

- In all three sample cases, the iterative improvement method found a cut with the same weight as the brute-force optimum (ratio = 1.00).
- Note that the *partition* returned by the heuristic may differ from the exact solver even when the weights match, because minimum cuts are not always unique.

8.7.4 Interpretation and scalability discussion

Brute force is valuable as:

- an exact solver for very small graphs,
- a verification tool to confirm correctness of the heuristic implementation,
- a baseline for measuring the heuristic's empirical approximation quality.

However, due to exponential growth in the number of cuts, brute force becomes impractical as n increases.

Iterative improvement evaluates local changes and usually converges in a relatively small number of iterations compared to enumerating all cuts. With multi-start, it often produces near-optimal cuts in practice, but it has no guarantee to always find the global minimum cut.

8.8 Alternative approaches

8.8.1 Exact Global Minimum Cut in Undirected Graphs (Stoer–Wagner)

For undirected weighted graphs, the **Stoer-Wagner algorithm** finds the global minimum cut exactly in polynomial time. This is important because it shows that, under the assumptions used in this report (undirected graph, global min cut), there exists an exact method that is dramatically faster than brute force enumeration and does not suffer from local optimum issues like iterative improvement.

In practice, Stoer-Wagner repeatedly performs a “minimum cut phase” that grows a set of vertices using maximum adjacency weights, producing a cut candidate at the end of each phase, and then contracting vertices to reduce the problem size. The minimum over all phases is the global minimum cut. While the internal details are not implemented in this project, the main takeaway is that it provides an exact solution with much better scalability than brute force.

8.8.2 Exact s – t Minimum Cut via Maximum Flow (Max-Flow Min-Cut)

If a problem instance specifies two special vertices s (source) and t (sink), then the task becomes the **s – t minimum cut** problem. In that case, the minimum cut can be solved exactly by computing a **maximum flow**, due to the **max-flow min-cut theorem**. This approach is widely used in network design, routing, and many optimization applications.

Several polynomial-time max-flow algorithms exist (e.g., Edmonds–Karp, Dinic, and push–relabel). Their performance depends on the graph size and density, but they are typically efficient on large inputs compared to brute force. Max-flow is an important alternative to mention because it is a standard exact method for cut problems when the cut is defined with respect to specific terminals.

8.8.3 Randomized Algorithms for Min Cut (Alternative Exact/Probabilistic Perspective)

Another direction is **randomized algorithms**. For undirected graphs, contraction-based randomized algorithms can find a global minimum cut with some probability, and repeating the algorithm increases the success probability. These methods can be attractive when:

- a fast algorithm is needed,
- some probability of failure is acceptable, or
- the algorithm is repeated enough times to make failure extremely unlikely.

This perspective is different from local improvement: instead of improving a single solution, randomized methods sample the solution space through random contractions.

8.8.4 Stronger Heuristics than Basic Local Improvement (Escaping Local Minima)

The iterative improvement implemented in this project uses strict descent (only accept moves that reduce the cut weight). A known limitation of strict descent local search is that it can become trapped in a poor local optimum. Multi-start partially mitigates this, but other heuristic frameworks can further improve solution quality:

- **Simulated annealing:** occasionally accepts worse moves early to escape local minima, then becomes more selective later.
- **Tabu search:** uses short-term memory (tabu list) to avoid cycling and to encourage exploration.
- **Genetic algorithms:** maintain a population of cuts and combine them to create new candidate solutions.

These techniques typically require parameter tuning and more complex implementation, but they can outperform basic local improvement on difficult instances where local minima are common.

8.9 Sample input and output

```
4 5
0 1 2
0 2 3
0 3 1
1 3 4
2 3 5
=== Iterative Improvement (Approx) ===
Best cut weight = 6
S: 1 2 3
T: 0
Total local moves (across restarts) = 34
Restarts = 0

=== Brute Force (Exact) ===
Minimum cut weight = 6
S: 0 2 3
T: 1
```

Figure 46

```
2 1
0 1 7
=== Iterative Improvement (Approx) ===
Best cut weight = 7
S: 1
T: 0
Total local moves (across restarts) = 0
Restarts = 0

=== Brute Force (Exact) ===
Minimum cut weight = 7
S: 0
T: 1
```

Figure 47

```
6 7
0 1 10
1 2 10
0 2 10
3 4 10
4 5 10
3 5 10
2 3 1
=== Iterative Improvement (Approx) ===
Best cut weight = 1
S: 0 1 2
T: 3 4 5
Total local moves (across restarts) = 55
Restarts = 0

=== Brute Force (Exact) ===
Minimum cut weight = 1
S: 0 1 2
T: 3 4 5
```

Figure 48

9. Cluster Points into Clusters

9.1 Problem description

Cluster points into k clusters. Use iterative improvement, divide-and-conquer, and Brute Force to solve this problem.

Clustering is a fundamental task in data mining and unsupervised learning, focused on partitioning a set of objects into groups, or clusters, such that objects within the same group are more like each other than to those in other groups. This section explores the implementation and performance of three distinct algorithmic paradigms: Divide and Conquer (Agglomerative), Brute Force (Exhaustive Search), and Iterative Improvement (K-Means).

9.2 Assumptions

1. Points are 2D with x- and y-coordinates.
2. Euclidean distance: similarity between points and clusters is measured using Euclidean distance.
3. Fixed cluster distance metric: the distance between two clusters is defined as the distance between their centroids.
4. Centroid-based clustering: each cluster is represented by its centroid.
5. Clustering Bias: points are assigned to clusters based on the minimum distance to a cluster's centroid.
6. Convergence criterion: In iterative improvement, convergence is declared when all new centroids in an iteration are exactly equal to the previous one.
7. Initial centroids are data points: For iterative improvement, initial centroids are randomly chosen from the given points, not generated arbitrarily in continuous space.
8. Base case: defined as ``k == 1`` or ``k >= number of points``, handled by returning either all points in one cluster or each point as its own cluster.
9. Deterministic brute force: Brute force enumerates all distinct assignments.
10. Divide-and-conquer order: points are split by index order (first half / second half).
11. No ties in nearest centroid: in nearest-centroid assignment, if a point is equidistant from two centroids, the code picks the first encountered centroid.
12. No randomness control: Random shuffling for iterative improvement uses ``random_shuffle`` without seeding, so results may vary across runs.
13. Single-run output: each algorithm returns the first converged/optimal solution without multiple restarts to avoid local optima.

9.3 Methodology

9.3.1 Brute-Force

Step-by-Step description (pseudocode in English).

1. Generate all possible combinations of clustering points into k clusters.
2. Try each one using the points.
3. Compute the score of each combination.
 - a. The score is the distance between each cluster's centroid and the points inside of it for a given combination.
 - b. The best score is the minimum distance across all clusters.
4. The combination with the best score is the optimal result.
5. Print the best combination.

9.3.2 Divide and Conquer

1. Divide the points two parts: left and right.
2. For each part divide again until the number of points is equal to or less than the number of clusters (k).
3. Then merge all the parts together.
 - a. Append the right clusters to the left clusters, generating $2k$ clusters.
 - b. While the number of clusters in the combined list is greater than k , repeatedly merge the two closest clusters.
 - c. For every pair of distinct clusters i and j , calculate the distance between them using the centroid distance method:
 - i. Compute the centroid (average point) of each cluster.
 - ii. Calculate the Euclidean distance between these two centroids.
 - d. Find the closest pair and merge it by adding all points from the second cluster to the first cluster.
 - e. Remove the second cluster from the list.
 - f. Repeat until only k clusters remain.
4. Return the merged clusters.
5. At the end of the recursion cycle, the final clustering remains.

9.3.3 Iterative Improvement

1. Create k clusters each containing a random point from the points list; thus, a cluster is never empty.
2. Repeat until convergence:
 - a. Assign each point to its nearest cluster.
 - b. Find the mean point of these points, this will be the new centroid of the cluster (iterative improvement step).
 - c. If the new centroid is nearly equal to the old one, then convergence occur. If not, repeat.
3. Return the final clusters.

9.4 Pseudocode

9.4.1 Helper Functions

```
function eachPointAsCluster(points):
```

```
    clusters <- empty list
```

```
    for each point in points:
```

```
        clusters.append([point]) // singleton cluster
```

```
    return clusters
```

```
function clusterDistance(clusterA, clusterB):
```

```
    centroidA <- computeCentroid(clusterA)
```

```
    centroidB <- computeCentroid(clusterB)
```

```
    return distance(centroidA, centroidB)
```

```
function computeCentroid(cluster):
```

```
// The centroid is the mean point of all the points in a cluster
```

```
    if cluster is empty:
```

```
        return Point(0, 0)
```

```
    sumX <- 0
```

```
    sumY <- 0
```

```
    for each point in cluster:
```

```
        sumX <- sumX + point.x
```

```
    sumY <- sumY + point.y

    return Point(sumX / length(cluster), sumY / length(cluster))

function distance(pointA, pointB):

    // Euclidean distance

    dx = pointA.x - pointB.x

    dy = pointA.y - pointB.y

    return sqrt(dx*dx + dy*dy)

function isBaseCase(points, k):

    return (k == 1) or (k >= length(points))
```


9.4.2 Brute-Force

```
function bruteForceClusters(points, k):  
  
    if k = 1:  
  
        return [points]  
  
    if length(points) <= k:  
  
        return eachPointAsCluster(points)  
  
  
    // Generate all valid partitions  
  
    allAssignments <- empty list  
  
    combination <- array (length(points))  
  
    generateAllCombinations(points, 0, combination, k, 0, allAssignments)  
  
  
    // Find assignment with minimum score  
  
    bestScore <- INFINITY  
  
    bestAssignment <- empty list  
  
  
    for each combination in allAssignments:  
  
        score <- calculatePartitionScore(points, combination, k)  
  
        if score < bestScore:
```

```

    bestScore <- score

    bestAssignment <- assignment

// Build clusters from best combination

clusters <- empty list

for i <- 0 to length(points)-1:

    clusterId = bestAssignment[i]

    clusters[clusterId].append(points[i])

return clusters

function generateAllCombinations(points, index, combination, k, usedClusters, allAssignments):

    n <- length(points)

    remaining <- n - index

    neededClusters <- k - usedClusters

// Pruning: not enough remaining points to fill needed clusters

if remaining < neededClusters:

    return

// Base case: all points assigned

if index = n:

```

```

    if usedClusters = k: // only keep full cluster assignments

        allAssignments.append(copy of combination)

    return

// First point always goes to cluster 0 (to avoid symmetric duplicates)

if index = 0:

    combination[index] <- 0

    generateAllCombinations(points, index+1, combination, k, 1, allAssignments)

    return

// Case 1: assign to an existing cluster

for cluster <- 0 to usedClusters-1:

    combination[index] <- cluster

    generateAllCombinations(points, index+1, combination, k, usedClusters, allAssignments)

// Case 2: assign to a new cluster (if available)

if usedClusters < k:

    combination[index] <- usedClusters

    generateAllCombinations(points, index+1, combination, k, usedClusters+1, allAssignments)

```

```

function calculatePartitionScore(points, combination, k):

    // Output: sum of distances from each point to its cluster's centroid

    // Build clusters from combination

    clusters <- empty list

    for i <- 0 to k-1:

        clusters.append(empty list)

    for i <- 0 to length(points)-1:

        clusterId <- combination [i]

        clusters[clusterId].append(points[i])

    // Calculate total score

    totalScore <- 0

    for each cluster in clusters:

        centroid <- computeCentroid(cluster)

        for each point in cluster:

            totalScore <- totalScore + distance(point, centroid)

    return totalScore

```

9.4.3 Divide and Conquer

```
function divideAndConquerClusters(points, k):  
  
  if length(points) <= k:  
  
    return eachPointAsCluster(points)  
  
  
  // Divide  
  
  middle <- floor(length(points) / 2)  
  
  leftPoints <- points[0 .. middle-1]  
  
  rightPoints <- points[middle .. end-1]  
  
  
  // Conquer  
  
  leftClusters <- divideAndConquerClusters(leftPoints, k)  
  
  rightClusters <- divideAndConquerClusters(rightPoints, k)  
  
  
  // Combine  
  
  combined <- leftClusters + rightClusters // concatenate lists  
  
  return mergeClusters(combined, k)  
  
function mergeClusters(clusters, targetCount):  
  
  // integer targetCount is k
```

```

while length(clusters) > targetCount:

    // Find closest pair of clusters

    minDistance <- INFINITY

    firstIndex <- 0

    secondIndex <- 1

    for i <- 0 to length(clusters)-1:

        for j <- i+1 to length(clusters)-1:

            distance <- clusterDistance(clusters[i], clusters[j])

            if distance < minDistance:

                minDistance <- distance

                firstIndex <- i

                secondIndex <- j

    // Merge the closest pair

    for each point in clusters[secondIndex]:

        clusters[firstIndex].append(point)

    remove clusters[secondIndex]      // Remove the second cluster

return clusters

```

9.4.4 Iterative Improvement

```
function iterativeImprovementClusters(points, k):  
  
    // Base cases  
  
    if k = 1:  
  
        return [points]                // single cluster containing all points  
  
    if length(points) <= k:  
  
        return eachPointAsCluster(points)  
  
  
    // Initialize centroids with k random distinct points from the input  
  
    indices <- [0, 1, 2, ..., length(points)-1]  
  
    randomShuffle(indices)              // generates random order  
  
  
    centroids <- empty list  
  
    for i <- 0 to k-1:  
  
        randomIndex <- indices[i]  
  
        centroids.append(points[randomIndex])  
  
  
    // Iterative improvement loop  
  
    while true:
```

```
// Assignment step: assign each point to nearest centroid  
  
clusters <- assignPointsToNearestCluster(points, centroids)
```

```
// Update step: compute new centroids  
  
newCentroids <- empty list  
  
converged <- true  
  
for i <- 0 to k-1:  
  
    newCentroid <- computeCentroid(clusters[i])  
  
    newCentroids.append(newCentroid)  
  
    if newCentroid != centroids[i]:  
  
        converged <- false  
  
if converged:  
  
    return clusters  
  
// Prepare for next iteration  
  
centroids <- newCentroids
```

```
function assignPointsToNearestCluster(points, centroids):  
  
    clusters <- empty list  
  
    for i <- 0 to length(centroids)-1:
```



```
clusters.append(empty list)

for each point in points:

    nearestIndex <- 0

    minDistance <- distance(point, centroids[0])

    for i <- 1 to length(centroids)-1:

        dist <- distance(point, centroids[i])

        if dist < minDistance:

            minDistance <- dist

            nearestIndex <- i

    clusters[nearestIndex].append(point)

return clusters
```

9.5 Code implementation

9.5.1 Helper code

```
#pragma once

#include <bits/stdc++.h>

using namespace std;

class Point
{
private:
    double x, y;

    static constexpr double EPSILON = 1.0;

public:
    Point(double x, double y) : x(x), y(y) {}

    double getX() const { return x; }

    double getY() const { return y; }

    double distanceTo(const Point &other) const
    {
        double dx = x - other.x;

        double dy = y - other.y;

        return sqrt(dx * dx + dy * dy);
    }
}
```

```

    }

    bool operator==(const Point &other) const
    {
        return abs(x - other.x) < EPSILON && abs(y - other.y) < EPSILON;
    }

    bool operator!=(const Point &other) const
    {
        return abs(x - other.x) >= EPSILON || abs(y - other.y) >= EPSILON;
    }

    void print() const
    {
        cout << "(" << x << ", " << y << ")", ";
    }
};

using Cluster = vector<Point>;

bool isBaseCase(const vector<Point> &points, int k)

```

```

{
    return k == 1 || k >= (int)points.size();
}

vector<Cluster> eachPointAsCluster(const vector<Point> &points)
{
    vector<Cluster> clusters;

    for (const Point &p : points)
        clusters.push_back({p});

    return clusters;
}

Point computeCentroid(const vector<Point> &points)
{
    double totalX = 0, totalY = 0;

    for (const Point &p : points)
    {
        totalX += p.getX();
        totalY += p.getY();
    }

    return Point(totalX / points.size(), totalY / points.size());
}

double clusterDistance(const Cluster &a, const Cluster &b)

```

```

{
    Point centroidA = computeCentroid(a);
    Point centroidB = computeCentroid(b);
    return centroidA.distanceTo(centroidB);
}

void printClusters(const vector<Cluster> &clusters)
{
    for (int i = 0; i < (int)clusters.size(); i++)
    {
        cout << "  Cluster " << i + 1 << ": ";
        for (const Point &p : clusters[i])
            p.print();
        cout << "\n";
    }
}

```

9.5.2 Brute-Force

```
#include "ClustersHelper.hpp"

using namespace std;

// Using Stirling numbers of the second kind

void generateAllCombinations(
    const vector<Point> &points,
    int index,
    vector<int> &combination,
    int k,
    int usedClusters,
    vector<vector<int>> &allAssignments)
{
    int n = points.size();

    // Prune when not enough remaining points
    int remaining = n - index;
    int neededClusters = k - usedClusters;
    if (remaining < neededClusters)
        return;

    // Return when all points assigned
    if (index == n)
    {
```

```

        if (usedClusters == k) // Only assignments that have filled clusters
            allAssignments.push_back(combination);

        return;
    }

    // Always assign to cluster 0 to avoid symmetric duplicates
    if (index == 0)
    {
        combination[index] = 0;

        generateAllCombinations(points, index + 1, combination, k, 1,
allAssignments);

        return;
    }

    // Assign to existing clusters
    for (int cluster = 0; cluster < usedClusters; cluster++)
    {
        combination[index] = cluster;

        generateAllCombinations(points, index + 1, combination, k, usedClusters,
allAssignments);
    }

    // Assign to new cluster if there are any
    if (usedClusters < k)

```

```

    {
        combination[index] = usedClusters;

        generateAllCombinations(points, index + 1, combination, k, usedClusters +
1, allAssignments);
    }
}

// Score is evaluated as sum of distances to centroid

double calculatePartitionScore(const vector<Point> &points, const vector<int>
&combination, int k)
{
    vector<Cluster> clusters(k);

    for (int i = 0; i < (int)points.size(); i++)
        clusters[combination[i]].push_back(points[i]);

    double totalScore = 0;
    for (const Cluster &cluster : clusters)
    {
        Point centroid = computeCentroid(cluster);

        for (const Point &point : cluster)
            totalScore += point.distanceTo(centroid);
    }

    return totalScore;
}

vector<Cluster> bruteForceClusters(const vector<Point> &points, int k)

```



```

{
    if (isBaseCase(points, k))
        return k == 1 ? vector<Cluster>{points} : eachPointAsCluster(points);

    vector<vector<int>> allPartitions;
    vector<int> assignment(points.size(), 0);
    generateAllCombinations(points, 0, assignment, k, 0, allPartitions);

    double bestScore = INT_MAX; // minimum distance to centroid
    vector<int> bestAssignment;
    for (const vector<int> &partition : allPartitions)
    {
        double score = calculatePartitionScore(points, partition, k);
        if (score >= bestScore)
            continue;

        bestScore = score;
        bestAssignment = partition;
    }

    vector<Cluster> clusters(k);
    for (int i = 0; i < points.size(); i++)
    {
        auto cluster = bestAssignment[i];
    }
}

```

```

        clusters[cluster].push_back(points[i]);
    }

    return clusters;
}

int main()
{
    vector<Point> points = {
        Point(1, 2), Point(3, 4), Point(5, 6),
        Point(7, 8), Point(9, 10), Point(11, 12),
        Point(13, 14), Point(15, 16), Point(17, 18), Point{19, 20}};

    vector<int> testCases = {2, 3, 4, 5};
    for (int test : testCases)
    {
        cout << "==== Brute Force Testing for " << test << " Clusters
=====\\n";

        printClusters(bruteForceClusters(points, test));

        cout << "\\n";
    }
}

```

9.5.3 Divide and Conquer

```
#include "ClustersHelper.hpp"

using namespace std;

vector<Cluster> mergeClusters(vector<Cluster> clusters, int targetCount)
{
    while ((int)clusters.size() > targetCount)
    {
        int firstIndex = 0, secondIndex = 1;
        double minDistance = clusterDistance(clusters[0], clusters[1]);

        for (int i = 0; i < (int)clusters.size(); i++)
        {
            for (int j = i + 1; j < (int)clusters.size(); j++)
            {
                double distance = clusterDistance(clusters[i], clusters[j]);
                if (distance < minDistance)
                {
                    minDistance = distance;
                    firstIndex = i;
                    secondIndex = j;
                }
            }
        }
    }
}
```

```

        for (const Point &p : clusters[secondIndex])
            clusters[firstIndex].push_back(p);

        clusters.erase(clusters.begin() + secondIndex);
    }

    return clusters;
}

vector<Cluster> divideAndConquerClusters(const vector<Point> &points, int k)
{
    if ((int)points.size() <= k)
        return eachPointAsCluster(points);

    int middle = points.size() / 2;
    vector<Point> leftPoints(points.begin(), points.begin() + middle);
    vector<Point> rightPoints(points.begin() + middle, points.end());

    vector<Cluster> leftClusters = divideAndConquerClusters(leftPoints, k);
    vector<Cluster> rightClusters = divideAndConquerClusters(rightPoints, k);
    vector<Cluster> combined = leftClusters;
    combined.insert(combined.end(), rightClusters.begin(), rightClusters.end());

    return mergeClusters(combined, k);
}

```

9.5.4 Iterative Improvement

```
#include "ClustersHelper.hpp"

using namespace std;

vector<Cluster> assignPointsToNearestCluster(const vector<Point> &points, const
vector<Point> &centroids)
{
    vector<Cluster> clusters(centroids.size());

    for (const Point &point : points)
    {
        int nearestIndex = 0;

        double minDistance = point.distanceTo(centroids[0]);

        for (int i = 1; i < (int)centroids.size(); i++)
        {
            double distance = point.distanceTo(centroids[i]);

            if (distance < minDistance)
            {
                minDistance = distance;

                nearestIndex = i;
            }
        }

        clusters[nearestIndex].push_back(point);
    }

    return clusters;
}
```

```

}

vector<Cluster> iterativeImprovementClusters(const vector<Point> &points, int k)
{
    if (isBaseCase(points, k))
        return k == 1 ? vector<Cluster>{points} : eachPointAsCluster(points);

    // Choose centroids from random points inside the points vector
    vector<Point> centroids;
    vector<int> pointsIndices(points.size());
    iota(pointsIndices.begin(), pointsIndices.end(), 0);
    random_shuffle(pointsIndices.begin(), pointsIndices.end());
    for (int i = 0; i < k; i++){
        int randomPoint = pointsIndices[i];
        centroids.push_back(points[randomPoint]);
    }

    while (true)
    {
        vector<Cluster> clusters = assignPointsToNearestCluster(points,
centroids);

        vector<Point> newCentroids;

        bool converged = true;

```

```

    for (int i = 0; i < k; i++)
    {
        // compute the mean point of the points in a cluster
        // this mean point is the new cluster centroid (iterative improvement
step)

        Point newCentroid = computeCentroid(clusters[i]);

        if (newCentroid != centroids[i])
            converged = false;

        newCentroids.push_back(newCentroid);
    }

    if (converged)
        return clusters;

    centroids = newCentroids;
}
}

```

9.6 Test cases

9.6.1 Chosen test cases

Test Case for Brute Force (Only 10 Points due to its the super-exponential nature):

```
vector<Point> points = {  
  
    Point(1, 2), Point(3, 4), Point(5, 6),  
  
    Point(7, 8), Point(9, 10), Point(11, 12),  
  
    Point(13, 14), Point(15, 16), Point(17, 18), Point{19, 20}  
  
};
```

Test Case for Iterative Improvement and Divide-and-Conquer:

```
vector<Point> points = {  
  
    Point(2, 3), Point(3, 2), Point(4, 5), Point(5, 4), Point(6, 3),  
  
    Point(1, 6), Point(7, 2), Point(3, 7), Point(4, 4), Point(5, 5),  
  
    Point(11, 12), Point(12, 11), Point(13, 14), Point(14, 13), Point(15, 12),  
  
    Point(10, 15), Point(16, 11), Point(12, 16), Point(13, 13), Point(14, 14),  
  
    Point(20, 21), Point(21, 20), Point(22, 23), Point(23, 22), Point(24, 21),  
  
    Point(20, 24), Point(25, 22), Point(22, 25), Point(23, 23), Point(24, 24)  
  
};
```


9.6.2 Output

```
===== Brute Force Testing for 2 Clusters =====
Cluster 1: (1, 2), (3, 4), (5, 6), (7, 8), (9, 10),
Cluster 2: (11, 12), (13, 14), (15, 16), (17, 18), (19, 20),

===== Brute Force Testing for 3 Clusters =====
Cluster 1: (1, 2), (3, 4), (5, 6), (7, 8),
Cluster 2: (9, 10), (11, 12), (13, 14),
Cluster 3: (15, 16), (17, 18), (19, 20),

===== Brute Force Testing for 4 Clusters =====
Cluster 1: (1, 2), (3, 4), (5, 6),
Cluster 2: (7, 8), (9, 10), (11, 12),
Cluster 3: (13, 14), (15, 16), (17, 18),
Cluster 4: (19, 20),

===== Brute Force Testing for 5 Clusters =====
Cluster 1: (1, 2), (3, 4), (5, 6),
Cluster 2: (7, 8), (9, 10),
Cluster 3: (11, 12), (13, 14), (15, 16),
Cluster 4: (17, 18),
Cluster 5: (19, 20),
```

Figure 49

```
===== Divide & Conquer Testing for 2 Clusters =====
Cluster 1: (2, 3), (3, 2), (4, 5), (5, 4), (6, 3), (7, 2), (3, 7), (4, 4), (5, 5), (1, 6), (11, 12), (12, 11), (13, 14), (14, 13), (15, 12), (10, 15), (12, 16), (13, 13), (14, 14), (16, 11),
Cluster 2: (20, 21), (21, 20), (22, 23), (23, 22), (24, 21), (25, 22), (23, 23), (24, 24), (20, 24), (22, 25),

===== Divide & Conquer Testing for 3 Clusters =====
Cluster 1: (2, 3), (3, 2), (4, 5), (5, 4), (6, 3), (7, 2), (3, 7), (4, 4), (5, 5), (1, 6),
Cluster 2: (11, 12), (12, 11), (13, 14), (14, 13), (15, 12), (10, 15), (12, 16), (13, 13), (14, 14), (16, 11),
Cluster 3: (20, 21), (21, 20), (22, 23), (23, 22), (23, 23), (24, 24), (24, 21), (25, 22), (20, 24), (22, 25),

===== Divide & Conquer Testing for 4 Clusters =====
Cluster 1: (2, 3), (3, 2), (4, 5), (5, 4), (4, 4), (5, 5), (6, 3), (7, 2), (1, 6), (3, 7),
Cluster 2: (11, 12), (12, 11), (13, 14), (14, 13), (15, 12), (10, 15), (12, 16), (13, 13), (14, 14),
Cluster 3: (16, 11),
Cluster 4: (20, 21), (21, 20), (22, 23), (23, 23), (23, 22), (24, 24), (24, 21), (25, 22), (20, 24), (22, 25),

===== Divide & Conquer Testing for 5 Clusters =====
Cluster 1: (2, 3), (3, 2), (4, 5), (5, 4), (4, 4), (5, 5), (6, 3), (7, 2),
Cluster 2: (1, 6), (3, 7),
Cluster 3: (11, 12), (12, 11), (13, 14), (14, 13), (15, 12), (10, 15), (12, 16), (13, 13), (14, 14),
Cluster 4: (16, 11),
Cluster 5: (20, 21), (21, 20), (22, 23), (23, 23), (23, 22), (24, 24), (24, 21), (25, 22), (20, 24), (22, 25),
```

Figure 50

```

===== Iterative Improvement Testing for 2 Clusters =====
Cluster 1: (11, 12), (12, 11), (13, 14), (14, 13), (15, 12), (18, 15), (16, 11), (12, 16), (13, 13), (14, 14), (20, 21), (21, 20), (22, 23), (23, 22), (24, 21),
(20, 24), (25, 22), (22, 25), (23, 23), (24, 24),
Cluster 2: (2, 3), (3, 2), (4, 5), (5, 4), (6, 3), (1, 6), (7, 2), (3, 7), (4, 4), (5, 5),

===== Iterative Improvement Testing for 3 Clusters =====
Cluster 1: (4, 5), (1, 6), (3, 7),
Cluster 2: (2, 3), (3, 2), (5, 4), (6, 3), (7, 2), (4, 4), (5, 5),
Cluster 3: (11, 12), (12, 11), (13, 14), (14, 13), (15, 12), (18, 15), (16, 11), (12, 16), (13, 13), (14, 14), (20, 21), (21, 20), (22, 23), (23, 22), (24, 21),
(20, 24), (25, 22), (22, 25), (23, 23), (24, 24),

===== Iterative Improvement Testing for 4 Clusters =====
Cluster 1: (20, 21), (21, 20), (20, 24),
Cluster 2: (22, 23), (23, 22), (24, 21), (25, 22), (22, 25), (23, 23), (24, 24),
Cluster 3: (11, 12), (12, 11), (13, 14), (14, 13), (15, 12), (18, 15), (16, 11), (12, 16), (13, 13), (14, 14),
Cluster 4: (2, 3), (3, 2), (4, 5), (5, 4), (6, 3), (1, 6), (7, 2), (3, 7), (4, 4), (5, 5),

===== Iterative Improvement Testing for 5 Clusters =====
Cluster 1: (5, 4), (6, 3), (7, 2), (5, 5),
Cluster 2: (2, 3), (3, 2), (4, 5), (1, 6), (3, 7), (4, 4),
Cluster 3: (20, 21), (21, 20), (22, 23), (23, 22), (24, 21), (25, 22), (23, 23),
Cluster 4: (11, 12), (12, 11), (13, 14), (14, 13), (15, 12), (18, 15), (16, 11), (12, 16), (13, 13), (14, 14),
Cluster 5: (20, 24), (22, 25), (24, 24),

```

Figure 51

In the iterative improvement implementation, A significant drawback of selecting initial centroids randomly is the algorithm's extreme sensitivity to these starting positions, which often leads to local optima where the final clusters fail to represent the true underlying patterns of the data.

This randomness can cause "crowding," where multiple centroids begin in the same dense region, or "isolation," where a centroid is placed so far from data points that it becomes orphaned or produces suboptimal, unbalanced partitions.

Other methods to overcome these weaknesses:

- K-means++: is the industry-standard alternative; it initializes centroids by selecting points that are mathematically distant from one another, significantly improving the speed of convergence and the quality of the final clustering.
- Bradley-Fayyad-Reina (BFR) approach: which utilizes statistical summaries to handle very large datasets.
- The CURE algorithm, which uses multiple representative points instead of a single centroid to better accommodate clusters with non-spherical or irregular shapes.

9.7 Complexity analysis

9.7.1 Brute Force

9.7.1.1 Time complexity

The core of this code is generating every possible way to partition n points into k non-empty sets.

generateAllCombinations: This function generates partitions related to Stirling Numbers of the Second Kind.

- The number of ways to partition n elements into k non-empty subsets grows nearly exponentially.
- The exact number of iterations is roughly $O\left(\frac{k^n}{k!}\right)$, for which **we can ignore the factorial term** because the exponential is way higher.

$$\text{Iterations} \approx \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n \approx O(k^n / k!)$$

Figure 52

calculatePartitionScore: For every single partition generated, the code calculates a score:

- Iterate points: $O(n)$ to assign points to clusters.
- Compute Centroids: $O(n)$ total across all clusters.
- Calculation: $O(n)$ to sum distances from points to centroids.

Total per partition: $O(n)$.

Total Time Complexity:

In Big O notation, this is roughly $O(nk^n)$. This is why brute-forcing clusters is only feasible for very small datasets (usually $n < 15$).

9.7.1.2 Space complexity

allPartitions Storage: Stores every valid assignment in a `vector<vector<int>>`.

Recursion Stack: The depth is $O(n)$, which is negligible compared to the storage of the partitions.

Total Space: $O(nk^n)$.

9.7.2 Divide and Conquer

9.7.2.1 Time complexity

1. Complexity of mergeClusters

This function is the bottleneck. It repeatedly merges the two closest clusters until the count reaches targetCount.

Outer While Loop: Runs $m-k$ times (where m is the initial number of clusters). In the merge step of the recursion, $m=2k$, so it runs roughly k times.

Nested For Loops: To find the minDistance, it compares every pair of clusters. If there are m clusters, this takes $O(m^2)$ comparisons.

Distance Calculation: `clusterDistance` calculates centroids. This depends on the number of points in the clusters, which can be up to N .

Vector Operations: `clusters.erase()` is $O(m)$ because it shifts elements.

Complexity for one merge call: Since m is proportional to k during the recursion, this function behaves as $O(k^3 + N)$.

2. Complexity of divideAndConquerClusters

Recurrence Relation:

$$T(N) = 2T\left(\frac{N}{2}\right) + \text{mergeClusters}(2k, k)$$

Base Case: When $N \leq k$, it returns N clusters (each point is its own cluster).

Recursive Work: The "Divide" step takes $O(N)$ to copy points into `leftPoints` and `rightPoints`.

Combining Step: The `mergeClusters` function is called at every level of the tree.

3. Overall Time Complexity

Because the `mergeClusters` step is applied at each level of the recursion and its complexity depends heavily on k (which is constant relative to N in most calls), the overall complexity is:

$$O(N \log N + \text{levels} \times k^3)$$

However, if k is small, the $O(N \log N)$ dominates due to the slicing and copying of vectors at each level. If k is large, the $O(k^3)$ merges at each level become the primary cost.

9.7.2.2 Space complexity

The space complexity is dominated by the recursion stack and the creation of new vectors:

Stack Depth: $O(\log N)$ due to the halving of points.

Vector Storage: At each level, you are creating new `vector<Point>` objects. Since each point exists in only one branch of the tree at a time, the storage for points across one level is $O(N)$.

Total Space: $O(N \log N)$ in the worst case due to the way vectors are copied and passed in this specific C++ implementation. (Note: Passing by reference or using `std::span` could reduce this to $O(N)$).

9.7.3 Iterative Improvement

9.7.3.1 Time complexity

The time complexity is sensitive to the number of points (n), the number of clusters (k), and the number of iterations required to reach convergence (I).

A. Initialization

- Generating indices and shuffling: $O(n)$ using `random_shuffle` (or `std::shuffle`).
- Selecting the first k points: $O(k)$.

Total: $O(n)$.

B. Assignment Phase (`assignPointsToNearestCluster`)

- For each of the n points, the algorithm calculates the distance to all k centroids.

Total per iteration: $O(n \cdot k)$.

C. Update Phase (`computeCentroid`)

- The algorithm iterates through all clusters and their constituent points to calculate the new mean.
- Since every point belongs to exactly one cluster, the total work across all clusters is $O(n)$.

Total per iteration: $O(n)$.

D. Overall Time Complexity

The total complexity is expressed as:

$$O(I \times n \times k) \approx O(n^3)$$

- n : Number of points (30 in the example).

- k: Number of clusters.
- l: Number of iterations until convergence.

9.7.3.2 Space complexity

The algorithm is highly memory efficient as it does not need to store all possible combinations.

- Point Storage: The original points vector: $O(n)$.
- Cluster Storage: The clusters vector (a partition of points): $O(n)$.
- Centroid Storage: A small vector of k points: $O(k)$.

Total Space: $O(n + k) \approx O(n)$

9.8 Technique comparison

This section provides a comparative analysis of the three implemented clustering strategies: Divide & Conquer, Brute Force (Exhaustive), and Iterative Improvement (K-Means). Each approach offers distinct trade-offs between computational speed, memory usage, and result quality.

9.8.1 Comparative Summary Table

Table 47

Feature	Divide & Conquer	Brute Force	Iterative Improvement
Time Complexity	$O(N \log N + \text{levels} \times k^3)$	$O(n \cdot kn)$	$O(l \cdot n \cdot k)$
Space Complexity	$O(N \log N)$	$O(n \cdot kn)$	$O(n+k)$

Optimality	Sub-optimal	Global Optimum	Local Optimum
Scalability	High	Very Low	Very High
Best Use Case	Small to medium datasets with hierarchical structures.	Establishing a "Gold Standard" for very small n (<15).	General purpose, large-scale data mining.

9.8.2 Deep dive analysis

The Brute Force approach is the only method guaranteed to find the absolute best clustering solution. However, due to its combinatorial nature (Stirling numbers of the second kind), the search space grows at a "super-exponential" rate, making it a theoretical tool rather than a practical one for real-world data.

K-Means is the most scalable of the three. By starting with k random centroids and refining them, it avoids the exponential search space. While it can get "stuck" in a local minimum (a solution that is better than its neighbors but not the best possible), its space requirement and fast convergence make it the industry standard for large datasets.

The Divide-and-Conquer approach provides a middle ground. By recursively breaking the problem into smaller subsets and merging them, it builds a "tree" of clusters. It is faster than Brute Force and provides more structural information than K-Means, but the repeated centroid recalculations and vector copying make it more memory-intensive than the iterative approach.

9.9 Alternative approaches

In addition to standard hierarchical and point assignment methods, several specialized algorithms address large datasets and non-standard cluster shapes.

9.9.1 BFR (Bradley-Fayyad-Reina) Algorithm

BFR is a variant of the k-means algorithm designed specifically for very large, disk-resident datasets.

- **Assumptions:** It assumes clusters are normally distributed around a centroid in Euclidean space, though standard deviations may vary across different dimensions.
- **Data Processing:** Data is processed one main-memory-load at a time, with points categorized into three classes:
 - **Discard Set (DS):** Points close enough to a cluster centroid to be summarized by simple statistics (N, SUM, and SUMSQ).
 - **Compression Set (CS):** Groups of points that are close to each other but not to any existing cluster centroid.
 - **Retained Set (RS):** Isolated points that cannot be summarized.
- **Efficiency:** Summarizing sets using N, SUM, and SUMSQ allows any number of points to be represented by $2d+1$ values, where d is the number of dimensions.

9.9.2 CURE (Clustering Using REpresentatives) Algorithm

CURE addresses limitations in BFR and standard k-means, which often struggle with non-spherical cluster shapes or clusters at varying angles.

- **Cluster Shape:** Unlike centroid-based methods, CURE allows clusters to assume any shape in Euclidean space.
- **Representative Points:** Instead of a single centroid, each cluster is represented by a set of dispersed points moved approximately 20% toward the cluster's center.
- **Process:**
 1. Pick a random sample of points for initial hierarchical clustering.
 2. Select a sample of widely dispersed points for each resulting cluster.

3. Move these points toward the cluster centroid to create representatives.
4. Assign all remaining points in the dataset to the cluster with the nearest representative point.

9.10 Summary

- For Accuracy: Use Brute Force only for validation and testing on datasets where $n \leq 15$.
- For Speed and Scale: Use Iterative Improvement for any dataset exceeding 50 points or for applications requiring real-time results.
- For Relationship Mapping: Use Divide & Conquer if the goal is to understand how smaller clusters naturally merge into larger ones (hierarchical relationships).

10. Coin Flipping

10.1 Problem description

Given a line of n coins. Each coin shows either Heads 'H' or Tails 'T' in an arbitrary order. The goal is to remove all coins using a sequence of moves. A move consists of:

1. Choose a coin that is currently Heads-up.
2. Remove that coin from the line.
3. If the removed coin had an immediate neighbor on the left or right that is still present, flip that neighbor.

A gap is formed when a coin is removed. Neighbors are the coins immediately beside the removed coin, and you can't flip past a gap. The puzzle's goal is to remove all coins on the table and output the sequence of moves done to reach the goal or output impossible if there is no possible sequence.

10.2 Assumptions

- The input is a string consisting only of the characters 'H' and 'T', with length $n \geq 1$.
- Coin indices are 1-based in the output, matching the natural counting from left to right.
- Flipping happens only on the immediate original neighbors that are still present
- If a neighbor does not exist (edge coins) or has already been removed, no flip occurs for that side.
- The program must output one valid sequence if one exists; if multiple sequences exist, any is acceptable.
- The program outputs "Impossible" if no sequence can empty the board.

10.3 Algorithm Design

Backtracking (Brute Force with pruning) is chosen as it tries every legal move sequence, undoing choices when a dead end is reached. This makes sure that if a solution exists it will be found.

10.3.1 Step-by-step Execution:

1. Base Case: If `coins_left == 0`, all coins have been removed, success.
2. Iterate: For every index `i` from 0 to `n-1`:
3. Check if coin `i` is still on the board (`! removed[i]`) and shows 'H'.
4. Make a move:
5. Mark `removed[i] = true`, record the 1-based index `i+1` in path.
6. Flip the left neighbor `i-1` if it exists and is still present.
7. Flip the right neighbor `i+1` if it exists and is still present.
8. Recurse: Call solve with `coins_left - 1`. If it returns true it means a solution is found so it returns true to the previous call.
9. Backtrack if failure:
10. Undo the flips in the exact reverse order (right neighbor then left neighbor) to restore the state.
11. Mark `removed[i] = false` and remove the index from path.
12. After loop: If no 'H' coin is available and `coins_left > 0`, return false.

10.3.2 Pseudocode:

Algorithm: Coin Removal Backtracking

```
function Solve(board, removed, path, coins_left)
  if coins_left = 0 then
    return true    // all coins successfully removed

  for each position i from 1 to n (1-based) do
    if coin i is still present and shows Heads then
      // choose coin i as the next move
      mark coin i as removed
      add i to the end of path

      if coin i has a left or right neighbor that is still present then
        flip those neighbors over

      // try to continue from the new state
      if Solve (board, removed, path, coins_left – 1) then
        return true

      // undo the move (backtrack)
      if coin i has a right neighbor that is still present then
        Turn that neighbor over
      if coin i has a left neighbor that is still present then
        turn that neighbor over
      mark coin i as present again
      remove the last index from path

  return false // no move possible from this state

function Main()
  for each test_case in test_cases do
    path = empty list
    removed = array of false
    if Solve(test_case, removed, path, length of test_case) then
      print elements of path separated by spaces
    else
      print "Impossible"
```

10.3.3 Code Implementation:

```
int main()
{
    vector<string> test_cases = {"H", "T", "HHT", "HHH", "HHHH", "HTH", "TTTT", "THHTHHH"};
    for (size_t i = 0; i < test_cases.size(); i++)
    {
        string s = test_cases[i];
        vector<int> path;
        vector<bool> removed(s.length(), false);
        if (solve(s, path, removed, s.length()))
        {
            for (size_t i = 0; i < path.size(); i++)
            {
                cout << path[i] << (i == path.size() - 1 ? "" : ":");
            }
            cout << "\n";
        }
        else
        {
            cout << "Impossible\n";
        }
    }
    return 0;
}
```

Figure 53

Figure 54 Main function

```

0
7 bool solve(string &state, vector<int> &path, vector<bool> &removed, int coins_left)
8 {
9     if (coins_left == 0)
10         return true;
11
12     int n = state.length();
13     for (int i = 0; i < n; i++)
14     {
15         if (!removed[i] && state[i] == 'H')
16         {
17             // Remove the coin at position i
18             removed[i] = true;
19             path.push_back(i + 1);
20
21             // Flip immediate left neighbor if it still exists
22             if (i - 1 >= 0 && !removed[i - 1])
23             {
24                 state[i - 1] = (state[i - 1] == 'H') ? 'T' : 'H';
25             }
26
27             // Flip immediate right neighbor if it still exists
28             if (i + 1 < n && !removed[i + 1])
29             {
30                 state[i + 1] = (state[i + 1] == 'H') ? 'T' : 'H';
31             }
32
33             // Recurse
34             if (solve(state, path, removed, coins_left - 1))
35                 return true;
36
37             // Backtrack
38             if (i + 1 < n && !removed[i + 1])
39             {
40                 state[i + 1] = (state[i + 1] == 'H') ? 'T' : 'H';
41             }
42             if (i - 1 >= 0 && !removed[i - 1])
43             {
44                 state[i - 1] = (state[i - 1] == 'H') ? 'T' : 'H';
45             }
46             removed[i] = false;
47             path.pop_back();
48         }
49     }
50     return false;
51 }
52

```

Figure 55

Figure 56 Backtracking function

10.4 Correctness and case coverage

Correctness:

Before removing coin i , the code verifies $!removed[i]$ and $state[i] == 'H'$. Then flips its neighbors ' $i-1$ ' and ' $i+1$ ' but making sure that those positions are not empty or out of bounds first. The undo block re-flips the same two positions in the same conditional order, and since flipping is its own inverse, the state is restored exactly.

The algorithm iterates over every index from 0 to $n-1$ at each recursion level and attempts a move on every coin that is present and heads-up. A branch returns false only when the inner loop completes without finding any valid move while $coins_left > 0$. At that point no 'H' coin is present, which means no legal move exists in that state, so, no valid branch is ever skipped. If a solution exists anywhere in the search tree, the algorithm reaches it.

Each recursion level decrements $coins_left$ by exactly one. The depth is therefore bounded by ' n ', and at each level a finite loop runs. The algorithm always terminates.

Case Coverage:

Single coin (H/T): If it's 'H' it gets removed immediately with no neighbors to flip, done in one move. If it's 'T' the algorithm has nothing to work with and returns Impossible straight away.

All heads: Solvability depends on whether n is odd or even. When n is odd, a valid sequence always exists. When n is even, the puzzle is always Impossible. This parity rule arises because each removal flips one or two neighbors, and with an even-length all-heads line the flipping pattern always leaves at least one stranded T coin with no path to recovery.

All tails: Nothing to remove from the start. Returns Impossible without doing any work.

10.5 Complexity Analysis

Time Complexity:

In the worst case when the puzzle has no solution the code must try every sequence of heads-up removals. At the first call, there can be up to n heads, after we remove one, up to $n-1$ remain, and so on. That gives a search tree of size $O(n!)$. Fortunately, most configurations don't stay that dense; many branches hit a dead end early because the board quickly turns all tails.

Space Complexity:

The recursion depth is at most n , because only one coin is removed per level. Each level only needs a few local variables plus our removed array and path list. So, the space usage is $O(n)$.

10.6 Algorithms Comparative Analysis

Brute Force: A pure brute-force approach would generate all $n!$ orderings of coins and simulate each one. This works but is wasteful as it explores sequences that contain moves on 'T' coins or already-removed coins, which are invalid and can be discarded immediately. The backtracking approach makes the search space bounded, and strict un-doing of local flips guarantees isolation between recursive states.

Greedy: A greedy approach picks one H coin at each step according to a fixed rule say, always the leftmost, or the one whose removal creates the newest 'H' coins. This runs in $O(n^2)$ and is simple to implement.

The problem is that greedy is not correct in general. Any greedy ordering gets stuck because every path leads to a configuration with no 'H' coins but coins remaining. The greedy method cannot backtrack, so it cannot discover this. Only exhaustive search reveals that the configuration is impossible.

Divide and Conquer: Removing a coin creates a gap that splits the remaining coins into two independent subgames because the new gap prevents any future flip from crossing it. The left and right halves do not interact at all, a valid sequence in one cannot invalidate a sequence in the other. However, the two sequences produced independently must then be merged in order into a single chronological ordering. Finding a correct merging across potentially many split points is highly complex and is what makes divide-and-conquer an impractical compared to backtracking.

Iterative Improvement: It starts with some solution and refines it. This problem has no natural notion of a partial or approximate solution.

Dynamic Programming: Could work in theory since states can be cached, but maintaining the sequence order across subproblems makes it more complex to implement than backtracking.

Table 48 Algorithmic Comparison.

Table 49

Technique	Applicability	Advantages/Disadvantages
Brute Force (full enumeration)	Generate all $n!$ orderings and simulate each.	Simple to implement but so slow $O(n!)$.
Backtracking	Explore moves, undo when stuck.	Always correct and far smaller search tree than naive enumeration but still worst-case exponential.
Greedy	Always pick a specific 'H' Like leftmost first	Very fast ($O(n^2)$) but incorrect for many inputs because a wrong local choice can block the solution.
Divide & Conquer	Independent subgames are created after each removal, but merging the resulting sequences into one valid chronological ordering is complex	Correct in theory but harder to implement than backtracking.
Dynamic Programming	Use subproblem solvability on intervals with boundary conditions	Could reduce redundant work by caching already-explored states, avoiding re-exploration of the same configuration reached via different orderings. Still exponential in the worst case but significantly faster in practice, but sequence reconstruction across independent chunks is too complex
Iterative Improvement	Start with a solution and refine it.	there's no way to take a partial solution and refine it.

Conclusion: Backtracking balances between correctness and implementation complexity for this task. The factorial worst-case time is acceptable for small n , and the algorithm is simple to understand, debug, and prove correct.

10.7 Sample outputs

Tests:

```
vector<string> test_cases = {"H", "T", "HHT", "HHH", "HHHH", "HTH", "TTTT", "THHTHHH"};
```

Figure 57 Test cases

Output:

```
● 1
Impossible
Impossible
1 3 2
Impossible
Impossible
Impossible
2 1 5 4 3 7 6
```

Figure 58 Console outputs

Explanation:

- “H”: Only one coin, heads-up, removed immediately. Sequence: 1.
- “T”: No heads-up coin, Impossible.
- “HHT”: Unsolvble. Removing coin 1 leaves - T T. Removing coin 2 leaves T - T
- “HHH”: Works; sequence 1 3 2 is printed.
- “HHHH”: Unsolvble. Removing coin 1 leaves - T H H. The only valid next move is coin 3, which leaves - T - T. starting with any other coin also gives only 'T' configurations. The algorithm exhausts all paths and returns Impossible.
- “HTH”: Unsolvble. Removing coin 1 leaves - H H. Removing coin 3 leaves H H -. Removing the middle coin 2 leaves T - T.
- “TTTT”: No head so it's Impossible.
- “THHTHHH” Backtracking finds a valid removal sequence: 2 1 5 4 3 7 6

Steps:

- Remove coin 2, flips 1 and 3, state: H - T T H H H
- Remove coin 1, state: - - T H H H
- Remove coin 5, flips 4 and 6, state: - - T H - T H
- Remove coin 4, flips 3, state: - - H - - T H
- Remove coin 3, state: - - - - T H

- Remove coin 7, flips 6, state: - - - - - H -
- Remove coin 6, state: - - - - - - -

10.8 Approach downsides

The algorithm's factorial worst-case complexity renders it impractical for large inputs. Because it lacks state memoization, the algorithm fails to recognize previously encountered board configurations reached via different move sequences. Consequently, it redundantly recomputes identical sub-problems, leading to exhaustive, inefficient searches, particularly in unsolvable cases where every possible sequence must be evaluated before termination.

10.9 Improvements

A Dynamic Programming approach could reduce the redundant work by caching the states that have already been explored. Since the full state of the coins at any point is which coins are still present and what face each one shows, it can be represented compactly and stored. If the algorithm reaches a state which has been seen before and already determined unsolvable, it can skip it immediately instead of re-exploring it. This would not change the worst case in theory but in practice would cut out a significant amount of repeated work, especially on larger inputs where the number of possible orderings explodes quickly.

10.10 Conclusion

The backtracking implementation correctly finds a valid solution sequence when one exists. The main limitation is the factorial worst-case time, which becomes a real problem as n grows since impossible configurations force the algorithm to exhaust every possible branch before concluding. A Dynamic Programming approach could improve this by caching already explored states and avoiding redundant work, since the same board configuration can be reached through different removal orderings.

11. Research Conclusions

This research project has provided a rigorous exploration of ten diverse computational challenges, ranging from geometric puzzles like Tromino Tiling and Lattice Crossing to optimization problems such as Job Scheduling and Minimum Cut. By applying multiple algorithmic paradigms, including Divide-and-Conquer, Dynamic Programming, Greedy algorithms, Brute Force, and Iterative Improvement, this study has highlighted the critical trade-offs between computational optimality and practical scalability.

A recurring theme across all tasks is the **limitation of Brute Force** methods. While these approaches guarantee global optimality and serve as a "Gold Standard" for validating small-scale inputs, their exponential or factorial time complexity renders them impractical as problem sizes grow. This was notably observed in the Coin Flipping and Clustering tasks, where the lack of state memoization or the super-exponential growth of the search space led to computational stagnation.

In contrast, optimized paradigms provided more viable solutions:

- Divide-and-Conquer proved highly effective for problems with recursive geometric structures, such as the Tromino puzzle and the Moving Target hunt, offering systematic and efficient coverage of the search space.
- Dynamic Programming was essential for solving complex optimization problems like the Four-Peg Tower of Hanoi and Job Scheduling, demonstrating how storing subproblem results can drastically reduce redundant calculations.
- Iterative Improvement and Heuristics, such as K-Means for clustering and Multi-Start for Min Cut, emerged as the most scalable techniques for large-scale data. Although these methods may sometimes converge to local rather than global optima, their performance and memory efficiency make them the industry standard for real-world applications.

In conclusion, this project reinforces the principle that no single algorithm is universally superior. Successful software engineering requires a deep understanding of a problem's constraints and the ability to select an approach that balances accuracy with resource efficiency. The analytical frameworks developed through these ten tasks serve as a foundation for applying principled algorithmic thinking to future computational challenges.

12. References

- Aarts, E., & Lenstra, J. K. (Eds.). (2003). *Local search in combinatorial optimization*. Princeton University Press.
- Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network flows: Theory, algorithms, and applications*. Prentice Hall.
- Applegate, D., Bixby, R., Chvátal, V., & Cook, W. (2006). *The traveling salesman problem: A computational study*. Princeton University Press.
- Arora, A. (2017). *Analysis and design of algorithms* (2nd ed.). Cognella Academic Publishing.
- Bellman, R. E. (1957). *Dynamic programming*. Princeton University Press.
- Bousch, T. (2014). La quatrième tour de Hanoi. *Bulletin of the Belgian Mathematical Society*, 21(5), 895–912.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- Dasgupta, S., Papadimitriou, C., & Vazirani, U. (2006). *Algorithms*. McGraw-Hill.
<https://people.eecs.berkeley.edu/~vazirani/algorithms/chap2.pdf>
- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational geometry: Algorithms and applications* (3rd ed.). Springer.
- Gardner, M. (1978). *Aha! Insight*. W. H. Freeman.
- GeeksforGeeks. (2024). *Job sequencing problem*. <https://www.geeksforgeeks.org/job-sequencing-problem/>
- GeeksforGeeks. (2026). *Introduction to divide and conquer algorithm*.
<https://www.geeksforgeeks.org/introduction-to-divide-and-conquer-algorithm/>
- GeeksforGeeks. (n.d.). *Dynamic programming or DP*. Retrieved from
<https://www.geeksforgeeks.org/dsa/dynamic-programming/>
- Golomb, S. W. (1954). Checker boards and polyominoes. *American Mathematical Monthly*, 61(10), 675–682.
- Golomb, S. W. (1994). *Polyominoes: Puzzles, patterns, problems, and packings* (2nd ed.). Princeton University Press.
- Iacono, A. (2017, October 7). *Backtracking explained*. Medium.
<https://medium.com/@andreaiacono/backtracking-explained-7450d6ef9e1a>
- Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3rd ed.). Addison-Wesley.
- Korf, R. E. (1985). Depth-first iterative deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), 97–109.

- Leskovec, J., & Rajaraman, A. (n.d.). *Clustering algorithms* [Lecture slides]. CS345a: Data Mining, Stanford University.
- Levitin, A. (2012). *Introduction to the design and analysis of algorithms* (3rd ed.). Pearson.
- Pearl, J. (1984). *Heuristics: Intelligent search strategies for computer problem solving*. Addison-Wesley.
- Pinedo, M. L. (2016). *Scheduling: Theory, algorithms, and systems* (5th ed.). Springer.
- Preparata, F. P., & Shamos, M. I. (1985). *Computational geometry: An introduction*. Springer.
- Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- Skiena, S. S. (2008). *The algorithm design manual* (2nd ed.). Springer.
- Stewart, B. M. (1941). Solution to problem 3918. *American Mathematical Monthly*, 48(3), 217–219.
- Stoer, M., & Wagner, F. (1997). A simple min-cut algorithm. *Journal of the ACM*, 44(4), 585–591.
- Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.
- Talbi, E.-G. (2009). *Metaheuristics: From design to implementation*. Wiley.
- Wikipedia contributors. (2026, April 30). *Stirling numbers of the second kind*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Stirling_numbers_of_the_second_kind
- Wikipedia contributors. (2026, May 4). *Divide-and-conquer algorithm*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Divide-and-conquer_algorithm
- Wikipedia contributors. (n.d.). *Tromino*. Wikipedia, The Free Encyclopedia. Retrieved May 4, 2026, from <https://en.wikipedia.org/wiki/Tromino>